Instituto Superior
de Engenharia

Politécnico de Coimbra

DEPARTMENT OF SYSTEMS AND COMPUTER ENGINEERING

Decentralized Federated Learning Study Based on Distributed Ledgers

Project Report to fulfill the Master's degree in Informatics Engineering

Specialization Area of Intelligent Data Analysis

Author

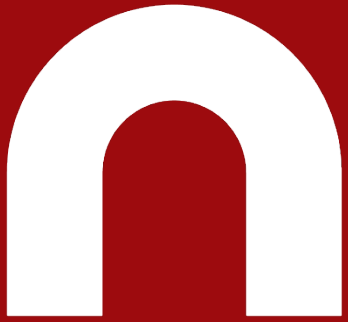
Carlos Tiago Martins Cortez Ferreira

Supervisor

João António Pereira Almeida Durães (ISEC)

Co-Supervisor

Raul Clemente Pinho Fonseca (WIT Software)



INSTITUTO POLITÉCNICO
DE COIMBRA

INSTITUTO SUPERIOR
DE ENGENHARIA
DE COIMBRA

Coimbra, April 2025

Abstract

Decentralized Federated Learning (DFL) presents a promising paradigm for collaborative machine learning without the need for centralized data aggregation, addressing growing concerns over data privacy and regulatory compliance. This study proposes and evaluates a DFL system specifically designed for audio-to-text conversion, employing a convolutional neural network as the learning model. The system integrates the Ethereum blockchain for consensus and the InterPlanetary File System for decentralized model storage, enabling deployment in environments where participants cannot be fully managed, trusted, or predetermined. The evaluation assessed not only the system's performance under safeguards against inference attacks but also its resilience in scenarios involving data imbalance across nodes and poisoning attacks. Results indicate that the DFL system maintained competitive performance compared to centralized counterparts under most conditions and demonstrated substantial resilience to both data imbalance and poisoning. However, significant performance degradation was observed when high levels of noise were introduced to enhance protection against inference attacks. Overall, this work contributes a practical implementation of a privacy-preserving, decentralized learning system for audio-to-text conversion, laying a foundation for future research and applications in secure, user-driven, and regulation-compliant distributed machine learning environments.

Keywords: Decentralized Federated Learning, Convolutional Neural Network, Audio-to-Text Conversion, Blockchain, Inference Attacks, Poisoning Attacks, Data Imbalance

Resumo

A Aprendizagem Federada Descentralizada (DFL) apresenta um paradigma promissor para a aprendizagem automática colaborativa sem a necessidade de agregação centralizada de dados, respondendo a preocupações crescentes sobre a privacidade de dados e cumprimento com regulamentações. Este estudo propõe e avalia um sistema DFL especificamente concebido para conversão de áudio em texto, utilizando uma rede neural convolucional como modelo de aprendizagem. O sistema integra a blockchain Ethereum para consenso e o InterPlanetary File System para armazenamento descentralizado de modelos, permitindo a sua utilização em ambientes onde não existe total confiança sobre os participantes e onde os mesmos não podem ser previamente determinados ou totalmente geridos. A avaliação consistiu não apenas no desempenho do sistema sob mecanismos de proteção contra ataques de inferência, mas também na sua resiliência em cenários envolvendo desequilíbrio de dados entre nós e ataques de envenenamento. Os resultados indicam que o sistema de DFL manteve um desempenho competitivo em comparação com soluções centralizadas na maioria dos cenários, demonstrando uma resiliência substancial tanto ao desequilíbrio de dados como aos ataques de envenenamento. No entanto, foi observada uma degradação significativa do desempenho quando elevados níveis de ruído foram introduzidos para reforçar a proteção contra ataques de inferência. De um modo geral, este trabalho contribui para a implementação prática de um sistema de aprendizagem descentralizado e preservador da privacidade para conversão de áudio em texto, lançando as bases para futuras investigações e aplicações em ambientes de aprendizagem distribuída segura, centrada no utilizador e em conformidade com regulamentações.

Palavras-chave: Aprendizagem Federada Descentralizada, Rede Neural Convolucional, Conversão de Áudio em Texto, Blockchain, Ataques de Envenenamento, Ataques de Inferência, Desequilíbrio de Dados

Epigraph

Throughout the centuries there were men who took first steps down new roads
armed with nothing but their own vision.
Ayn Rand

Dedication

To all those who, through personal and professional sacrifices, helped make this work possible.

Acknowledgements

This thesis would not have been possible without the support of several individuals, and I would like to express my sincere gratitude to them.

First and foremost, I want to thank my supervisors. I am grateful to Engineer Raul Fonseca for supporting this thesis and for his flexibility with my often-challenging schedule. I extend my deepest gratitude to Professor João Durães, whose guidance has been invaluable from the beginning, and whose support, interest, and flexibility have been crucial throughout this journey.

I would also like to thank all the teachers and colleagues who have accompanied me during my time at the Coimbra Institute of Engineering. I owe much of who I am today to your influence and support. In particular, I would like to express my appreciation to Rodrigo Rafael for his friendship and academic assistance.

I owe special thanks to those who have inspired and challenged me, stimulating my interest in the topics covered in this work. In this regard, I am especially grateful to Engineer Francisco Pires and Professors Francisco Pereira, Cristina Chuva, José Marinho, and João Cunha.

I am particularly thankful to Luís Silvestre and Márcio Guia for their unwavering support and motivation, whether personal, academic, or professional. Without them, this work would not have come to fruition.

I would also like to express my immense gratitude to my family, especially my parents, for their unwavering support, encouragement, and faith in my potential. Their assistance, particularly in recent years, has been crucial, and without it, this project would not have been possible.

Finally, I would like to extend my heartfelt thanks to Inês Sousa for her constant motivation throughout this process. Her encouragement was instrumental in bringing this thesis to completion.

Table of contents

Index of tables

Index of figures

List of abbreviations

ETH Ether

List of acronyms

AI	Artificial Intelligence
ANN	Artificial Neural Network
CCPA	California Consumer Privacy Act
CID	Content Identifier
CNN	Convolutional Neural Network
DAG	Directed Acyclic Graph
dApp	decentralized Application
DFL	Decentralized Federated Learning
DL	Distributed Ledger
DLT	Distributed Ledger Technology
DRL	Deep Reinforcement Learning
FedAvg	Federated Averaging
FELT	Federated Learning Token
FL	Federated Learning
FN	False Negatives
FP	False Positives
GBDT	Gradient Boosting Decision Tree
GDPR	General Data Protection Regulation
GPD	Gradient Purification Defense
IoT	Internet of Things
IPFS	InterPlanetary File System
ISEC	Coimbra Institute of Engineering
MIA	Membership Inference Attack
ML	Machine Learning
PoC	Proof of Concept
PoRep	Proof-of-Replication
PoS	Proof-of-Stake
PoST	Proof of Space-Time
PoW	Proof-of-Work
SGD	Stochastic Gradient Descent
TN	True Negatives
TP	True Positives

List of symbols

$\sum$ Summation

1 Introduction

This document describes the work that was part of a master's degree project in Computer Engineering at the Coimbra Institute of Engineering (ISEC) in partnership with WIT Software. It details the work made to investigate the feasibility of training Artificial Intelligence (AI) models in a decentralized manner using Decentralized Federated Learning (DFL) techniques. DFL consists of utilizing a Distributed Ledger (DL) to decentralize decision-making while training AI models using the Federated Learning (FL) approach. This method helps mitigate common FL threats, such as inference and poisoning attacks.

This study emphasizes the use of blockchain-based technologies, a subset of DLs, to decentralize these decision-making processes. This chapter introduces the work by detailing the associated entities in Section 1.1, the context of the work and its motivations in Section 1.2, the work contributions in Section 1.3, the execution plan in Section 1.4, and the report's structure in Section 1.5.

1.1 Associated Entities

ISEC is a public higher education institution established in 1979, located in Coimbra, a coastal city in the center of Portugal, historically known as the "city of students." It is one of Portugal's ten largest higher education institutions, comprising six teaching units that cover a wide range of training areas, from agriculture and the environment to education, communication, tourism, the arts, management, accounting and marketing, and health and engineering.

WIT Software, a company that creates products and develops projects for the telecommunications industry, inspired this project by presenting a challenge to master's degree students at ISEC. With over 20 years of solid experience, WIT Software has successfully delivered numerous projects that adhere to strict standards for high availability, security, performance, scalability, and advanced user experience. Their main areas of expertise include the fifth generation of wireless cellular technology applications, Rich Communication Services messaging, Voice over Internet Protocol and video, unified communications, television technology, mobile money, digital channels, m-commerce, and mobile security.

1.2 Context, Motivation, and Problem Identification

The quality and authenticity of the data used to train AI models directly affect their performance. However, regulations such as the European Union's General Data Protection Regulation (GDPR) [1] and the California Consumer Privacy Act (CCPA) [2] have been recently established to protect individuals' privacy rights, enhance control over personal data, and regulate how organizations collect, process, and share personal information.

These regulations have emerged partly due to growing concerns about data misuse and privacy violations. Recent technological advancements, particularly in AI's ability to replicate people's voices and generate realistic fake images and videos resembling real individuals, have increased the risk of data-related crimes, thereby corroborating these concerns.

The misalignment between data privacy and the use of personal information is particularly significant when considering elements such as voicemails and private call records. For instance, voice-based data can be transcribed into text using speech-to-text models. This textual information can then be analyzed to extract insights about user preferences, behaviors, and interests, which can be utilized to improve targeted advertising systems. However, effectively training AI models typically requires access to extensive datasets, which raises serious privacy concerns when the data originates from sensitive personal communications. Balancing the protection of data privacy with the benefits of AI technology presents a significant challenge in today's real-world scenarios.

WIT Software was interested in having the know-how on technologies that could solve this challenge, as they faced it in an internal project. To this end, the company challenged master's students at ISEC to investigate possible solutions, particularly through the use of blockchain-based technologies, an area that they were also interested in gaining more expertise in, and which can be used to decentralize systems.

In this regard, a possible strategy for training AI without direct access to the data being trained is to utilize the FL training method [3]. FL is a decentralized Machine Learning (ML) approach that allows multiple devices or servers to collaboratively train a model without sharing raw data, thereby enhancing privacy and security. Instead of sending data to a central server, local models are trained on individual devices, and only aggregated updates, such as model weights, are shared to reduce the risk of exposing sensitive data.

However, this strategy is currently more used in closed environments or where there is tight control over the participants of a network, as in Google products [4], because it is crucial to ensure that all network nodes are honest and reliable. Otherwise, there may be tampering in the global model, whether intentionally or accidentally. Such tampering can lead to what are known as poisoning attacks. The WIT Software's in-

ternal project was prone to these attacks because it did not operate in a completely closed environment and lacked control over the messaging system. This lack of control prevented them from identifying which network participants might have misled the model.

Furthermore, another type of attack in FL systems is known as inference attacks. This type of attack involves data mining techniques that analyze data to acquire information about a subject or database illegitimately. If successful, the attacker can extract data from the user by reverse-engineering the AI model used in the FL system. Both poisoning attacks and inference attacks will be examined in greater detail later in this work.

To prevent these attacks, WIT was interested in exploring the application of DLs in several of its AI projects. This is the case because one significant property of DLs, such as blockchain, is their ability to create distributed and immutable networks. These networks can execute Turing-complete programs known as smart contracts. In a DL network, transactions are secured without relying on trust among all participants, as the network members continuously validate transactions internally. As long as the majority of the network is not malicious, DLs also ensure that participants cannot undermine the network's integrity. This characteristic helps to safeguard the network from harmful or unreliable minorities.

Because the main gap in FL networks is one of the problems that DL networks can address, training methods that combine DL and AI technologies have recently been developed, known as DFL. DFLs aim to achieve the best of both worlds, creating FL systems that don't need to rely on almost all participants in their network to achieve relevant results.

With this project, we aspired to achieve two main objectives. First, we sought to develop a potential solution using DFL for training AI models on data that cannot be transferred off the data owners' devices or made public due to privacy and legal concerns. This solution needed to operate as a trustless system, as participants may not always be reliable, and it should prioritize energy efficiency. Second, we aimed to evaluate the impact of our proposed solution on the feasibility and efficiency of an AI model trained on data similar to that expected in WIT Software's internal project. The purpose of these objectives was to evaluate the practical viability of this technology.

As such, we defined the following research questions for the project:

1. Is it feasible to implement a robust and resilient audio-to-text DFL system in an environment where the network participants cannot be managed, trusted, or predetermined?
2. What is the impact of employing methods to defend against inference attacks on the performance of such an audio-to-text DFL system?

3. What is the impact of employing methods to defend against poisoning attacks on the performance of such an audio-to-text DFL system?
4. How does the performance and resiliency of an audio-to-text DFL system compare to that of a centralized counterpart, including when methods to defend against inference and poisoning attacks are employed?

1.3 Work Contributions

In this project, we evaluate the feasibility of a DFL system based on the scenario proposed by WIT Software. We compare the advantages and disadvantages of this DFL approach with more centralized training methods. Additionally, we evaluate the performance of the proposed system when implementing strategies to defend against inference and poisoning attacks. Finally, we compare all results with those from more traditional AI solutions, including a centralized AI model and a standard FL system trained on the same dataset.

1.4 Execution Plan

The project was organized into three main phases: the exploratory phase, the implementation phase, and the evaluation phase. In the exploratory phase, we concentrated on identifying a solution that would address the challenges presented by WIT Software. We researched various DFL techniques used in other studies and proposed a software architecture along with the appropriate technologies to tackle these issues.

The implementation phase focused on executing and deploying the DFL, utilizing the software architectures and technologies studied in the first phase to develop a Proof of Concept (PoC). During this phase, we also built the necessary features to evaluate the system's resilience against inference and poisoning attacks.

During the evaluation phase, we conducted the necessary tests and assessed the results. We also compared these findings with existing alternatives, including a centralized AI system and a standard FL system. This comparison considered not only the resilience of each model to attacks but also its overall effectiveness.

1.5 Report Structure

This report consists of seven chapters, followed by bibliographical references and annexes. The chapters are organized as follows:

- Chapter 1: "Introduction" - Provides context and introduces the problems that inspired this project. It also analyzes the expected contributions;

- Chapter 2: "Fundamental Concepts" - Describes and explains the main concepts needed to understand this project;
- Chapter 3: "Related Work" - Reviews existing studies and methodologies that are closely related to the approaches and objectives of this thesis;
- Chapter 4: "Approach and Architecture" - Describes the key decisions made regarding the architecture design and foundational technologies used in the implemented DFL system;
- Chapter 5: "Implementation" - Outlines the decisions made regarding the implementation of this project;
- Chapter 6: "Case Studies and Results Analysis" - Presents the case studies conducted in this research, outlining their rationale and the results obtained;
- Chapter 7: "Conclusion" - A final analysis of this work is made, as well as a forecast of its future development.

2 Fundamental Concepts

This chapter introduces key concepts and technologies that are essential for understanding this work. It analyzes the challenges and potential solutions associated with each technology, specifically in Sections 2.1, 2.2, 2.3, and 2.4, which focus on AI and ML, centralized AI, decentralized AI and FL, and Distributed Ledger Technology (DLT), respectively.

The chapter then explores the relationship between federated learning and DLT. It highlights the significant challenges faced by FL, which motivates the adoption of decentralized technologies like DLT. This relationship is examined in Section 2.5. In the same section, security threats related to this relationship are addressed, along with strategies designed to mitigate these threats.

2.1 Artificial Intelligence and Machine Learning

AI refers to the creation of computer systems that can perform tasks usually requiring human intelligence [5], [6]. These tasks include recognizing patterns, solving problems, and making decisions. AI systems can operate based on predefined rules or learned behaviors.

ML is a subset of AI that allows computer systems to learn from data and enhance their performance over time without explicit programming [7]. ML algorithms discern regularities in datasets, build predictive models, and make decisions or forecasts based on the data input. Common applications of ML include image and voice recognition, natural language processing, and predictive analytics.

ML employs various models, which are mathematical frameworks designed to recognize patterns, make predictions, or perform specific tasks based on data input. Initially, these models have limited accuracy and must undergo a process known as training to enhance their performance.

ML models can be trained using various approaches, depending on the nature of the data and the learning objectives. One such approach is unsupervised learning [8], [9], where the model is trained on datasets that do not include labeled outputs. Instead, the algorithm analyzes the data to identify inherent patterns, relationships, or structures without explicit guidance. The goal is to uncover hidden groupings or distributions that may not be immediately apparent.

Common applications of unsupervised learning include customer segmentation, grouping similar images, and anomaly detection. For instance, an unsupervised model analyzing consumer behavior can automatically categorize customers based on their purchase patterns, even without predefined labels.

In contrast, supervised learning involves training models on datasets that contain input data paired with their corresponding correct outputs, or labels [9]. These outputs are commonly grouped into categories known as classes, where each class represents a collection of identical output values. This approach can be employed for a range of tasks, including spam detection, medical diagnosis, predicting housing prices, and recognizing voice commands.

During supervised training, the model generates predictions based on the input data provided. These predictions are then compared to the correct outputs from the dataset, and the difference between the model's predictions and the true values is known as the prediction error.

To evaluate this error, the model uses a loss function, a mathematical expression that quantifies the inaccuracy of the predictions. A larger loss value indicates a greater discrepancy, while a smaller value suggests that the predictions are closer to the desired output.

To improve its accuracy, the model adjusts its internal parameters, known as weights, in order to reduce the loss. To determine the contribution of each weight to the prediction error, the algorithm calculates gradients, which are derivatives of the loss function that indicate the direction and magnitude of change needed for each weight to minimize the error.

These gradients are then utilized to update the model's weights through an optimization algorithm. A widely used technique is Stochastic Gradient Descent (SGD), which updates the weights incrementally using small batches of data, making the training process more efficient and scalable.

This process of making predictions, measuring errors, calculating gradients, and adjusting weights is repeated over many cycles. Through these iterations, the model gradually improves its ability to generalize and make accurate predictions on new, unseen data.

When training a model, a common challenge that can occur is overfitting [10]. Overfitting arises when a model learns not only the underlying patterns in the training data but also the noise or random fluctuations that do not generalize well to unseen data. As a result, the model may perform exceptionally well on the training dataset but poorly on new inputs. This issue typically arises when the model is too complex relative to the amount or quality of the training data. To address overfitting and enhance the model's ability to generalize, several techniques can be employed:

- **Regularization**: Involves adding a penalty term to the loss function to discourage the model from assigning excessively large weights. This technique helps prevent the model from becoming overly sensitive to small fluctuations in the training data;
- **Dropout**: A technique used in training involves temporarily deactivating random subsets of neurons in an ANN. This approach forces the model to learn redundant representations, enhancing its robustness and reducing dependence on specific neurons;
- **Early stopping**: A strategy that involves monitoring the model's performance on a validation dataset throughout the training process. Training is halted when the performance no longer improves and begins to decline. This approach helps prevent the model from overfitting to the training data at the expense of its ability to generalize effectively.

After a model has been trained, using it to make predictions on new, previously unseen input data is called inference. During this phase, the model utilizes the patterns and relationships it learned during training to generate outputs without altering its internal parameters.

To assess the effectiveness of a model in classification tasks, various evaluation metrics are used. A crucial tool in this assessment is the confusion matrix [11], which classifies prediction outcomes into four main categories:

- **True Positives** (**TP**): Cases where the model accurately predicts the positive class. For example, a patient with cancer correctly identified as having cancer;
- **True Negatives** (**TN**): Cases where the model accurately predicts the negative class. For example, a healthy patient correctly identified as not having cancer;
- **False Positives** (**FP**): Cases where the model incorrectly predicts the positive class when the true class is negative. For example, a healthy patient misdiagnosed as having cancer;
- **False Negatives** (**FN**): Cases where the model incorrectly predicts the negative class when the true class is positive. For example, a patient with cancer misdiagnosed as healthy.

A confusion matrix can be presented in two principal forms: unnormalized and normalized [12]. The unnormalized version displays the raw counts of predictions for each category, whereas the normalized version converts these counts into proportions, usually by row. This transformation enables more accurate performance comparisons across different classes, particularly when there is an imbalance among them. In a normalized confusion matrix with raw counts converted by row, each row sums to 1, highlighting how well the model performs for each actual class, regardless of the class's

frequency in the dataset.

Based on the categories of TP, TN, FP, and FN, several performance metrics can be calculated, such as [11]:

- **Accuracy** measures the proportion of total predictions that the model correctly classified. Is defined by Equation (2.1):

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{2.1}$$

- **Precision** measures the proportion of correctly predicted positive observations out of all predicted positives. Is defined by Equation (2.2):

$$Precision = \frac{TP}{TP + FP} \tag{2.2}$$

- **Recall** measures the proportion of correctly predicted positive observations out of all actual positives. Is defined by Equation (2.3):

$$Recall = \frac{TP}{TP + FN} \tag{2.3}$$

- **F1-Score** is the harmonic mean of precision and recall. It provides a single score that balances the trade-off between the two. Is defined by Equation (2.4):

$$F1\text{-}Score = 2 * \frac{Precision \times Recall}{Precision + Recall} \tag{2.4}$$

Accuracy offers a broad indication of a model's performance, but it can be misleading in situations where there is class imbalance. For instance, in a dataset where 95% of the instances belong to one class, a model that consistently predicts the majority class would still achieve 95% accuracy. However, this does not mean the model is effective at identifying the minority class.

Precision is particularly important in situations where the cost of an FP is significant. For instance, in email spam detection, mistakenly labeling a legitimate email as spam can lead to the loss of important messages. Therefore, achieving high precision is preferred to reduce the likelihood of such errors.

Conversely, recall is crucial in situations where failing to identify FN has serious consequences. For example, in medical diagnostics, overlooking a patient with a serious condition such as cancer can delay treatment and potentially lead to life-threatening outcomes. In these cases, models are designed to prioritize high recall, ensuring that as many actual positive cases as possible are detected, even if this increases the number of FN.

The F1-score is especially valuable when the dataset contains classes that are significantly more frequent than others, and when it is important to balance precision and recall. In these situations, relying solely on accuracy can be misleading, and the F1-score offers a more nuanced perspective on model performance. In multi-class classification, where the model needs to differentiate between more than two classes, F1-scores can be calculated in two main ways [11]:

- **Macro F1-score**: Calculates the F1-score for each class individually and then averages them, treating all classes equally. This method does not consider class imbalance. Is defined by Equation (2.5):

$$Macro\ F1\text{-}Score = \frac{1}{C}\sum_{i=1}^{C} \times F1_i \tag{2.5}$$

  Where:

  - C is the total number of classes
  - $F1$ is the $F1\text{-}Score$ for class i

- **Weighted F1-score**: Calculates the F1-score for each class and averages them, weighted by the number of true instances in each class. This ensures that classes with more examples have a proportional impact on the final score. Is defined by Equation (2.6):

$$Weighted\ F1\text{-}Score = \sum_{i=1}^{C} \frac{N_i}{N} \times F1_i \tag{2.6}$$

  Where:

  - C is the total number of classes
  - N_i is the number of true instances in class i
  - N is te total number of instances across all classes: $N = \sum_{i=1}^{C} \times N_i$
  - $F1$ is the $F1\text{-}Score$ for class i

These evaluation metrics, along with the loss function when used to evaluate a model's generalization performance on unseen data, offer a comprehensive understanding of a model's predictive capabilities. For effective training and evaluation, it is crucial to utilize high-quality datasets, whether structured or unstructured, that are relevant to the specific task. Datasets should be large, diverse, and representative to facilitate robust learning. Poor or inadequate datasets often result in inaccurate or biased ML models [13].

ML models can be categorized into several types, including Artificial Neural Networks (ANNs) [14], [15], decision trees [16], and support vector machines [17], [18]. ANNs are a fundamental type of ML model inspired by the biological neural networks of the

human brain. An ANN consists of interconnected units, called neurons, which are organized into layers. Each neuron processes input values by computing a weighted sum, applying an activation function, and transmitting the result to the next layer. Typically, an ANN includes an input layer, one or more hidden layers, and an output layer. During training, the model adjusts the connection weights between neurons to minimize the difference between predicted outputs and actual targets, allowing it to learn and recognize complex patterns within the data.

When conducting experiments with ANNs, the dataset used for training these models is typically divided into three subsets: the training set, the validation set, and the test set. The training set is used to adjust the network's internal parameters, allowing it to learn patterns from the data. The validation set is utilized during training to monitor the model's performance, which helps prevent overfitting. Finally, the test set is used to provide an unbiased evaluation of the network's ability to generalize to new, unseen data. This division of the dataset is essential for ensuring that ANN models are robust and reliable.

Convolutional Neural Network (CNN) is a fundamental type of ANN, particularly suited for tasks involving spatial, audio, or visual data, such as audio or image classification, object detection, and facial recognition [19]. CNNs are designed to automatically and adaptively learn spatial hierarchies of features. This is accomplished through convolutional layers that apply filters to the input data, enabling the detection of patterns such as edges, textures, and shapes.

2.2 Centralized Artificial Intelligence

Centralized AI refers to an approach where the data collection, processing, and model training are all consolidated on a single server or a centralized group of servers [20]. In this setup, data gathered from peripheral devices and systems is transmitted to that centralized infrastructure, where it is aggregated into extensive datasets. This aggregation enables the creation of extensive datasets, allowing ML algorithms to analyze information more efficiently and supporting the development of comprehensive models with improved predictive accuracy.

One primary advantage of using centralized AI, particularly in the context of model training and deployment, is the simplicity of management and data handling. Centralization streamlines data handling, allowing algorithms to access consistent and uniform datasets. This is particularly beneficial for training complex models that require significant computational resources. In addition, it simplifies maintenance, model updates, and monitoring, as all operations occur in one location, thus reducing the overhead and complexity associated with distributed systems. Furthermore, centralized systems often benefit from economies of scale, providing cost-effective computing

power and storage [21].

However, centralized AI also presents significant challenges, particularly in terms of privacy, scalability, and security. Storing large volumes of data in a single location creates vulnerabilities, including increased risks of data breaches, unauthorized access, and single points of failure. This centralization can expose sensitive data to malicious external attacks, such as cyber intrusions, as well as internal misuse by personnel [22], [23].

Privacy concerns are especially pronounced in centralized systems. Regulations like the GDPR and CCPA heighten the urgency of protecting sensitive information, imposing strict requirements on data storage and transfer. Moreover, regulatory and institutional restrictions often prevent data from being moved freely, especially across jurisdictional boundaries. This limitation complicates compliance and can pose significant barriers to effective centralized data management [23].

Centralized AI systems can also face limitations related to latency and bandwidth. Transferring large volumes of data to centralized servers introduces latency, potentially compromising the efficiency and responsiveness of real-time applications, such as autonomous driving systems [24].

Therefore, while centralized AI offers clear advantages in terms of manageability, consistency, cost efficiency, and training effectiveness, it also requires careful consideration of the associated privacy, regulatory compliance, and performance trade-offs.

2.3 Decentralized Artificial Intelligence and Federated Learning

Decentralized AI refers to a variety of computational methods that distribute intelligence across multiple nodes or devices rather than depending on centralized systems. This approach aims to improve scalability, robustness, and privacy by reducing reliance on a single point of control. Key forms of decentralized AI include swarm intelligence [25], inspired by collective behavior in nature, where simple agents interact locally to solve complex tasks, and multi-agent systems [26], where autonomous agents coordinate or compete to achieve individual or shared goals.

FL has emerged as another unique approach within the field of decentralized AI. Introduced by Google in a 2017 blog post [3], it allows multiple devices or nodes to collaboratively train a shared model while keeping their data local. Each client trains a local model using its own private dataset, ensuring that sensitive information remains on the device. After completing local training, the model updates, such as gradients or weights, are sent to a central server or coordinating entity.

These updates are then combined through a process called model aggregation to create

an improved global model. This global model is redistributed to the clients for further local training in subsequent rounds. Through iterative communication and aggregation, FL facilitates efficient learning across distributed systems while maintaining data privacy and reducing communication overhead. A diagram of an FL architecture can be found in Figure 2.1.

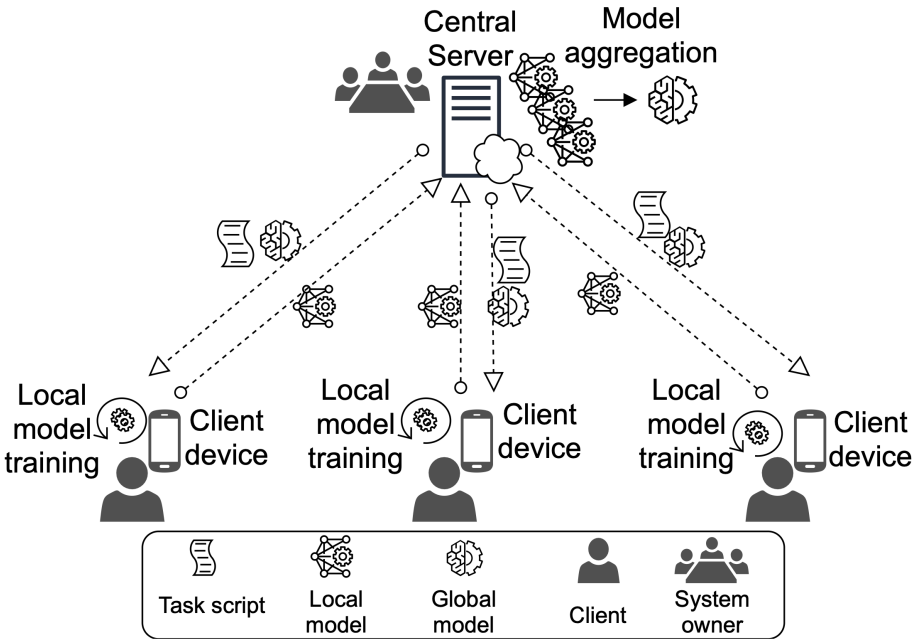


Figure 2.1: Federated Learning Architecture [27].

FL can use several algorithms to aggregate different nodes' models. Some examples of such algorithms are outlined below:

- **Federated Averaging (FedAvg)**: It utilizes weighted averaging of local models to combine updates, enhancing robustness and minimizing communication needs. Its simplicity makes it ideal for devices with limited resources [28];
- **Federated Stochastic Gradient Descent**: This algorithm sends gradients calculated on local devices to a central server, which uses them to update the global model. It is particularly sensitive to data heterogeneity [28];
- **Federated Averaging with Momentum**: In addition to the average, this algorithm incorporates a momentum term to stabilize the updates, which accelerates convergence in situations with high variability [29];
- **Federated Iterative Refinement**: This method makes iterative adjustments to address unbalanced data, prioritizing under-represented samples [30];
- **Federated Variance Correction**: Designed to address data variance, particularly when devices differ in distribution [30];
- **Federated Proximal**: It modifies the loss function to penalize divergences from

the global model, reducing the effects of heterogeneity [31].

By ensuring that data stays on the devices, this structure significantly reduces the risk of data leaks. For example, in medical applications, sensitive data such as patient histories does not need to be transferred, which helps comply with privacy regulations like the Health Insurance Portability and Accountability Act [32]. Furthermore, FL enhances scalability and improves bandwidth efficiency by enabling a wide range of devices, from smartphones to Internet of Things (IoT) sensors [33], to contribute to training. This approach distributes the computational load and helps minimize bottlenecks.

Federated Learning Challenges

Despite its advantages in privacy and decentralization, FL faces several challenges that can impact its effectiveness. Those include [34]:

- **Single Point of Failure**: In most FL implementations, a central server coordinates the training process and aggregates model updates. This centralized role, however, can create bottlenecks and introduce a single point of failure. For example, if the server suffers a hardware malfunction or is targeted by a denial-of-service attack [35], the entire learning process can come to a halt. Additionally, if the central server is compromised due to a misconfiguration in the cloud infrastructure, it may act maliciously by injecting backdoors into the global model or manipulating updates without being easily detected;
- **Scalability Issues**: As the number of participants in an FL system increases, the central server faces a significantly larger volume of model updates to manage. This scenario creates high computational and memory demands, along with increased communication overhead. For example, in an IoT deployment with thousands of smart devices, the frequent exchange of updates can overwhelm both the server and the network. This can result in delays and can drain the batteries of powered devices;
- **Heterogeneity Challenges**: Devices in an FL system may possess vastly different amounts of data or have data that follows distinct distributions. For instance, one smartphone might record thousands of user interactions each day, while another may collect only a few. When a centralized server aggregates model updates, it typically averages them, often giving equal weight to all participating devices. This can lead to a global model that is biased toward devices with more dominant or frequent data patterns while underrepresenting smaller or less common datasets.

  Additionally, device variability impacts computational performance. Low-resource devices may take longer to complete local training, causing the central

server to wait and slowing down the entire process. To avoid these delays, the server might exclude slower devices from the aggregation process. Over time, this exclusion reduces the diversity of participating devices and can result in a model that performs poorly for underrepresented user groups, ultimately affecting both fairness and generalizability;

- **Trust Assumptions**: Participants of an FL system assume that the central server will honestly perform aggregation without tampering with updates. However, when the server infrastructure is provided by a third party, such as a cloud provider, there is no guarantee of honest behavior. Additionally, a malicious actor could replace the legitimate server with a compromised version, either by exploiting vulnerabilities in the server software or by using social engineering to manipulate system administrators.

  An attacker or an administrator with control over a centralized server can alter model updates, introduce hidden functionalities like backdoors, or selectively omit updates from certain clients, which can bias the model's outcomes. This risk highlights the need for secure and verifiable aggregation mechanisms, especially in sensitive areas such as finance, healthcare, or national security.

FL also faces privacy and security vulnerabilities. While FL enhances privacy by keeping raw data on local devices, the process of exchanging gradients or model weights still poses a risk of exposing sensitive information. Additionally, there needs to be a higher degree of trust among the participants in the network, as they can jeopardize the system by sending misleading data, whether intentionally or accidentally [36].

Two major types of attacks on FL systems are poisoning attacks and inference attacks. Poisoning attacks consist of introducing malicious or biased data into the local training processes of devices [37]. When executed with harmful intent, the primary goal is to manipulate the global model that is being trained, leading to undesired behavior or compromising its functionality. These attacks can be classified into these categories:

1. **Data Poisoning Attacks**: These attacks manipulate the local training dataset to harm the global model's performance. For example, attackers might alter the data labels in a classification dataset, causing the model to learn incorrect associations;
2. **Model Poisoning**: In this type of attack, the attacker directly manipulates the gradients or weights that are sent to the central server. This can be done by adding certain noise or inserting malicious patterns into the model's weights, leading it to behave in ways that the attacker desires during specific scenarios.

A key feature of poisoning attacks is their ability to go undetected. Attackers may choose to alter only a small number of the participating devices or use adaptive strategies to avoid detection by aggregation methods [38].

Inference attacks, on the other hand, seek to extract sensitive information from the

local data of devices participating in a network by exploiting the updates they send to a central server [39], [40]. These attacks can reveal confidential information, even if the raw data itself never leaves the devices.

If an inference attacker gains access to the central server, they could potentially reconstruct user input data, such as private messages or health records, by reverse-engineering the models. Even if they cannot directly access the raw data, the central aggregation point remains a significant target for attackers looking to infer private attributes.

The main issue with inference attacks in FL systems is that they can undermine the primary goal of FL: protecting user privacy. This concern is especially significant in critical areas such as healthcare and finance. Although FL minimizes the risk of data leakage by not sharing raw data, these attacks can take advantage of indirect information like model parameters, gradients, or predictions. Inference attacks can be categorized into these principal types:

1. **Model Inversion Attack**: When an attacker uses the model's outputs to reconstruct or infer individual data points. By observing how the model behaves or updates in response to different data inputs, the attacker could reverse-engineer sensitive information about the training data [41];
2. **Membership Inference Attack** (**MIA**): When an adversary attempts to find out whether a specific data point was included in the training set. Even without direct access to the data, the attacker can do this attack by analyzing the model's responses, such as confidence levels or probabilities for particular predictions [42];
3. **Gradient Inference Attack**: When an attacker analyzes gradients to extract information about the data stored on a participant's device. Depending on how the gradients are transmitted and aggregated, an adversary might be able to infer characteristics of the local dataset, even if that dataset is never directly shared [43];
4. **Data Reconstruction Attack**: When an adversary uses the model's responses to reconstruct the training data. Even without direct access to the original data, adversaries can estimate sensitive information, particularly if they have access to multiple model parameters or updates [44], [45].

2.4 Distributed Ledger Technology

DLT is a system for recording data in which information is stored across multiple locations, rather than in a single centralized database [46], [47]. This structure usually ensures that no single entity has complete control over the entire system, thereby enhancing security, transparency, and reliability.

DLTs can be either decentralized or centralized, depending on their design and governance structure. In a decentralized DLT, no single authority controls the ledger. Instead, all participants or nodes in the network have an equal role in validating and recording data through consensus mechanisms. This decentralization promotes transparency, reduces the risk of manipulation, and fosters trust among participants.

However, some DLTs may not be entirely decentralized. In these systems, a centralized entity or a smaller group of trusted participants may have greater control over the network or the validation process. This centralization can diminish some of the advantages of decentralization, such as resilience to single points of failure [47].

When a DLT is explicitly designed to be fully decentralized, it is more accurately referred to as a DL. In this work, we used this terminology to highlight the absence of central control, emphasizing the system's focus on trustless validation by all participants and ensuring that no single individual can dominate or manipulate the ledger.

2.4.1 Distributed Ledgers

DL technologies enable the creation of decentralized and immutable networks [48]. In these networks, transactions are secured without needing to trust all participants through constant internal validation by the participants themselves. Unless the majority of the network is malicious, DLs can also ensure that other participants do not compromise the network's integrity. This trait assures that most participants safeguard the network from malicious or unreliable minorities. The key features of DLs are:

- **Transparency**: Every participant in the network has the ability to verify and audit transactions;
- **Immutability**: Once recorded, entries in the ledger cannot be altered without consensus, ensuring the integrity of the data;
- **Decentralization**: Control is shared across the network, eliminating the risk of a single point of failure and decreasing the chances of manipulation or overall system collapse.

2.4.2 Blockchain, Bitcoin, and Proof-of-Work

A blockchain is a specific type of DL that organizes data into blocks. Each block contains a list of transactions, a timestamp, and a cryptographic hash of the previous block, creating a secure chain of records. The technology was introduced to the world in 2008 by an anonymous author or group using the pseudonym Satoshi Nakamoto, through the publication of the whitepaper titled "Bitcoin: A Peer-to-Peer Electronic Cash System" [49]. Bitcoin, which started its development in 2007 and was officially launched in 2009, was also the first practical application of blockchain, serving as a DL that records

financial transactions in a public, secure, and immutable manner.

In the Bitcoin network, transactions are validated and added to the blockchain through a consensus protocol called Proof-of-Work (PoW). This protocol ensures that all participants in the network agree on the state of the blockchain and prevents double-spending, where the same Bitcoin could be spent more than once.

The protocol relies on miners, who are individuals or entities participating in this process. They use powerful computers to solve complex mathematical problems that are part of a cryptographic puzzle. Miners must solve these puzzles in order to add a new block to the blockchain. In the Bitcoin network, the targeted interval for generating a new block is approximately 10 minutes.

Once a miner successfully solves the puzzle, they create a new block containing the latest verified transactions. This block is then broadcast to the entire Bitcoin network, where other miners and nodes verify the solution. The first miner to propagate their solution—meaning they successfully broadcast the newly mined block to the network—receives a reward for that block. This reward consists of newly minted bitcoins and the transaction fees associated with the transactions included in that block.

The miner who successfully propagates the block and has it accepted by the network is the one who receives the reward. This process is competitive, with miners racing to solve the puzzle and propagate their blocks in order to claim the rewards. Over time, the reward for mining a block decreases through a process called "halving," which occurs approximately every four years. This ultimately reduces the total number of bitcoins to 21 million. This built-in scarcity contributes to Bitcoin's value and continues to incentivize miners to secure the network and process transactions. Additionally, the propagation of blocks is vital for maintaining the decentralized nature of Bitcoin, as it ensures that all participants in the network have the same, up-to-date version of the blockchain.

2.4.3 Ethereum and Smart Contracts

Bitcoin was initially developed as a decentralized digital currency to record financial transactions. However, its lack of programmability limited its functionality and scope beyond basic monetary applications. As the potential of blockchain technology for diverse decentralized systems became evident, the demand for a programmable blockchain emerged.

In response to the increasing demand for innovation within the blockchain sector, Vitalik Buterin proposed the establishment of Ethereum in 2014, with the network officially launching in 2015 [50]. Ethereum was developed to enhance the original blockchain concept by integrating smart contracts, which are self-executing programs that function on the blockchain. In contrast to Bitcoin, which primarily emphasizes peer-to-peer

transactions, Ethereum was specifically designed to enable the creation and implementation of these smart contracts.

On the Ethereum blockchain, a decentralized application (dApp) represents a single instance of a broader category of software designed to run without centralized control. Smart contracts facilitate the operation of a diverse array of dApps on the Ethereum blockchain. These dApps are capable of automating processes, enhancing security requirements, and ensuring transparency across various sectors, including decentralized finance, digital identity management, supply chain tracking, digital assets, healthcare, and voting systems [51].

2.4.4 Tokens

In the context of blockchain technology, a token is a digital unit of value that is created, managed, and transferred using smart contracts on a blockchain network. Smart contracts are self-executing programs stored on the blockchain that define the rules and logic governing how tokens are issued, transferred, and interacted with. For example, on platforms like Ethereum, tokens are implemented through standardized smart contract protocols such as ERC-20 or ERC-721 [52]. These contracts ensure that transactions involving tokens, such as sending, receiving, or checking balances, are carried out automatically and securely without the need for a central authority.

Tokens can represent a wide range of assets or functions, including currency, digital collectibles, voting rights, or access to services within dApps. Smart contracts are the foundational technology that enables their operation.

2.4.5 Ethereum and Proof-of-Stake

In its early stages, Ethereum used a PoW consensus model, similar to Bitcoin. However, the Ethereum community intended to shift to a more energy-efficient consensus mechanism. While PoW effectively maintains network security, it faces criticism due to its high energy consumption and environmental impact [53]. Acknowledging these limitations, Ethereum developers spent several years planning and developing a transition to a Proof-of-Stake (PoS) consensus model [54].

In a PoS system, new blocks are not created through energy-intensive operations. Instead, validators take the place of miners. Validators are participants in the network who lock up or stake their Ether (ETH), which is the native cryptocurrency of Ethereum, as collateral to engage in the consensus process. They are selected to propose new blocks and validate transactions based on the amount of ETH they have staked. The more ETH they hold, the higher their chances of being chosen. If a validator acts maliciously, they face the risk of losing a portion of their staked ETH through a process known as slashing.

In addition to enhancing energy efficiency, PoS presents several advantages over PoW. These benefits include lower transaction costs, quicker block times, and better scalability. Ethereum's successful transition from PoW to PoS has been a significant milestone for the Ethereum blockchain and has also initiated a wider discussion about alternative consensus protocols in the blockchain space. As blockchain technology continues to develop, various projects are exploring and implementing different consensus mechanisms to enhance the technology. These new consensus protocols present various trade-offs and benefits for decentralized networks. Examples of such protocols include delegated proof-of-stake, Practical Byzantine Fault Tolerance, proof-of-authority, and Proof of Space-Time (PoST) [55].

2.4.6 Permissionless and Permissioned Distributed Ledgers

DLs can be classified as permissionless or permissioned, depending on who can participate in the network and access its data [56]. Permissionless DLs are decentralized and open to everyone, allowing any individual or organization to join the network, validate transactions, and help maintain the blockchain. These blockchains are designed to be transparent, immutable, and trustless. This means participants do not need to depend on a central authority to verify the legitimacy of the transactions recorded on the blockchain. Public blockchains are typically permissionless, meaning users do not need to obtain permission to participate in the network or engage in operations related to consensus protocols.

In contrast, permissioned DLs are private networks, which means that access is restricted to authorized participants only. These networks are usually controlled by a central entity or a consortium of organizations. As a result, they often prioritize privacy, speed, and efficiency over decentralization. Permissioned DLs are typically used in scenarios where only trusted individuals are allowed to make consensus decisions or access the DL data.

2.4.7 Decentralized Storage

Blockchains like Bitcoin and Ethereum have limitations on block size, meaning that each block on the blockchain can only hold a fixed amount of data. For instance, Bitcoin typically allows a maximum of 1 MB of data per block [57].

These block space limitations create competition for space within each block, which can drive up the cost of adding data to the blockchain. During times of high demand, congestion occurs, leading to an increase in transaction fees. Users often have to pay higher fees to incentivize miners or validators to prioritize their transactions and include them in the next block [57], [58].

Additionally, public blockchains usually operate on a full node model, where every

participant in the network must store the entire history of the blockchain. This approach ensures that all transactions and data on the blockchain are publicly verifiable and secure. Full nodes are required to maintain a record of every transaction ever made, which can become quite large as the blockchain expands over time. For instance, Ethereum's blockchain, with its smart contract interactions, is significantly larger than Bitcoin's [59].

As full nodes must store all historical data, the increased volume of data added to the blockchain necessitates greater storage capacity from network participants, particularly miners and validators. This demand for decentralized storage raises the costs of maintaining the network and contributes to the rising expenses associated with adding data to the blockchain.

Decentralized storage solutions were developed as a natural extension of the concepts introduced by DLTs to address the challenges related to scalability, efficiency, and immutability in blockchain data storage. Several DLs, such as the InterPlanetary File System (IPFS), Filecoin, and Arweave, have emerged to address this issue [60].

InterPlanetary File System

The IPFS, introduced in 2014 by Juan Benet [61], represents a significant advancement in decentralized storage. It organizes data using a content-addressing system built on a peer-to-peer architecture. Unlike the traditional web, which locates files based on their physical location, IPFS identifies content through cryptographic hashes.

When a file is uploaded to IPFS, it is divided into blocks, each of which is hashed using a cryptographic algorithm. The unique hash of the content, known as a Content Identifier (CID), is used to reference that content, making it addressable on the network. This hash acts as a digital fingerprint of the file's content. Even a small change in the file will result in a different hash, ensuring the integrity of the content. Consequently, once files are added to the IPFS, their content cannot be altered.

This method enhances efficiency, ensures immutability, and provides resistance to censorship. IPFS has also been quickly integrated into the blockchain ecosystem, allowing it to reference external data without overwhelming its infrastructure [62], [63]. For instance, a dApp could store its media files or documents on IPFS and save the CID on the blockchain. This approach keeps the content decentralized, while its integrity can be verified through the blockchain.

Filecoin

Filecoin was proposed by Juan Benet in 2017 and launched in 2020 to extend the IPFS protocol by creating a decentralized market for file storage [64], [65]. While IPFS allows for decentralized file sharing, Filecoin adds an economic layer that incentivizes

participants to offer storage space to the network.

Filecoin operates on its own blockchain and uses the cryptocurrency FIL to create a system where storage providers are rewarded for storing data. Users pay FIL to storage providers, who earn FIL as rewards for maintaining the availability and verifiability of the data they store over time.

Filecoin employs two consensus mechanisms: Proof-of-Replication (PoRep) and PoST. PoRep ensures that the data stored by a miner is physically and verifiably replicated in a unique manner, while PoST ensures that the data is continuously stored over time and not just momentarily. Storage providers must regularly demonstrate that they are still storing the data, and they are rewarded for providing reliable and consistent storage.

Arweave

Another notable innovation is Arweave, which was proposed by Sam Williams and Will Jones in 2017 under the name Archain and launched in 2018 [66]–[68]. It offers a solution where data can be stored indefinitely for a one-time payment. The architecture is designed to cover storage costs through a combination of upfront fees and sustainable payments, considering the future cost of storage over the next 200 years. This approach facilitates the long-term preservation of information, such as historical documents and dApps.

Arweave uses its own DL called blockweave. Unlike a traditional blockchain, where each block references only one previous block, a blockweave allows each block to reference multiple previous blocks. This structure makes data retrieval faster and more efficient while ensuring redundancy in the storage of information.

Arweave also employs a unique consensus mechanism known as proof-of-access. In this protocol, a miner must prove that they have access to a specific amount of historical data. When a miner creates a new block, they must access previous blocks stored in the blockweave, necessitating that they store a portion of these previous blocks along with the new one. This requirement ensures that miners continuously contribute to data storage, enhancing the permanence and accessibility of the network.

Although Arweave's focus is on permanent storage, recently new projects have emerged to complement the DL by providing non-permanent storage solutions. These initiatives facilitate storage agreements between clients and providers. Clients outline their storage needs, including data size, duration, and redundancy requirements, and pay for the service using AR, the native cryptocurrency of Arweave. Providers then supply the necessary storage resources. The agreements are recorded on Arweave, and periodic verification challenges are conducted to ensure that providers maintain the data as agreed. One such project, ArFleet, was introduced in 2024 [69].

2.5 Decentralized Federated Learning

Blockchain technology provides unique features that can greatly enhance FL systems by improving trust, security, and coordination among participating devices. The integration of blockchain and FL has led to the development of DFL, which combines the advantages of both technologies [70]. DFL aims to leverage the strengths of DLs and FL to enable secure collaboration and deliver reliable results in FL systems while reducing reliance on any single participant or centralized control.

In a DFL architecture, devices collaborate to train a shared model while keeping their local data private, similar to FL. However, in this system, a DL acts as a distributed and immutable ledger that records model updates, eliminating the need for a centralized server. This setup guarantees transparency, traceability, and resistance to tampering. An example of a DFL architecture can be seen in Figure 2.2.

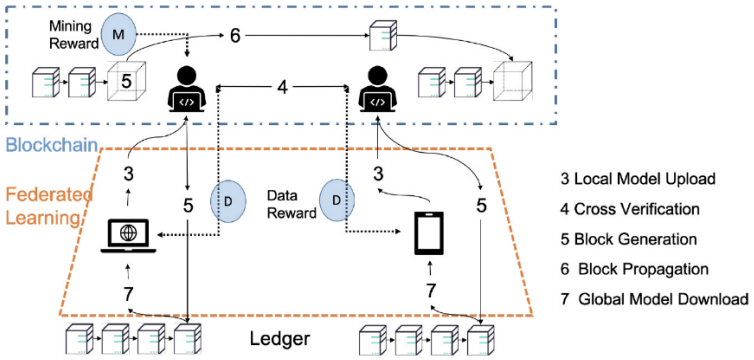


Figure 2.2: Example of a Decentralized Federated Learning Architecture [71].

DFL also enables the aggregation of local models to occur in a decentralized way. This can be achieved through consensus-based mechanisms or by rotating aggregation responsibilities among trusted nodes. This approach improves fault tolerance, security, and trustworthiness, especially in situations where a central authority is either undesirable or impractical.

Furthermore, blockchain technology can be utilized to reward participants in a DFL network. For instance, smart contracts can distribute tokens to the top contributors of the global model in a decentralized manner. Furthermore, smart contracts can also automate processes such as selecting participants to aggregate the local models and generate the global model.

2.5.1 Decentralized Federated Learning Challenges

By being decentralized, DFL systems address the challenges of single points of failure that FL faces. However, it experiences similar scalability and heterogeneity challenges compared to standard FL systems.

Regarding trust assumptions and privacy and security vulnerabilities, DFL poses unique risks compared to traditional FL systems. In many DFL architectures, model weights or updates are stored in a decentralized and often public infrastructure, such as a blockchain or IPFS. This level of openness increases the risk of inference attacks [72]. For instance, an adversary can more easily access model updates from multiple clients and attempt methods like gradient inversion or membership inference to reconstruct private training data.

Furthermore, the absence of a central server in DFL means that the system relies more on trust among the participating nodes. In a centralized FL setup, the central server coordinates and filters data, allowing it to validate updates, detect outliers, and eliminate potentially harmful contributions. However, in DFL, there is no single point of control, which requires each node to trust that others are following the protocol honestly. This lack of oversight makes the system more susceptible to poisoning attacks [73]–[75]. Once a malicious update spreads across the network, it becomes difficult to retract or neutralize, especially in systems where updates are recorded immutably, such as on a blockchain.

Therefore, DFL systems not only heighten the risk of inference attacks because model updates are publicly accessible, but they also make defending against poisoning attacks more challenging. This is due to the absence of centralized oversight and a greater dependence on the honesty of participants. This change in trust dynamics represents a fundamental trade-off when transitioning from centralized to decentralized FL architectures.

2.5.2 Strategies Against Privacy and Security Vulnerabilities

Defense strategies against DFL attacks are essential. Several approaches exist. This section outlines the protective measures against the two primary attacks identified on DFL systems: poisoning and inference attacks.

Current strategies for defending against poisoning attacks in both FL and DFL employ various approaches. The most common method involves rejecting suspicious gradients during each training iteration. This is typically achieved by analyzing the statistical properties of client updates and discarding those that deviate significantly from the majority. Such deviations may indicate malicious intent and could lead to the propagation of corrupted parameters throughout the network. However, this method can result in delays and increased communication costs. A comprehensive study on malicious gradient rejection techniques can be found in [76].

In the context of DFL, a defense strategy against poisoning attacks incorporates blockchain technology into the training process. Blockchain provides immutable logging of model updates, enforces consensus rules for aggregation, and enhances ac-

countability among participants. This approach can be combined with the method for rejecting suspicious gradients, ensuring that model updates are validated in a decentralized manner before they are applied. Examples of blockchain-based protection mechanisms for Federated Learning can be found in [75], [77]–[79].

To address inference attacks, several advanced techniques have been proposed to reduce these risks in the context of DFL. These techniques generally focus on one of three approaches: adding noise to the training data or to the model, reducing the size of the model, or encrypting either the model training data or the model itself using methods such as homomorphic encryption [39], [43], [80]–[82].

To reduce the model size in the DFL context, three methods are commonly used: pruning, quantization, and knowledge distillation [39], [43], [82], [83]. All of the techniques against inference attacks share a common goal: to make it more challenging to retrieve the original data used to train the models. Below are further explanations of the specific techniques mentioned that can be employed to protect against inference attacks.

Adding Noise

Adding controlled noise to data is an effective method for preventing information leaks in privacy-preserving ML [39], [43]. This noise is usually calibrated based on mathematical privacy guarantees, such as those provided by differential privacy. This ensures a balance between protecting privacy and maintaining model utility. There are two principal data noise techniques:

- **Differential Privacy**: This method adds calibrated noise to the data in order to mask individual contributions. It ensures that the aggregate results do not reveal specific information about any individual. Common noise distributions used in differential privacy include the Gaussian noise, which adds normally distributed noise, and the Laplacian noise, which adds noise drawn from a Laplace distribution to achieve strict mathematical privacy guarantees [84];
- **Random Noise**: This approach introduces small perturbations to the data or models, making it challenging for attackers to accurately reconstruct the original information.

It is essential to carefully adjust the noise levels. Insufficient noise may fail to protect sensitive data adequately, whereas excessive noise can adversely affect model performance.

Model Compression

Model compression reduces the structural complexity of neural networks by decreasing the number of parameters and the amount of stored information [85]. By mini-

mizing the size of the model, the risk of inference attacks is significantly lowered, as sensitive data is less likely to be present in more compact structures. Additionally, this approach improves computational efficiency, making the models more suitable for devices with limited resources, which are commonly used in DFL systems.

Pruning

Pruning is a model compression technique that consists of removing redundant parameters or connections in ANNs [86]. This process simplifies the model's structure and reduces its information density, making it harder for adversaries to extract sensitive data. Additionally, pruning enhances efficiency by decreasing both the size and storage requirements of the model.

Pruning leads to an increase in sparsity, which is the proportion of zero weights in an ANN. A higher level of sparsity means there are fewer active parameters, resulting in more compact and efficient models. However, the method used to introduce sparsity during training can significantly impact model performance and stability.

Two common methods for introducing sparsity are polynomial decay and constant sparsity [87]. The polynomial decay gradually increases the level of sparsity over time, allowing the model to adapt smoothly. This approach prunes less in the early stages and more toward the end of training. In contrast, constant sparsity maintains a fixed level of sparsity throughout the entire pruning process. While this method is simpler, it may provide less flexibility for model adaptation.

Quantization

Quantization is a model compression technique that reduces the precision of model weights and activations by converting high-precision values, such as floating-point numbers, into more compact representations, such as integers [88], [89]. There are two main approaches to quantization:

- **Quantization-Aware Training:** This method occurs during the training process. The model is adjusted to handle lower precision values from the start;
- **Post-Training Quantization:** This method is applied after the training is complete, without the need to retrain the model.

Reducing precision not only enhances computational efficiency but also helps prevent adversaries from inferring sensitive information, which is more easily obtained in high-precision formats.

Knowledge Distillation

Knowledge distillation is a technique used to transfer knowledge from a complex model, known as the teacher, to a simpler and more compact model, referred to as the student [90]. In this process, the student model is trained based on the predictions made by the teacher instead of being trained directly on the original data. This strategy has several advantages:

- It ensures that sensitive information from the training data is not directly incorporated into the final model, protecting it against inference attacks;
- It produces lightweight and secure models, making them suitable for distributed systems such as DFL.

Homomorphic Encryption

Homomorphic encryption enables operations to be carried out directly on encrypted data, ensuring that the original information remains confidential during both training and inference [91]. In the context of DFL, this technique safeguards gradients, weights, and other sensitive data, even if the encrypted information is intercepted. However, despite its strong security features, this approach still faces challenges due to its high computational cost [92].

3 Related Work

In this chapter, we review existing studies and methodologies that are closely related to the approaches and objectives of this thesis. Section 3.1 examines research on DFL architectures, Section 3.2 explores methods for decentralized data storage in the context of DFL, and Section 3.3 outlines methods for protecting DFL systems.

3.1 Decentralized Federated Learning Architectures and Algorithms

When it comes to architectures, several studies have implemented DFL architectures using blockchain technology. In works referenced as [93], [94], two DFL architectures are presented that utilize permissioned blockchains based on Ethereum. Additionally, in studies cited as [95]–[97], DFL is explored using public blockchains, all of which employ PoS protocols for consensus.

DFL architectures are designed to support various AI algorithms that can be trained without the need for centralized data aggregation. While the most common application of FL involves aggregating gradients to train ANNs, alternative methods have also been explored.

For instance, a study mentioned in [98] describes a decentralized implementation of the Gradient Boosting Decision Tree (GBDT) algorithm. GBDT is a machine learning method that constructs a robust predictive model by combining multiple simple decision trees. Each tree in the sequence is trained to enhance the model's performance by learning from the errors made by the previous trees. Instead of relying on the original data for every iteration, each new tree focuses on the specific areas where the model is underperforming. This iterative process continues step by step, gradually reducing the overall error. The study shows that, similarly to ANNs, non-neural network models can adapt effectively to decentralized learning environments.

Regarding alternative DL architectures, the one presented in [99] contains a ring network that is supposed to work in parallel with the blockchain. In a ring network, each node connects to exactly two other nodes, forming a single continuous pathway for signals through each node.

In [100], a Directed Acyclic Graph (DAG) is used to store the private models in parallel with a blockchain to store the public models. A DAG is a type of graph where the

connections between nodes have a direction. This structure ensures that once a node is traversed, it cannot be revisited, preventing the formation of cycles. In the same work, the private models are encrypted when stored in the DAG, so only the participants of the network can access them with the corresponding keys.

Also in [100] and in [101], the selection of the node that updates the global model in the blockchain is done using Deep Reinforcement Learning (DRL). DRL is a subfield of ML that allows agents to learn complex behaviors through interaction with their environment. In this context, an agent is an autonomous entity that observes its surroundings, takes actions, and learns from the outcomes of those actions. The agent's goal is to develop a strategy, or policy, that maximizes the total reward it receives over time.

Agents in DRL perceive the current state of the environment, select an action based on their policy, and then receive feedback in the form of a reward and a new state. ANNs are employed to help the agent approximate critical functions, such as the expected future reward of a given state or the optimal action to take. This is especially useful in environments with high-dimensional or unstructured inputs, such as images or raw sensor data.

For scalability in the context of IoT, an architecture with multiple permissioned blockchains that create consensus and are then aggregated into one public blockchain is presented in [102], while in [103] the train of the data is managed by fog nodes. In this context, fog nodes are computing devices situated between end-user devices, like smartphones and sensors, and centralized cloud servers. They provide storage, processing, and networking services closer to the data source, reducing the need to send all information to the cloud. By managing tasks locally, fog nodes minimize latency, improve response times, and decrease bandwidth usage, which is crucial for real-time applications such as autonomous vehicles, industrial IoT, and smart cities.

The work presented in [96] adopts a different approach by introducing an architecture that is more aligned with standard FL. In this architecture, instead of using the private data from the nodes for validation, a centralized dataset is utilized to evaluate the private models prior to their submission. This method reflects a more conventional approach, typically found in standard FL architectures.

Additionally, the same work provides token incentives to those who improve the model. However, token incentives can increase the tendency to excessively train on the same data when used alongside a centralized validation dataset, because participants may optimize specifically for short-term performance gains on the evaluation set, rather than promoting true generalization through training on their private data. This can lead to the model overfitting and performing poorly on new, unseen data.

To address this issue, the test labels are concealed using word embedding, a technique

that represents words in a continuous vector space [104]. This approach captures the semantic and syntactic relationships between words, serving to protect the integrity of the actual validation data from the nodes.

3.2 Data Storage Methods for Decentralized Federated Learning

The data from global models trained using a DFL architecture needs to be stored in a manner that allows all nodes in the network to access it. The studies referenced in [98], [102], [103], [105]–[113] utilize blockchain as the sole method for data storage. However, storing global or local models on the blockchain is only practical when the blockchain is private and specifically designed to accommodate the required use cases, or when the decentralized algorithms involve minimal data storage. This limitation arises from constraints on the amount of data that can be stored in each blockchain block, as well as the potentially high transaction costs [114], which can render the DFL architecture impractical for real-world applications.

In [93] and [94], the authors address this issue by storing data in multiple blocks on the blockchain, dividing it into chunks to avoid exceeding the data limits. Additionally, the latter study employs data serialization techniques to further compress the stored information.

Other works, like the ones in [95]–[97], [99], [115]–[117], utilize or propose the IPFS as a partial or complete storage solution. This approach is preferable to relying solely on blockchain for data storage because it enables the sharing of a greater volume of data, allowing for more flexibility in handling private models within the network, and generally at a lower cost [118]. This architecture offers various configurations: some studies store only the AI models [95], while others maintain all data encrypted in IPFS, keeping only an encrypted link on the blockchain for traceability by network nodes, as seen in [96], [97], [99], [119]. Additionally, certain works store all information directly in IPFS without utilizing blockchain at all [115]. This solution provides the ability to adjust the amount of information stored on IPFS based on the specific use case that the DFL architecture is addressing.

3.3 Protection Methods for Decentralized Federated Learning

DFL systems must be protected against inference and poisoning attacks to ensure both model integrity and data privacy. Regarding inference attacks, various strategies have been proposed in multiple DFL studies. One such approach, suggested in [102], [119]

and implemented in [102], enhances security by executing a model compression technique that only aggregates a portion of the complete gradient. As a result, only compressed data or subsets of the data are considered during the process, making it more difficult to reverse-engineer the gradients to retrieve the original data. Additionally, this strategy reduces the amount of data shared between nodes, which can help alleviate issues related to communication bandwidth.

Another solution suggested for inference attacks is the use of encryption when using the models. As shown in [80], [81], the homomorphic encryption characteristics can be utilized in the context of FL to aggregate already encrypted gradients. This solution is used in [99], [103], [106] and suggested in [117].

Instead of modifying models after training to reduce the effectiveness of inference attacks, the studies referenced in [95], [97], [98], [119] introduce noise into the data before generating local gradients. The research in [109] suggests adding differential privacy at the nodes but does not implement it. All studies, except for [109], utilize Laplacian noise as the method for injecting noise. The work in [109] proposes using Gaussian noise instead. Additionally, in [119], a committee of nodes is selected through verifiable random functions to aggregate multiple private models, which, along with the use of differential privacy, results in even greater noise.

Some strategies to defend against inference attacks used in FL systems can also be applied to DFL systems. For instance, the work presented in [83] introduces a novel algorithm that integrates quantization and pruning techniques, achieving minimal impact on model accuracy. Additionally, the research in [82] describes a novel FL approach that executes model compression on the client side while maintaining a full model on the server. The experiments conducted in the same research demonstrate that this method effectively safeguards client privacy against MIAs while still achieving competitive classification accuracies. Furthermore, the study in [39] employs multiple techniques, such as gradient encryption, adding noise to gradients, and gradient pruning, suggesting that inference attacks can be mitigated with only slight losses in data utility.

Concerning poisoning attacks in DFL networks, the study in [75] addresses poisoning attacks by introducing Gradient Purification Defense (GPD), a method that works in conjunction with DFL aggregation to minimize harmful gradients while retaining valuable contributions. GPD implements a recording mechanism that tracks historically aggregated gradients, allowing benign clients to identify and counteract malicious updates through consistent historical checks. This approach enhances model accuracy by preserving beneficial components, even from compromised clients. The paper includes experimental results demonstrating that GPD effectively mitigates poisoning attacks.

Some studies propose using blockchains to enable participants to monitor and track malicious behavior in FL networks, thereby protecting the system from poisoning attacks. These strategies can be easily integrated into fully DFL systems. The works

cited in [77]–[79] are examples that employ this approach using multiple evaluation methods. All of these studies present experimental results demonstrating that these methods can significantly improve model accuracy against poisoning attacks when compared to standard FL systems.

4 Approach and architecture

This chapter describes the key decisions made regarding the architecture design and foundational technologies used in the implemented DFL system. These choices, along with the implementation choices presented in Chapter 5, establish the groundwork for the case studies and subsequent results discussed in Chapter 6.

In Section 4.1, we outline the databases and the AI model to be used. Furthermore, we discuss the architecture for the DFL system in Section 4.2. Lastly, we employed methods to protect against poisoning and inference attacks in the DFL system, which are discussed in Sections 4.4 and 4.3, respectively.

4.1 Dataset and Artificial Intelligence Model

The goal of this project is to evaluate the feasibility, robustness, and resilience of an audio-to-text DFL system. To achieve this, we chose a dataset that could be trained in less than one day using our hardware and AI model, allowing us to validate more scenarios. The dataset should also be large enough to provide more than 30 samples for each category under standard conditions while being distributed across the maximum number of nodes in the DFL system. This would ensure that each node has enough relevant data for effective training results. We chose to use the Speech Commands v2 dataset [120] in this project because it meets these requirements. The dataset was originally presented on April 9th 2018.

This dataset was originally designed to train simple ML models capable of detecting specific words in the presence of conversations or background noise. It contains 84,848 one-second .wav audio samples for training, 9,982 for testing, and 4,890 for validation. The audio samples are categorized into 35 words: "Yes," "No," "Up," "Down," "Left," "Right," "On," "Off," "Stop," "Go," "Zero," "One," "Two," "Three," "Four," "Five," "Six," "Seven," "Eight," "Nine," "Bed," "Bird," "Cat," "Dog," "Happy," "House," "Marvin," "Sheila," "Tree," "Wow," "Backward," "Forward," "Follow," "Learn," and "Visual."

Concerning the AI model, we chose to implement a CNN with this dataset because CNNs have shown strong performance in speech-to-text translation tasks. This effectiveness is largely due to their ability to efficiently capture local temporal and spectral features in audio signals [19], [121], [122]. Furthermore, CNNs are well-documented and supported within FL systems because the training of these algorithms can be distributed across decentralized nodes [123], [124].

4.2 Decentralized Federated Learning Architecture

Given the characteristics of this project and the challenges posed by WIT Software, it was essential to select a DL that would allow different nodes to reach a consensus on aggregating their local models and generating a global model for effectively training a CNN using a DFL system. In this context, we established the following key criteria, listed in order of priority:

1. **Permissionless access**: Ensures that any node can join the network without needing authorization, supporting a fully decentralized and open DFL system;
2. **Support for smart contracts**: Essential to implement logic for storing and synchronizing local model updates between nodes. Without the capabilities of smart contracts, automation would not be feasible;
3. **Network maturity and stability**: A well-established and reliable DL is less prone to downtime, critical bugs, or network splits, ensuring dependable training sessions;
4. **Energy efficiency**: Since model training already requires significant computational resources, the DL should prioritize energy efficiency to minimize environmental impact and operational costs;
5. **Active developer and community support**: A robust community and developer base are vital for ongoing maintenance, documentation availability, and troubleshooting assistance;
6. **Long-term viability**: The DL must have a sustainable economic model and a clear roadmap to ensure it remains operational and relevant in the future.

Ethereum was the first permissionless blockchain to introduce smart contract functionality and remains the most widely adopted platform for dApps [50], [125], [126]. Its recent transition from PoW to PoS in September 2022 significantly improved its energy efficiency, reducing energy consumption by over 99%, aligning with our goals [53], [54], [127]. Additionally, Ethereum benefits from a large and active ecosystem, ongoing development, and considerable community and institutional support [128]–[130]. Ethereum is also a broadly used solution in DFL, as supported by [93]–[96], [99], [113], [117]. These factors provide strong assurances of continuity, security, and future growth in the context of a DFL system. While other blockchains may offer similar features, Ethereum's combination of maturity, versatility, and proven resilience made it the best choice for our project.

By leveraging blockchain technology, Ethereum also enables the primary beneficiaries of the DFL system to financially reward network users for their contributions to training and validating AI models, as discussed in [131]. Although this aspect is not the primary focus of this work, it shouldn't be overlooked in the challenges presented by

the WIT Software's scenario. Encouraging users to utilize their computing resources can be difficult, as this often involves monetary expenses for the users and additional energy consumption. This challenge is particularly pronounced with mobile devices, where energy costs are increased due to battery drain and gradual deterioration. Introducing a blockchain reward system can help offset the extra effort and resources required.

To reach a consensus about the global model, it is crucial for the nodes in the DFL network to validate each other's local models to establish a trustless network. This means that all AI models trained by the nodes need to be accessible to every other node in the network. While it is possible to use Ethereum for this purpose, alternative storage methods are necessary for practical applications due to the high storage costs associated with Ethereum [132]. To explore a more cost-effective solution suited for these scenarios, we have chosen to implement IPFS alongside Ethereum. This combination offers a decentralized and affordable data storage option. IPFS is the most widely utilized decentralized storage solution, with over 9,000 projects relying on it within the Ethereum and EVM compatible chains alone [133]. Moreover, it is extensively referenced in DFL systems, as evidenced in [95]–[97], [99], [115]–[117].

Due to the decisions made regarding data storage and model aggregation consensus, we have chosen to use one of the code bases from the Federated Learning Token (FELT) project [134], [135] as the foundation for our DFL system. This project was selected because it utilizes the same DLs we had chosen for the same purposes, specifically Ethereum for synchronizing information and IPFS for storing the AI models.

The FELT implementation involves locally training models using data available to each specific node, as shown in Figure 4.1. These locally trained models are then aggregated to create a global model, as illustrated in Figure 4.2. The final model of the aggregation process is created by selecting a random node from the network. This node is responsible for performing the computation for the aggregation, which means that the other nodes must trust the outputs produced by this randomly selected node.

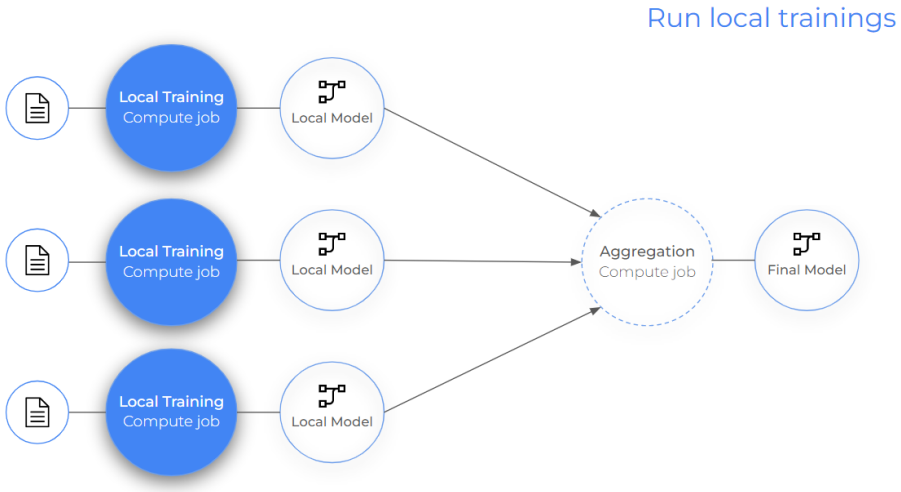


Figure 4.1: FELT Local AI Models Training Diagram [136].

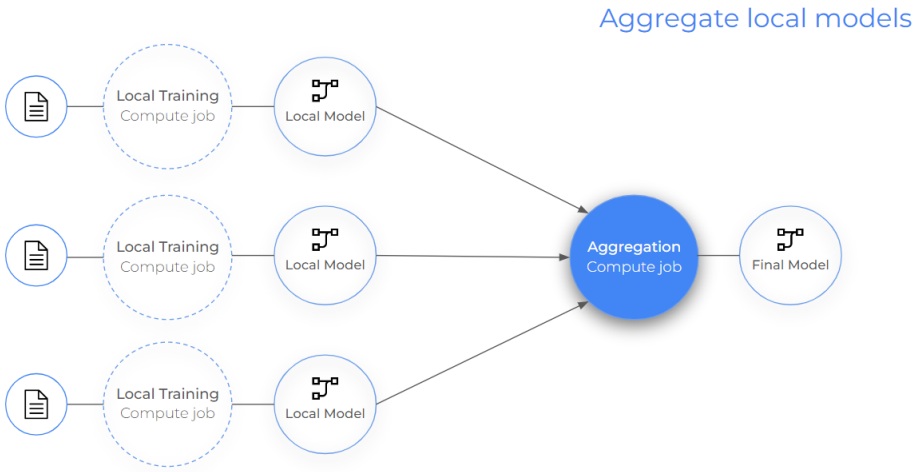


Figure 4.2: FELT AI Models Aggregation Diagram [136].

This work utilized the FedAvg as the aggregation method, the same one used in the FELT project. This algorithm is straightforward, averaging results from each node. We decided to use this method primarily due to its simplicity, as it serves as an effective tool for testing the impact of various strategies across the nodes and evaluating their effects on the DFL system. The FedAvg method is well-suited for the proposed scenario, as it consumes little energy, which reduces battery drain on mobile devices compared to other alternatives.

Considering the various characteristics of the challenge presented by WIT Software and the previously mentioned technology choices, including Ethereum, IPFS, and FELT architecture, the global DFL architecture proposed by this project is depicted in Figure 4.3:

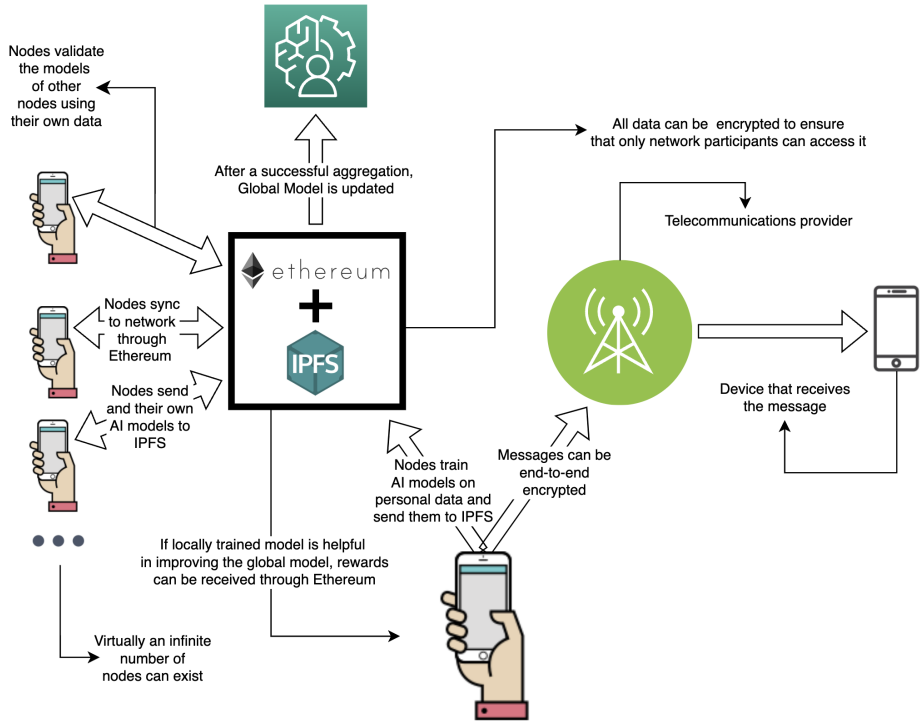


Figure 4.3: Global DFL Architecture of this Project.

It is important to emphasize that this architecture allows messages exchanged between different devices over the mobile communications network to be end-to-end encrypted. With this encryption in place, telecommunications operators cannot access conversations or voice messages at any point during transmission. While this aspect may not be the primary focus of this work, it becomes critical if the proposed system is deployed in a real-world scenario, such as the one envisioned by WIT Software, where ensuring user privacy and data security is essential.

4.3 Protective Measures Against Poisoning Attacks

In the original FELT version, a randomly selected node would be responsible for aggregating the various models. However, this approach is not suitable for our goal of creating a system resistant to poisoning attacks, as in such systems, there are no guarantees regarding the integrity of a specific node. If a malicious node is chosen, the entire system would have to place trust in that single node, creating a single-point vulnerability.

To safeguard the proposed system from poisoning attacks, a modification was made to the FELT project. We changed the aggregation method to one where all nodes aggregate all the nodes' local models. The global model considered by the system would then be the one represented by more than 50% of the nodes uploaded to IPFS. This approach ensures that if a node in the network is malicious, it cannot disrupt the en-

tire network merely by chance. It also guarantees that the model aggregation remains healthy as long as more than 50% of the nodes are trustworthy.

However, this approach does not fully protect the system against poisoning attacks. A malicious actor could intentionally or accidentally train a poorly performing model, and that model may still be considered during the aggregation process. To address this risk, we have implemented an additional validation feature within the system.

Our additional validation requests all nodes to evaluate the models of other participants locally using their own data and a weighted average F1-score. They will then store the evaluation results on IPFS. Only the models that achieve a weighted F1-score greater than 70% on more than 50% of the network participants will be eligible for final aggregation by the other network participants. This extra validation process ensures that the entire network can remain operational even if only 50% of the evaluations are trustworthy, similar to the principles of blockchain technology. The specific code employed for this strategy is publicly available in [137].

4.4 Protective Measures Against Inference Attacks

We assessed the main strategies for mitigating inference attacks, with a particular emphasis on reducing model size, introducing data noise, and utilizing encryption techniques. These approaches are well-established in the literature as effective means of enhancing privacy in machine learning models [42], [138]. To refine our analysis, we specifically focused this work on evaluating model compression methods, namely pruning, post-training quantization, and model conversion using a compression tool. We deliberately excluded techniques such as knowledge distillation due to the added complexity involved in evaluating the various interacting variables associated with those methods.

Regarding data noise, we again focused solely on testing the differential privacy method to limit the scope. We did not evaluate encryption methods, such as homomorphic encryption, because their high computational costs render them impractical for the resource-constrained environments we considered in this study [92].

5 Implementation

This chapter outlines the decisions made regarding the implementation of the project. These choices, along with the architectural design and foundational technology choices discussed in Chapter 4, establish the groundwork for the case studies and subsequent results discussed in Chapter 6.

In Section 5.1, we provide details about the hardware and initial setup required to conduct the tests. Section 5.2 explains how the dataset was redistributed between the training, validation, and test phases and the nodes of the DFL network. Section 5.3 describes the architecture of the CNN and its dependencies. The Section 5.4 detail the measures taken to protect against inference attacks. Finally, in Section 5.5, we discuss decisions made regarding the implementation of the DFL network.

In the GitHub repository associated with this work [137], we provide the code specifically developed for all the implementation decisions discussed in this chapter. It includes scripts for model training, data preprocessing, evaluation procedures, and model architecture definitions, as well as the complete implementation of the DFL system, including the aggregation strategy. The repository is organized to promote reproducibility and ease of access, ensuring that all methods, models, and results discussed in this work can be independently verified.

5.1 Hardware Setup

The experimental validation of this project requires hardware capable of testing the capabilities and behavior of a decentralized network of nodes. This validation can be conducted in two main ways:

1. **Simultaneous Testing**: Multiple devices perform tests in parallel, simulating realistic conditions. This approach is faster but requires more resources;
2. **Sequential Testing**: A single device sequentially runs each test for every node. While this method takes more time, it uses fewer resources.

Since the primary objective of the experiments is to validate the performance of the DFL system, both scenarios are acceptable. We have opted to run the tests on a single server using sequential testing to conserve resources.

For our server, we opted for a hardware service provider instead of using a personal computer to ensure that our results would be more reproducible and replicable. We

wanted a platform that was accessible, customizable, and user-friendly, which led us to choose Google Colaboratory. Also known as "Colab," this tool from Google Research allows users to write and execute code directly in their web browsers. It is especially effective for ML, data analysis, and educational purposes [139].

Colab is a hosted Jupyter Notebook service that requires no setup and offers free access to computing resources, including GPUs. Additional factors influencing our selection of Google Colaboratory include the simplicity of achieving standardization for our tests and its cost-effectiveness relative to our hardware requirements.

5.2 Dataset

With regard to the dataset used in this work to train the DFL system, it was necessary to determine how to redistribute the dataset, specifically which files would be allocated for the training, validation, and test sets. We adopted the same strategy outlined in TensorFlow's example code for this dataset [140].

This strategy relies on creating a hash derived from the file name, and then using that to design if the dataset file will be used for training, validation, or testing. The method ensures that the dataset is split into 80% for training, 10% for validation, and 10% for testing. We decided to change the default setting for the dataset from 20% for testing to 10%, since we are splitting the dataset across several nodes and therefore have more data available for training the models. This approach not only ensures the reproducibility of the results, but also maintains consistency in the dataset, even with the addition of new files in the future.

The original dataset included some audio recordings that consisted solely of noise. These recordings were intended to help train the model to be more resilient to noise in real-world situations. However, as will be explained later in 5.4, we developed our own technique for adding noise to the data. Consequently, we chose to remove all audio recordings that contained only noise from the dataset and exclude them from training in order to avoid any conflicts with our method.

5.3 Convolutional Neural Network and Dependencies

The specific choice of the CNN architecture was not a critical factor in this research, as the primary objective was to validate the feasibility of implementing a decentralized audio-to-text DFL system, compare it with a more centralized approach, and assess how the DFL system's performance varied under different protective measures against attacks. Consequently, the model selection was guided by a set of practical criteria outlined below:

- **Open-source accessibility**: The model must have publicly available architecture and code to ensure reproducibility;
- **Ease of integration**: Compatibility with widely-used frameworks, to simplify deployment and experimentation;
- **Simplicity and efficiency**: Preference for lightweight models suitable for resource-constrained devices.

The original Speech Commands CNN, developed by Google, is used to classify specific words from the first version of the Speech Commands dataset using the TensorFlow framework [141], [142], as demonstrated in one of Google's examples [143]. This CNN meets all our requirements and was therefore utilized as the basis for this study. For the loss function, we applied the standard cross-entropy loss, which is also used in Google's examples. Additionally, we selected the SGD optimization algorithm due to its lightweight implementation [144].

The CNN utilized in this experiment is a deep convolutional architecture specifically designed for multi-class audio classification. It processes input spectrograms through a series of four 2D convolutional layers, each followed by max pooling to progressively reduce the spatial dimensions while capturing local audio patterns.

The first convolutional layer uses eight filters, while each subsequent layer employs 32 filters. After a pooling layer, the output is flattened into a one-dimensional vector, which is then processed through a dense layer consisting of 2,000 units. This structure enables the model to learn high-level representations. To reduce the risk of the model memorizing the training data too closely and improve its ability to classify unseen inputs, dropout layers are included both before and after the dense layer.

The final dense layer produces probabilities for 35 classes, corresponding to the target labels. The total number of parameters in this model is 1,498,699, all of which are trainable. A complete summary of the model architecture, including the input shape of the model and output shapes and parameter counts for each layer, is presented in Table 5.1. Further details about the model are available in [140], [145]–[147].

Table 5.1: Convolutional Neural Network Architecture Details.

Input Shape	(43, 232, 1)	
Layer Type	Output Shape	Params
Conv2D	(None, 42, 225, 8)	136
Max Pooling2D	(None, 21, 112, 8)	0
Conv2D	(None, 20, 109, 32)	2080
Max Pooling2D	(None, 10, 54, 32)	0
Conv2D	(None, 9, 51, 32)	8224
Max Pooling2D	(None, 4, 25, 32)	0
Conv2D	(None, 3, 22, 32)	8224
Max Pooling2D	(None, 2, 11, 32)	0
Flatten	(None, 704)	0
Dropout	(None, 704)	0
Dense	(None, 2000)	1410000
Dropout	(None, 2000)	0
Dense	(None, 35)	70035
Total params	1,498,699	
Trainable params	1,498,699	
Non-trainable params	0	

To ensure a fair comparison between nodes, the same CNN architecture, loss function, and optimization algorithm were used for both local and global models within the DFL system. Similarly, to maintain consistency across experiments, these same methods were applied to all tests, including those involving the centralized approach.

The TensorFlow library relies on Keras [148], Python [149], and NumPy [150] as its dependencies. All of these libraries incorporate randomness by initializing by default random seeds, which are utilized in various algorithms. To ensure that our results are easily reproducible, we decided to initialize the seeds for all these libraries with the static value 42. Specifically for TensorFlow, we also used the function `config.experimental.enable_op_determinism()` and set the system environment variable `environ['TF_DETERMINISTIC_OPS']` to 1 in order to make TensorFlow operations as deterministic as possible.

In relation to the CNN, we needed to select various training parameters, including the batch size and the number of epochs. Although optimizing the performance of the AI model was not the primary objective of this work, we aimed to create a consistent and reliable model to facilitate meaningful comparisons between DFL and centralized approaches.

To achieve this, we based our parameter choices on Google's official examples, which demonstrate the model's effectiveness with the Speech Commands dataset. Specifically, we opted for a batch size of 32 and 25 training epochs, as these settings had already been validated for this task. By maintaining these parameters across all exper-

iments, we ensured fairness and reproducibility, thereby minimizing the risk of poor model tuning affecting the comparative analysis.

Some optional parameters were also available for training this model, such as applying transfer learning from a pre-trained audio model and training only the final layer. To better assess the impact of our modifications on the model's training process, we opted not to use these options and instead trained the entire model from scratch on our own data, while maintaining its original architecture.

5.4 Protective Measures Against Inference Attacks

To evaluate the effectiveness of the CNN with inference attack protection techniques in place, we applied a series of model size reduction techniques. These methods included pruning, converting the model using a dedicated tool, and post-training quantization. The compression techniques were applied sequentially. We started by pruning the fully trained, non-compressed model, then converted it using the dedicated tool, and finished with post-training quantization.

For pruning, we employed one of the techniques from the official TensorFlow documentation [151]. This method utilizes a polynomial decay function applied to all layers of the model, starting with 50% sparsity and gradually increasing to 80% sparsity over the course of training. We chose the polynomial decay function because it introduces sparsity slowly, allowing the model weights to adapt to this change and become more robust against the effects of weight removal. This process is somewhat similar to quantization-aware training used in quantization. Since we did not implement a quantization-aware method, we decided to use this pruning technique as it offered a different strategy.

For the model conversion process, the pruned model was converted using the TensorFlow Lite tool, now referred to as LiteRT [152], [153]. LiteRT is designed to implement various optimizations and quantization techniques aimed at reducing model size and inference latency, while maintaining minimal degradation in detection or classification accuracy. Specifically, LiteRT employs several optimizations to enhance model efficiency, particularly for resource-constrained devices. These include operator fusion, quantization, and memory layout optimizations that minimize both memory usage and computational overhead. Collectively, these techniques not only improve performance but also provide an extra layer of robustness against certain types of attacks by compressing and transforming model parameters.

While alternative frameworks such as ONNX Runtime [154] and NVIDIA's TensorRT [155] also provide compression and optimization capabilities, LiteRT was selected due to its comprehensive documentation, strong community support, and native compatibility with TensorFlow models. These factors made it a practical and reliable choice for

our objectives, which include ensuring reproducibility, ease of deployment, and maintaining performance in constrained environments. During the conversion, we enabled optimizations for all compatible layers of the model, and where this was not feasible, the default layer sizes were preserved.

For the post-training quantization, we utilized the the same LiteRT framework. We converted the model using the default parameters of the framework, which means that the model's weights were reduced to 8-bit instead of 32-bit float [156]. This decision was made to significantly reduce the model's size and better evaluate its impact on efficiency.

In addition to the model compression techniques, we also evaluated a differential privacy method. This method involved generating Gaussian noise centered at 0.0, with dimensions matching each audio sample used in training and validation. The generated noise was then added to the original audio, producing a noisy version of each sample. Gaussian noise was chosen because of its well-understood statistical properties and its established use in the ML context [15], [157]. To assess the impact of different noise levels on the model, we applied Gaussian distributions with standard deviations of 0.2, 0.4, 0.6, 0.8, and 1.0.

5.5 Decentralized Federated Learning

In addition to conducting tests to evaluate the viability of the system's results, as presented in Chapter 6, further assessments were carried out to determine its feasibility in addressing the first research question: "Is it feasible to implement a robust and resilient audio-to-text DFL system in an environment where the network participants cannot be managed, trusted, or predetermined?"

To achieve this, the DFL system was implemented on a computer running Ubuntu version 22.04.1 LTS. Three nodes were configured to process different parts of the Speech Commands v2 dataset simultaneously on the same machine using the FELT project.

For the aggregation of the DFL system, we utilized FedAvg. All the weights of the CNNs were aggregated and averaged based on the number of nodes. For models that were not compressed or pruned, we saved the models and aggregated them using the SavedModel format and the TensorFlow library. In contrast, for models that had undergone size reduction through LiteRT, we saved and aggregated them using the FlatBuffer serialization format. Since there were no external libraries available to directly support FlatBuffer aggregation, we developed a custom aggregation implementation specifically for this work to meet the system's requirements.

6 Case Studies and Results Analysis

This chapter presents the case studies conducted in this research, outlining their rationale and the results obtained. The case studies rely on the architectural design and implementation decisions outlined in Chapters 4 and 5, respectively.

The primary goal of the studies presented in this chapter is to address the research questions introduced in Chapter 1. Sections 6.2, 6.3, and 6.4 present results that focus on the second and fourth research questions of this work: “What is the impact of employing methods to defend against inference attacks on the performance of such an audio-to-text DFL system?” and “How does the performance and resiliency of an audio-to-text DFL system compare to that of a centralized counterpart, including when methods to defend against inference and poisoning attacks are employed?”

Additionally, Section 6.4 focus on the third research question: “What is the impact of employing methods to defend against poisoning attacks on the performance of such an audio-to-text DFL system?” All the sections in this chapter, along with Chapters 4 and 5, address the first research question: "Is it feasible to implement a robust and resilient audio-to-text DFL system in an environment where the network participants cannot be managed, trusted, or pre-determined?"

The chapter is organized as follows: after providing a contextual introduction and general information about the case studies presented across this chapter in Section 6.1, we analyze the performance of the DFL system when safeguards against inference attacks are in place throughout Sections 6.2, 6.3, and 6.4. Section 6.3 focused more on the models’ resilience in situations where the dataset is unbalanced, while Section 6.4 examined more its behavior under poisoning attacks, both with and without protective measures. Section 6.5 provides a synthesis and general discussion of the results obtained. Finally, Section 6.6 discusses the threats to the validity of this study.

6.1 Introductory Overview

The use case from WIT Software that inspired this work aimed to train AI models using private voice messages without direct access to the messages themselves. In this scenario, the data within a DFL system is not evenly distributed across different nodes. Each node would contain biased datasets, with a higher concentration of audio messages from specific individuals—typically the device owners and their close contacts. This leads to a tendency for certain terms and expressions to be overrepresented in

the data. Given this context, it is crucial to demonstrate the overall robustness and resilience of the DFL system. Therefore, we assessed the DFL system across multiple nodes, utilizing both balanced and unbalanced data distributions.

Regarding our evaluation of the DFL system across multiple nodes, we distributed the dataset among one, three, five, seven, and nine nodes. Each node was trained sequentially on the same Colab hardware, independently processing different files. This approach enabled us to assess the performance of the DFL system as it was progressively distributed across various nodes. We chose an odd number of nodes to prevent ties during the consensus-based decision-making process, which facilitated a more robust assessment of the protection mechanisms against poisoning attacks.

To maintain the integrity of the experiments, we limited the evaluation of the DFL to a maximum of nine nodes, since the validation dataset contains only 4,890 audio samples across 35 word categories. Distributing the data among nine nodes results in an average of approximately 15 audio samples per category per node. We ensured this minimum sample size to reduce the risk of extreme data scarcity, which could lead to model overfitting, poor generalization, and unreliable local updates—issues that are particularly critical in FL systems [15], [34].

All results are compared to a centralized baseline, referred to as “1 Node" in the relevant tables and graphs of this work. The centralized approach involves training a single model using data from all nodes combined, without any distribution. This method typically produces better performance since the model can learn from the entire dataset at once. In contrast, the distributed approach entails training separate models on individual nodes and then aggregating their outputs. This may lead to performance limitations due to factors such as data heterogeneity and potential loss of information during the aggregation process [34]. Taking this information into consideration, the results presented in this chapter primarily emphasize the F1-score performance losses observed in DFL experiments compared to their centralized counterparts.

The performance of the DFL system with safeguards against inference attacks was evaluated not only in Section 6.2, but also alongside other experiments. Section 6.3 combines defenses against inference attacks with unbalanced data, while 6.4 assesses safeguards against inference attacks under poisoning attacks. In all results related to inference attack protection, a value of "0.00" was used to represent a baseline condition, indicating that no noise was added to the data.

Throughout this chapter, we use the term "standard model" to refer to a model that has not been subjected to any model compression techniques. This means it has not undergone any form of pruning, has not been converted using the LiteRT tool, and has not been quantized.

In addition to the F1-score, results for cross-entropy, weighted average precision and

recall, and accuracy were also obtained for the same experiments discussed in this chapter. These results are presented in Appendix A.

With regard to poisoning attacks, Subsections 6.4.1 and 6.4.2 present only the results in which a minority of the nodes in the DFL network were poisoned, thereby preserving the integrity of the system by maintaining a majority of honest nodes above the 50% threshold. However, experiments involving a majority of poisoned nodes were also conducted and are presented in Appendix B.

All the results discussed in this chapter, along with those presented in Appendices A and B, are accessible in the results directory outlined in Appendix C. This database includes additional details regarding each experimental parameter, the results for every individual node in each DFL experiment, the models' weights across all experiments, and the dataset partitioning used throughout the experiments.

6.2 Performance With Protections Against Inference Attacks

One of the main objectives of an FL system is to protect the data of the various participants involved. Therefore, it is essential for such systems to have safeguards against inference attacks, which could allow participants to gain access to unauthorized data through reverse-engineering techniques. This concern is particularly relevant in the DFL system we have developed, as the data is stored on IPFS, making it accessible to anyone with internet access [72]. Even if the data is encrypted and only available to authorized DFL participants, there is still a risk that a malicious actor on the network could share the data publicly.

To address these concerns, several techniques have been developed to protect against inference attacks in the context of DFL systems, as discussed in Subsection 2.5.2. In this work, we analyzed the F1-score performance losses of the DFL system when four safeguards against poisoning attacks were in place. Three of these methods focus on reducing model size: pruning, model conversion using the LiteRT tool, and post-training quantization. The fourth method involved adding noise to the training and validation data by generating Gaussian noise centered at 0.0, with standard deviation levels set to 0.2, 0.4, 0.6, 0.8, and 1.0. To enhance readability, standard deviation levels are presented as "noise percentage" in all tables and figures where relevant. Further details about these approaches can be found in Sections 4.4 and 5.4.

Instead of using these techniques in isolation, we applied them in sequence. First, we introduced data noise. Next, we performed pruning. Then, we converted the model. Finally, we implemented post-training quantization. This step-by-step approach was designed to mimic a realistic deployment scenario where multiple layers of defense are

combined to strengthen privacy protection. As a result, all scenarios involving converting the model also include pruning, and all scenarios with post-training quantization include both pruning and converting the model.

6.2.1 Performance With Protections Against Inference Attacks Results

The standard models (Table 6.1 and Figure 6.1), demonstrate that adding noise to the data has a substantial negative impact on the DFL system performance. The average difference between a model without noise and one with a noise level of 1.00, across three, five, seven, and nine nodes, was 44.73%. Furthermore, the difference between a model with one node and no noise compared to a model with nine nodes and a noise level of 1.00 was 61.05%. This degradation is likely due to the noise interfering with the local models' ability to learn meaningful patterns from the data, thereby reducing the overall quality of the global model during the aggregation process.

Table 6.1: Standard Model - Weighted average F1-score considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.896	0.794	0.771	0.738	0.762
0.20	0.820	0.741	0.678	0.650	0.594
0.40	0.791	0.637	0.622	0.555	0.514
0.60	0.751	0.561	0.497	0.507	0.464
0.80	0.712	0.496	0.469	0.440	0.390
1.00	0.698	0.513	0.452	0.384	0.349

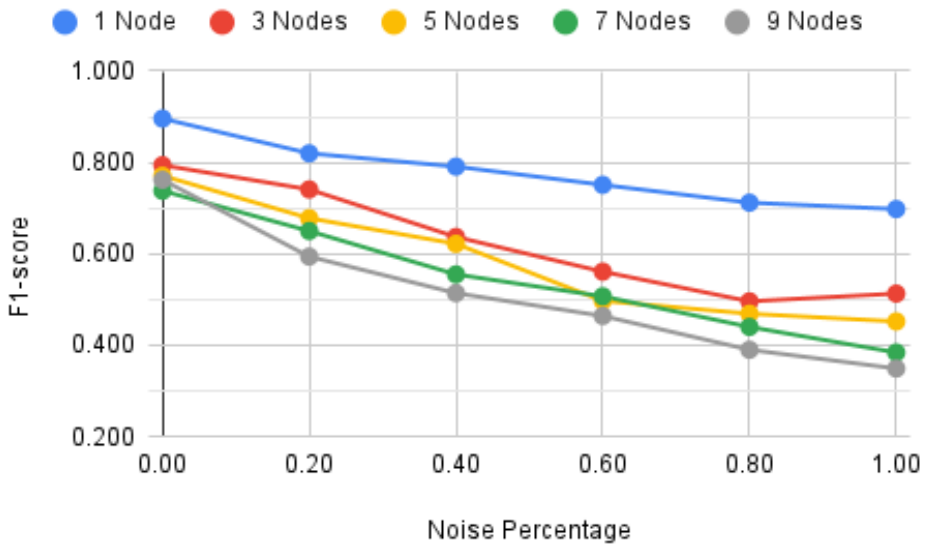


Figure 6.1: Standard Model - Weighted average F1-score considering noise in the data.

The results shown in Tables 6.2, and 6.3, as well as Figures 6.2, and 6.3, indicate that

the differences in effectiveness between the pruned and LiteRT-compressed models remained largely consistent. Moreover, in some cases, these models even outperformed the standard models. These findings suggest that the standard models may be slightly overfitted. These findings suggest that the standard models may be slightly overfitted. Under the evaluated conditions, it appears that these safeguards tested against inference attacks do not significantly impact the performance of these models.

Regarding the quantized models (Table 6.4 and Figure 6.4), the performance degradation became more pronounced. Across all noise levels with three nodes, the quantized models exhibited an average performance loss of 36.22% compared to the centralized quantized model without noise, while the standard models had a loss of 34.20% for the corresponding experiment. Under the same conditions but with nine nodes, the quantized models showed an average loss of 52.66%, whereas the standard models experienced a 48.42% loss. This corresponds to a difference of 2.02% with three nodes and 4.24% with nine nodes — more than double. This increased sensitivity is likely attributable to the combined effects of quantization, pruning, and LiteRT conversion in the quantized models, which may have further compromised their generalization capacity under adverse conditions.

Table 6.2: Pruning - Weighted average F1-score considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.906	0.806	0.782	0.738	0.765
0.20	0.831	0.771	0.678	0.652	0.600
0.40	0.794	0.635	0.633	0.571	0.522
0.60	0.754	0.567	0.529	0.518	0.466
0.80	0.716	0.495	0.488	0.434	0.395
1.00	0.684	0.500	0.444	0.384	0.359

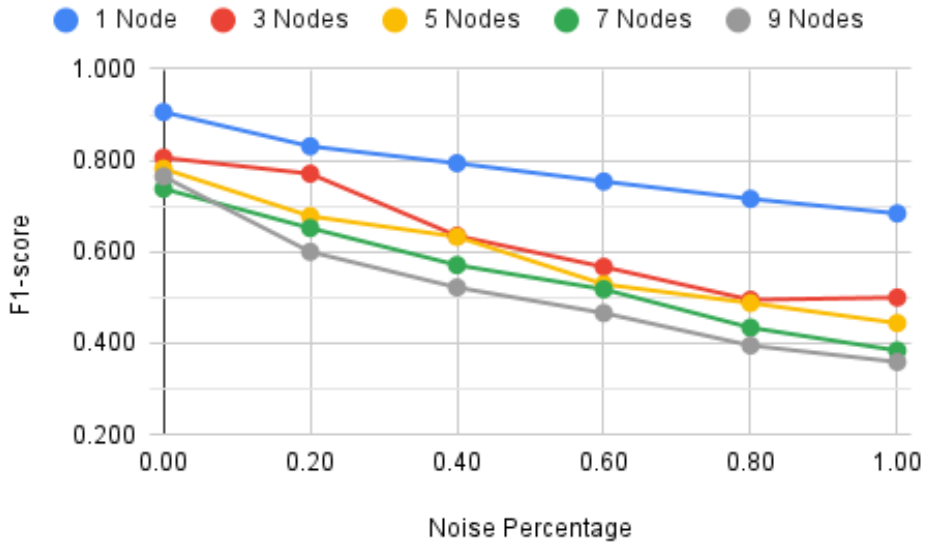


Figure 6.2: Pruning - Weighted average F1-score considering noise in the data.

Table 6.3: LiteRT - Weighted average F1-score considering noise in the data..

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.906	0.806	0.782	0.738	0.765
0.20	0.831	0.771	0.678	0.652	0.600
0.40	0.794	0.635	0.633	0.571	0.522
0.60	0.754	0.567	0.529	0.518	0.466
0.80	0.716	0.495	0.488	0.434	0.395
1.00	0.684	0.500	0.444	0.384	0.359

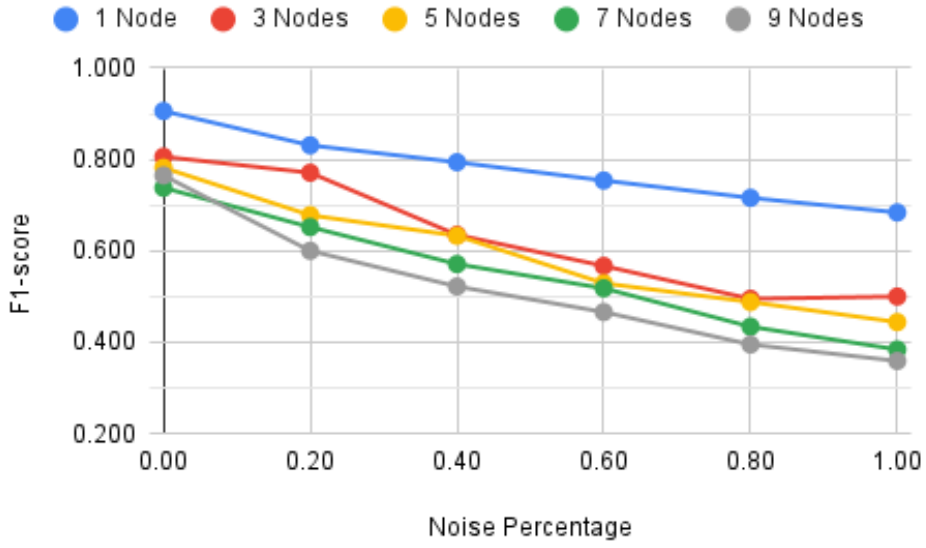


Figure 6.3: LiteRT - Weighted average F1-score considering noise in the data.

Table 6.4: Quantization - Weighted average F1-score considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.905	0.788	0.767	0.690	0.751
0.20	0.832	0.753	0.661	0.632	0.492
0.40	0.793	0.596	0.620	0.529	0.510
0.60	0.754	0.543	0.488	0.482	0.450
0.80	0.715	0.475	0.487	0.393	0.368
1.00	0.685	0.519	0.450	0.358	0.322

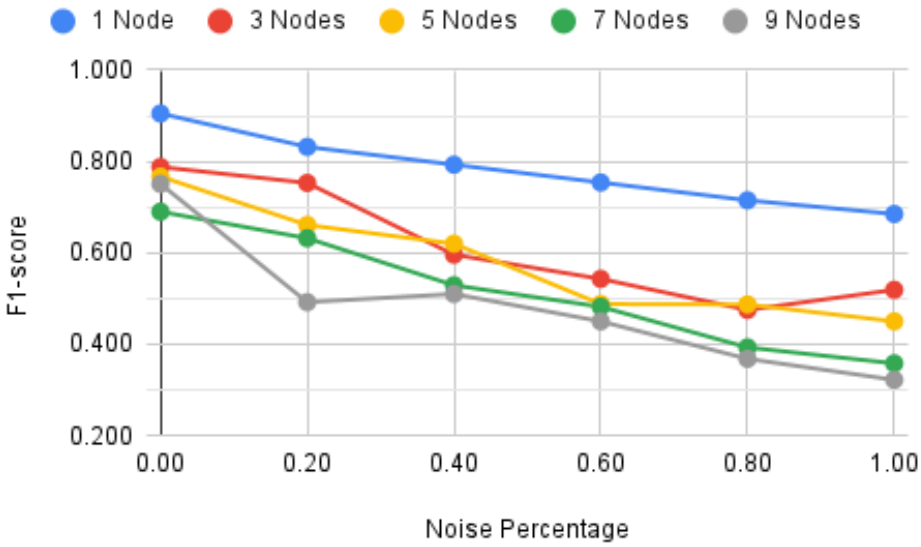


Figure 6.4: Quantization - Weighted average F1-score considering noise in the data.

6.3 Resilience Against Unbalanced Data

To evaluate the models' behavior with uneven data distribution among the nodes, we purposely increased the number of samples in half of the dataset categories for the nodes. In this context, the categories refer to the 35 spoken words used in the experiments, such as "Yes," "No," "Up," "Down," "Left," "Right," "On," "Off," among others. Specifically, we doubled, tripled, quadrupled, quintupled, or sextupled the amount of data for half of the categories in these nodes, while simultaneously reducing the quantity for the other half to half, one-third, one-quarter, one-fifth, or one-sixth of the original amount. For clarity, we referred to these variations as "unbalance levels," where 1 indicates no unbalance in the data, and 6 signifies that half of the categories have been sextupled, while the other half has been reduced to one-sixth.

To ensure balance across the dataset within the entire DFL system, when the number of samples was increased in an even node, it was decreased in an odd node, and vice versa. Detailed information on the specific sample distribution per node for each experiment

is available in the results directory, outlined in Appendix C.

Throughout the presented results of this section, the "1 Node" version represents a centralized system, which means it cannot have unbalanced data distributed across nodes. Consequently, there are no further entries in the tables related to unbalanced data experiments beyond the first level for centralized experiments. However, for easier visual comparison with other experiments, the F1-score value for the centralized system is depicted as a straight horizontal line in the figures.

6.3.1 Resilience Against Unbalanced Data Results

Based on the results shown in Table 6.5 and Figure 6.5, we observe that standard models experience an 14.96% decrease in performance from the centralized solution with one node to the solution with nine nodes, assuming the DFL system is balanced and no model size reduction techniques are applied. However, in cases of unbalanced data, this effectiveness gap widens in proportion to the degree of unbalance. Specifically, at a level 2 with nine nodes, the effectiveness difference rises to 19.87%, and at a level 6 it increases to 26.23%.

The degradation of performance across different node counts shows a trend with some variability. For instance, the performance difference between unbalance levels 1 and level 6 with three nodes in standard models is 5.04%. With five nodes, this difference is reduced to 2.85%, and with seven nodes, it rises to 3.93%. Notably, with nine nodes, the difference becomes more pronounced at 13.25%. This discrepancy is partly attributed to an unexpectedly high-performance result observed in the test conducted without any noise, yielding a score of 0.762. In contrast, when testing with seven nodes, a more consistent score of 0.738 was obtained.

The results indicate that the nodes significantly reduce the impact of unbalanced data by adjusting their weights in relation to those of other unbalanced nodes, ultimately achieving a state of equilibrium. This behavior is particularly important in scenarios involving highly unbalanced datasets, such as collections of voice messages, where the varying voices and biased language patterns of individuals should be balanced by the words and voices of others. While the DFL system appears resilient to biased training data within individual nodes, further research is necessary to determine whether these findings are applicable to different datasets, node distributions, or real-world applications.

When comparing the unbalanced data results with those from Subsection 6.2.1, it is clear that, in these experiments, the introduction of noise to the data used for training had a greater impact on the outcomes. In cases of unbalanced data, the average difference between no imbalance and total imbalance for three, five, seven, and nine nodes was 2.75%. However, when noise was added, the difference between no noise and a

noise level of 1.00 for the same number of nodes increased dramatically to 31.46%.

Additionally, when examining unbalanced data, the performance difference between a standard model with one node and one with nine nodes and a level 6 imbalance was 26.23%. In contrast, after introducing noise, the difference between a model with one node without noise and one with nine nodes at a noise level of 1.00 increased significantly to 61.05%.

Table 6.5: Standard Model - Weighted average F1-score considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.896	0.794	0.771	0.738	0.762
2	-	0.818	0.771	0.777	0.718
3	-	0.780	0.770	0.734	0.724
4	-	0.759	0.762	0.738	0.727
5	-	0.771	0.760	0.726	0.694
6	-	0.754	0.749	0.709	0.661

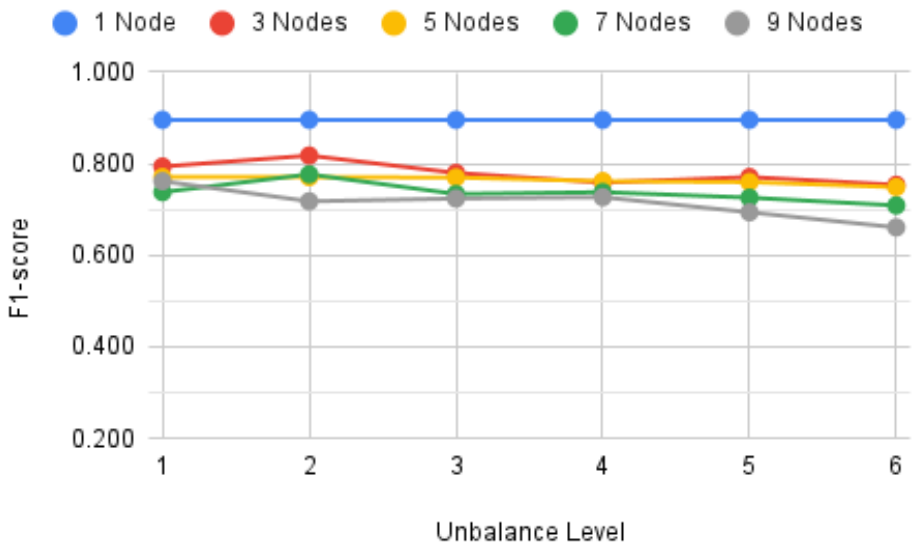


Figure 6.5: Standard Model - Weighted average F1-score considering unbalanced data.

For this use case, we also reduced the model size using the three aforementioned techniques: pruning (Table 6.6 and Figure 6.6), LiteRT (Table 6.7 and Figure 6.7), and post-training quantization (Table 6.8 and Figure 6.8). Similar to the results presented in 6.2, the results indicated slight performance improvements after applying these techniques. This suggests that the standard models might have been slightly overfitted, and reducing their complexity helped enhance generalization.

More importantly, these techniques did not result in any significant loss of model performance. This sugests that, within the conditions and range of reductions evaluated in this study, reducing the model size does not compromise performance. Therefore,

in this context, addressing data imbalance appears to be a more critical concern than simply optimizing the model size.

Table 6.6: Pruning - Weighted average F1-score considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.906	0.806	0.782	0.738	0.765
2	-	0.813	0.776	0.779	0.722
3	-	0.811	0.773	0.732	0.721
4	-	0.771	0.763	0.739	0.731
5	-	0.776	0.762	0.729	0.699
6	-	0.762	0.752	0.696	0.664

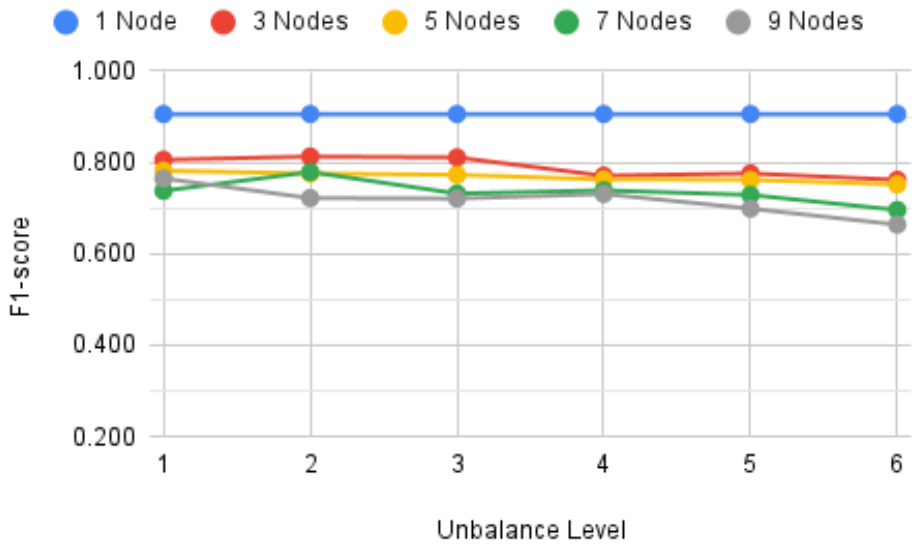


Figure 6.6: Pruning - Weighted average F1-score considering unbalanced data.

Table 6.7: LiteRT - Weighted average F1-score considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.906	0.806	0.782	0.738	0.765
2	-	0.813	0.776	0.779	0.722
3	-	0.811	0.773	0.732	0.721
4	-	0.771	0.763	0.739	0.731
5	-	0.776	0.762	0.729	0.699
6	-	0.762	0.752	0.696	0.664

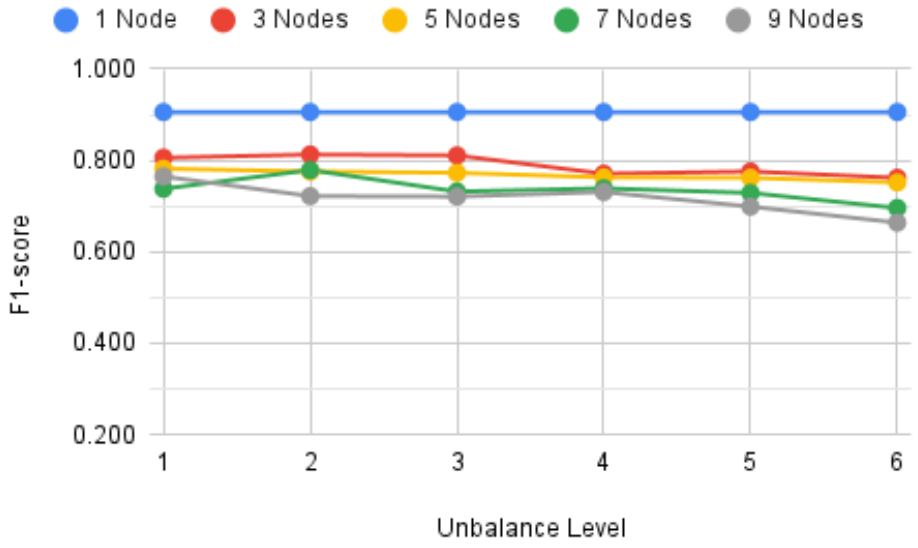


Figure 6.7: LiteRT - Weighted average F1-score considering unbalanced data.

Table 6.8: Quantization - Weighted average F1-score considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.905	0.788	0.767	0.690	0.751
2	-	0.799	0.767	0.771	0.690
3	-	0.791	0.742	0.701	0.699
4	-	0.728	0.750	0.719	0.709
5	-	0.739	0.731	0.653	0.689
6	-	0.729	0.737	0.690	0.660

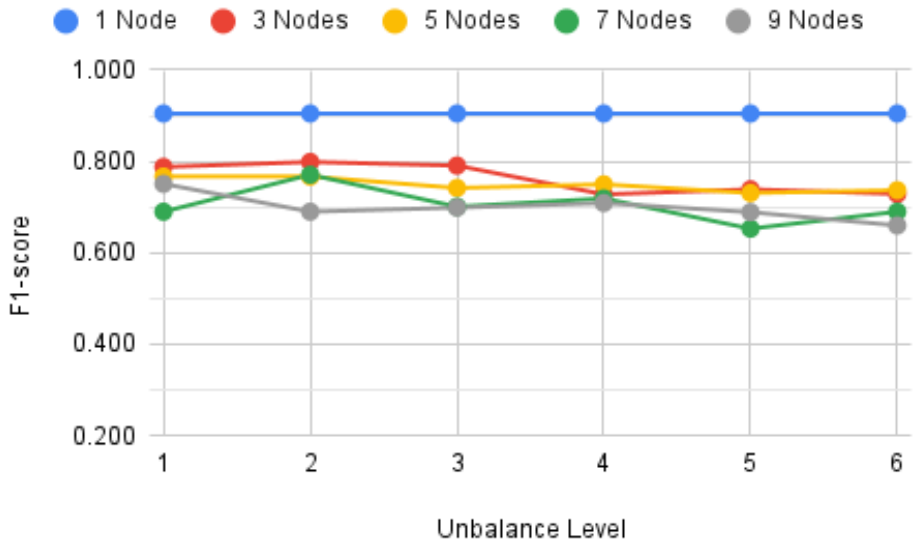


Figure 6.8: Quantization - Weighted average F1-score considering unbalanced data.

6.4 Resilience Against Poisoning Attacks

The effectiveness of a DFL system is closely tied to the quality of the models produced by its participants. If any participant delivers poor models, whether intentionally or not, it can lead to the final model performing poorly. Therefore, we decided to assess the system's effectiveness while implementing measures to protect it against poisoning attacks.

To achieve that assessment, we implemented a novel strategy where the nodes of the DFL system evaluate each other. The strategy consists of having each node assess the models of other nodes individually, making inferences based on its unique data that the other nodes do not possess. We established a threshold for a node's model to be accepted into the DLF global model, requiring an F1-score of greater than 70% in more than 50% of the total number of nodes. If a node falls below this score for more than 50% of the nodes, the winning majority will exclude it from the final aggregation, assuring that unless 50% or more of the network is compromised, the system is resilient to poisoning attacks. Further information about this strategy can be found in Section 4.3.

To test this approach, we conducted experiments with nine nodes, intentionally poisoning up to four of them. The poisoning was simulated using one of the strategies described in Section 6.2, which involves introducing noise into the data with scaling factors of 0.2, 0.4, 0.6, 0.8, and 1.0. We decided to add noise to the data for these experiments because, as shown in the results from Subsection 6.2.1, it has a significant impact on the overall outcomes. Therefore, this method could potentially be exploited by attackers to undermine the results of the DFL network. To evaluate the effectiveness of this strategy, we performed global model aggregations both with and without the protective mechanism in place.

This section presents the results related to the models' performance in the presence of poisoning attacks. It is divided into three parts:

1. Subsection 6.4.1 examines the outcomes when poisoned nodes are present in the DFL system and no protective measures are implemented;
2. Subsection 6.4.2 focuses on scenarios where protection against these attacks is in place. This protective mechanism relies on the nodes monitoring each other. As long as less than 50% of the network is poisoned, the nodes should safeguard the system by disregarding results that fall below a 70% F1-score;
3. Subsection 6.4.3 reevaluates the scenarios in Sections 6.2 and 6.3, applying the same protective techniques against poisoning attacks.

This structure aims to clarify the models' different responses to various conditions of poisoning attacks. As for the results presented in Subsections 6.4.1 and 6.4.2, all the tables include the outcomes obtained under identical conditions using a DFL system

with nine nodes and no poisoned nodes. In the figures, this result is shown at the 0.00 noise percentage mark on the x-axis of the corresponding graphs.

6.4.1 Resilience Against Poisoning Attacks Without Protection Results

When the DFL network was poisoned without protective measures, the standard models' performance dropped by as much as 18.34% compared to the unpoisoned DFL system, as shown in Table 6.9 and Figure 6.9. A performance degradation under these conditions is expected, as poisoned nodes introduce misleading models into the aggregation process, which can distort the global model and reduce its ability to generalize across the intended data distribution.

Table 6.9: Standard Model - Weighted average F1-score considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.762			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.741	0.723	0.715	0.719
0.40	0.725	0.716	0.690	0.690
0.60	0.733	0.705	0.693	0.672
0.80	0.726	0.700	0.696	0.613
1.00	0.724	0.697	0.678	0.620

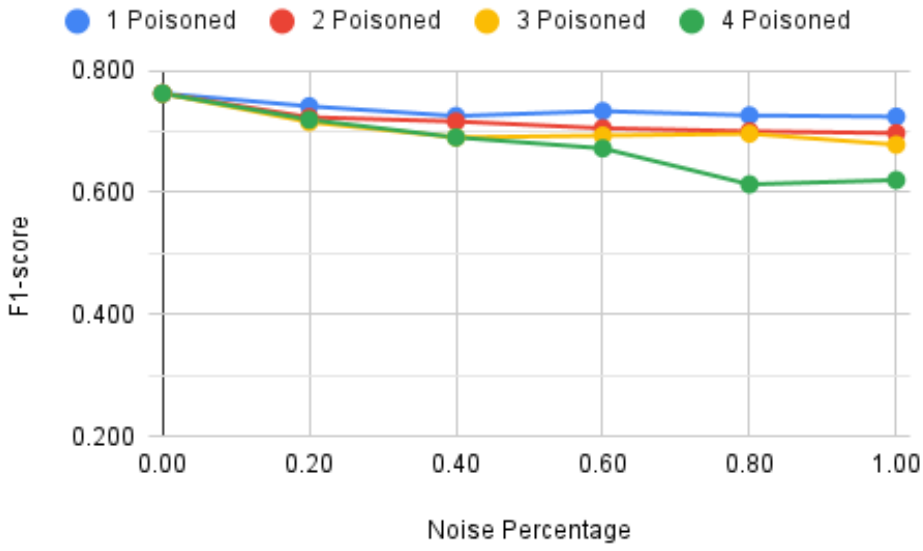


Figure 6.9: Standard Model - Weighted average F1-score considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Within the condition of this study, the results concerning the impact of poisoned nodes on compressed models indicate that, overall, model compression did not significantly

increase vulnerability to poisoning compared to the non-compressed models. Specifically, the pruned and LiteRT-converted models (Tables 6.10 and 6.11, and Figures 6.10 and 6.11) demonstrated performance under poisoning comparable to that of the standard models. These methods only resulted in a slightly higher average loss of quality in the models without poisoning.

However, the quantized models (Table 6.12 and Figure 6.12) shown greater sensitivity to poisoning, exhibiting an average performance that was 3.40% lower than that of the standard models when compared to the models without poisoning. This increased sensitivity is likely due to the fact that the quantized models were also subjected to pruning and LiteRT conversion, which together could have further reduced the models' generalization capacity.

Table 6.10: Pruning - Weighted average F1-score considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.742	0.727	0.721	0.722
0.40	0.729	0.723	0.696	0.687
0.60	0.739	0.710	0.694	0.674
0.80	0.736	0.709	0.695	0.611
1.00	0.727	0.708	0.680	0.630

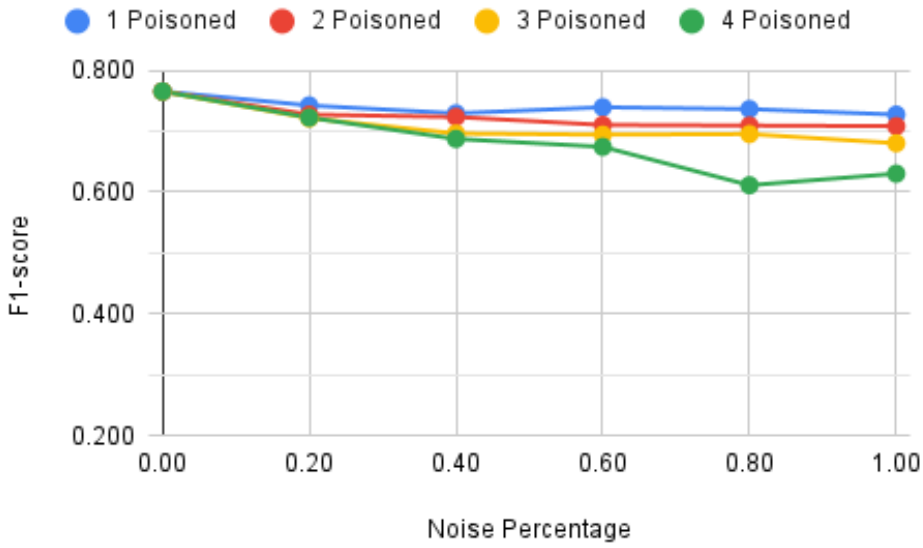


Figure 6.10: Pruning - Weighted average F1-score considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table 6.11: LiteRT - Weighted average F1-score considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.742	0.727	0.721	0.722
0.40	0.729	0.721	0.696	0.686
0.60	0.741	0.710	0.694	0.674
0.80	0.736	0.709	0.695	0.611
1.00	0.728	0.708	0.679	0.625

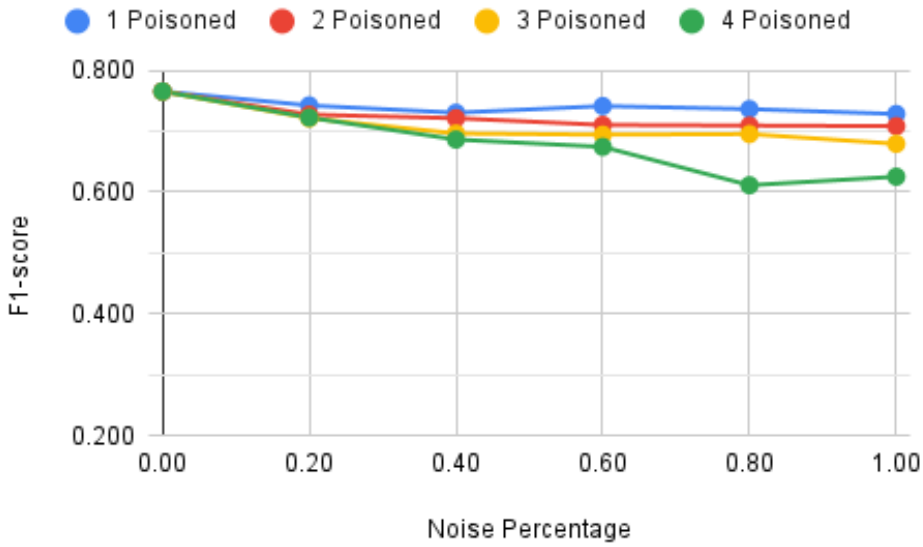


Figure 6.11: LiteRT - Weighted average F1-score considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table 6.12: Quantization - Weighted average F1-score considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.751			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.730	0.705	0.688	0.651
0.40	0.707	0.696	0.667	0.602
0.60	0.729	0.684	0.650	0.602
0.80	0.714	0.681	0.600	0.588
1.00	0.698	0.684	0.594	0.594

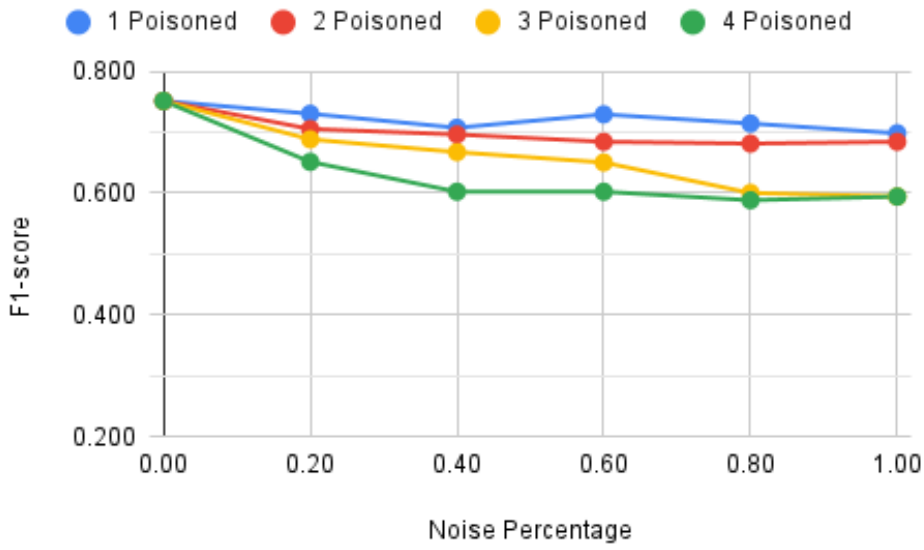


Figure 6.12: Quantization - Weighted average F1-score considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

6.4.2 Resilience Against Poisoning Attacks With Protection Results

Within the scope of this study, the results regarding poisoned nodes when the DFL network applied the strategy to prevent poisoning attacks impact were very promising, as illustrated in Table 6.13 and Figure 6.13, which correspond to the results of the standard models when until 4 nodes out of a 9 nodes network were infected. Without the removal of poisoned nodes from the DFL system, there was a significant performance loss of up to 18.64% in the final model compared to the DFL system without poisoned nodes as shown in Subsection 6.4.1. However, when these poisoned nodes were excluded, the maximum performance loss compared to the centralized model was reduced to only 3.53%.

Additionally, it was observed that implementing a strategy to consider only nodes with over 70% performance allowed for the correct detection of nearly all poisoned nodes, with only two false positives and four false negatives. This suggests that the strategy is quite effective and aligns with the desired outcomes, making it a viable option for similar scenarios. In real-world applications, the performance threshold could be adjusted to an optimal level based on the specific dataset and the quality requirements. Further studies across various datasets should aim to determine the optimal F1-score for different challenges.

It is important to note, however, that all four false negatives occurred in cases with minimal noise in the data, particularly within the 0.20 noise range. This outcome aligns with intuitive expectations: heavily poisoned nodes, which introduce substantial deviations, are more easily detected, whereas subtle poisoning attacks are inherently harder to identify. Nevertheless, this finding remains significant, as it confirms that the pro-

tective technique is effective at mitigating the most damaging attacks—those that cause the greatest degradation in model performance.

Table 6.13: Standard Model - Weighted average F1-score considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.762			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.741	0.735	0.742	0.753
0.40	0.745	0.745	0.742	0.766
0.60	0.752	0.745	0.752	0.766
0.80	0.749	0.758	0.768	0.756
1.00	0.747	0.758	0.768	0.747
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	1	1	3	3
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	1	0	0
1.00	0	1	0	0

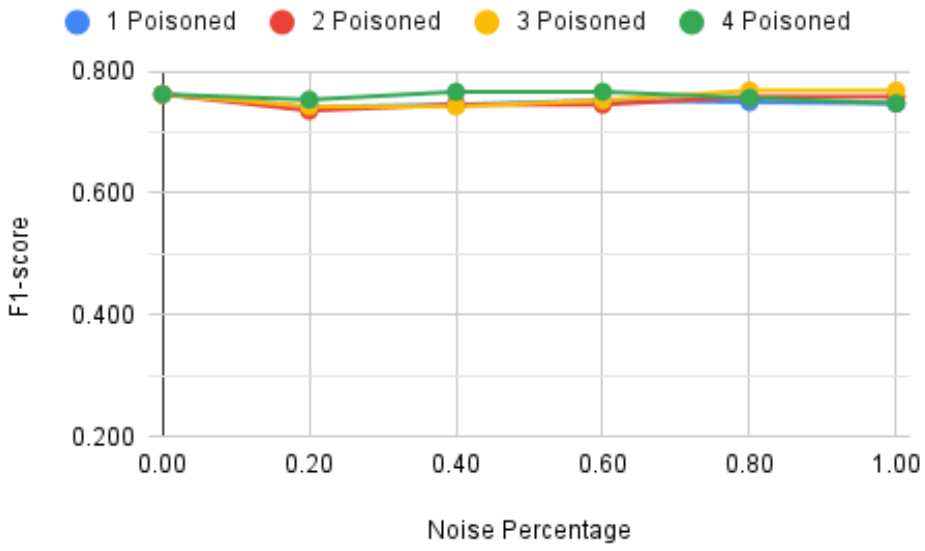


Figure 6.13: Standard Model - Weighted average F1-score considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

The results obtained from applying model reduction techniques were consistent with the findings in Section 6.2 and Subsection 6.4.1. There was a slight efficiency loss ob-

served in both the pruned and LiteRT-converted models (Tables 6.14 and 6.15 and Figures 6.14 and 6.15), with a maximum loss of 5.62% in each case.

In contrast, the quantization technique (Table 6.16 and Figure 6.16) exhibited a greater loss of efficiency, with one instance showing a loss of up to 13.32% and an average loss of 4.21%. Consistent with previous experiments, these results suggest that, under the conditions of this study, the quantized DFL system is more sensitive to adversarial conditions. This increased sensitivity likely arises because quantized models were combined with prior pruning and LiteRT conversion, further diminishing the models' generalization capacity, making them less resilient to data variations.

In terms of detecting poisoned nodes, all global models that employed model reduction techniques yielded identical results, with no false negatives. This represents a slight improvement over the standard models. However, the standard models were able to identify more poisoned nodes than the compressed models. This suggests that, under the tested conditions, while model compression techniques may enhance robustness against false negatives, they could also decrease sensitivity to less severely poisoned nodes. Therefore, there is a trade-off between improving model resilience and maintaining sensitivity to subtle poisoning attacks. This factor should be carefully considered when considering compression techniques for real-world applications.

Table 6.14: Pruning - Weighted average F1-score considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.742	0.727	0.725	0.722
0.40	0.744	0.748	0.746	0.768
0.60	0.756	0.748	0.753	0.768
0.80	0.759	0.760	0.770	0.750
1.00	0.751	0.760	0.770	0.757
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	1	0
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	0	0	0
1.00	0	0	0	0

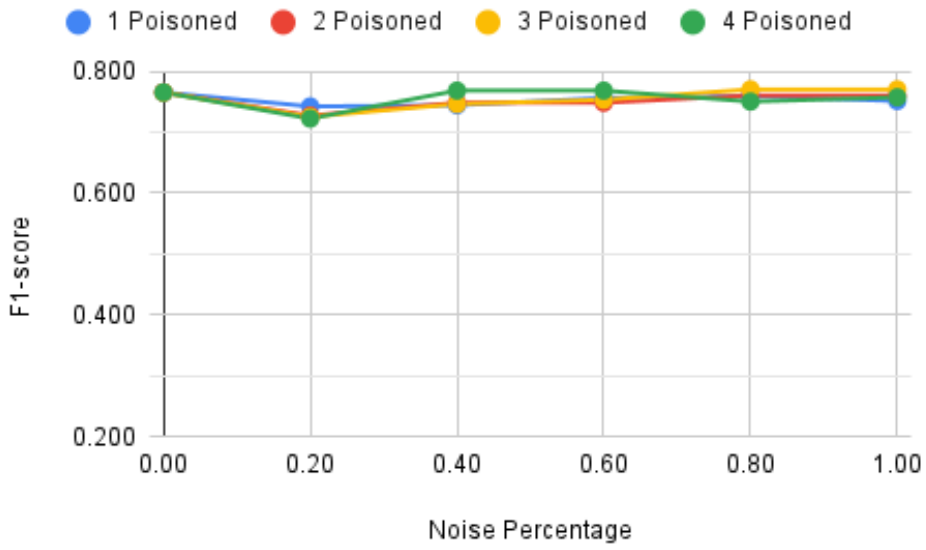


Figure 6.14: Pruning - Weighted average F1-score considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table 6.15: LiteRT - Weighted average F1-score considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.742	0.727	0.725	0.722
0.40	0.744	0.748	0.746	0.768
0.60	0.756	0.748	0.753	0.768
0.80	0.759	0.760	0.770	0.750
1.00	0.751	0.760	0.770	0.757
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	1	0
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	0	0	0
1.00	0	0	0	0

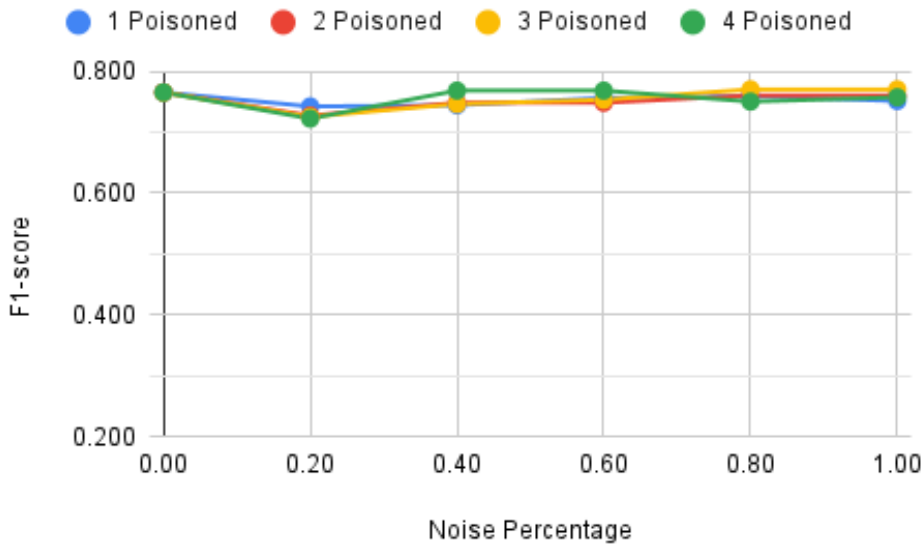


Figure 6.15: LiteRT - Weighted average F1-score considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table 6.16: Quantization - Weighted average F1-score considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.751			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.730	0.705	0.700	0.651
0.40	0.725	0.724	0.730	0.691
0.60	0.748	0.724	0.731	0.691
0.80	0.732	0.744	0.704	0.740
1.00	0.724	0.744	0.704	0.745
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	1	0
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	0	0	0
1.00	0	0	0	0

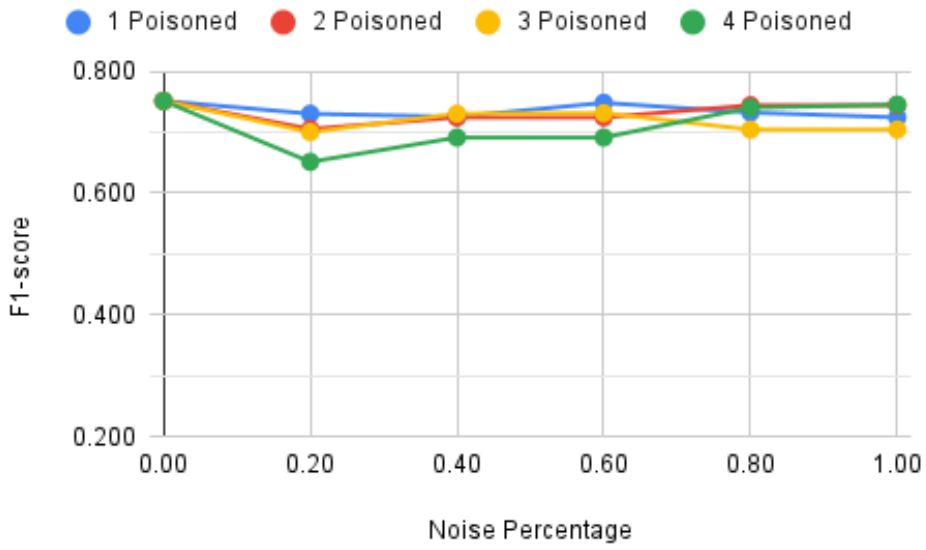


Figure 6.16: Quantization - Weighted average F1-score considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

6.4.3 Resilience Against Poisoning Attacks On Unbalanced Data Results

We applied the same strategy used against poisoning attacks in Subsection 6.4.2 to the models tested in Sections 6.2 and 6.3. The objective of this study was to determine whether the DFL system would function normally in situations without poisoned nodes, recognizing all nodes in the system as legitimate.

Overall, the system performed as expected. Exceptions, along with the number of nodes incorrectly classified as poisoned, are detailed in Tables 6.17, 6.18, 6.19, and 6.20. These tables focus on unbalanced data, as there was a 100% success rate in identifying non-poisoned nodes in the scenarios presented in Section 6.2. As depicted in these tables, incorrect classifications occurred only in the most complex scenarios, involving 7 to 9 nodes and higher levels of imbalance.

Table 6.17: Standard Model - Weighted average F1-score and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks. Only the differences from Table 6.5 are presented.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	0.714 (4)
5	0.730 (1)	0.663 (4)
6	0.689 (2)	0.705 (4)

Table 6.18: Pruning - Weighted average F1-score and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks. Only the differences from Table 6.6 are presented.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	0.712 (3)
5	-	0.664 (4)
6	0.697 (1)	0.707 (4)

Table 6.19: LiteRT - Weighted average F1-score and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks. Only the differences from Table 6.7 are presented.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	0.711 (3)
5	-	0.666 (4)
6	0.698 (1)	0.705 (4)

Table 6.20: Quantization - Weighted average F1-score and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks. Only the differences from Table 6.8 are presented.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	0.698 (3)
5	-	0.615 (4)
6	0.685 (1)	0.696 (4)

Nonetheless, in all scenarios, fewer than 50% of the nodes were misclassified, and the overall results were comparable to those presented in Section 6.3. Based on these findings, we conclude that this strategy is effective within the evaluated scenarios. However, it is important to recognize that the 70% threshold for the weighted average F1-score may need to be adjusted depending on the specifics of each real-world situation. In complex scenarios, modifying this value may be necessary to improve the accuracy of classifying truly poisoned and non-poisoned nodes.

6.5 Results Synthesis and Discussion

This chapter presents results demonstrating how a DFL system for audio-to-text performs in scenarios with protective measures against inference and poisoning attacks, including those involving unbalanced data distribution between nodes. It also compares these results with those of centralized systems.

The results of the inference attack protections implemented in the DFL system indicate that adding noise to the training and validation data has a significant impact on the

models' performance, particularly in comparison to various model compression strategies. Among these strategies, quantization had the most noticeable negative impact on performance. Interestingly, some outcomes from model compression techniques performed better than those without it, suggesting that the standard model may be slightly overfitted.

The results also suggest that the differences between standard pruning and pruning with the LiteRT framework are negligible. Although the differences may not be evident in the results presented, as they are limited to three decimal places, they can be observed in greater detail in the total results outlined in Appendix C. This occurs because, after converting to LiteRT, minor discrepancies in model predictions can arise due to factors such as numerical precision loss, quantization effects, or optimizations specific to the runtime environment. However, in practice, these variations are usually minimal, especially when using 32-bit float precision, as was the case.

In terms of resilience to unbalanced data, the global models demonstrated robust performance by effectively managing disparities in class representation at the local node level. Consistently good results were achieved by the global model, provided that the quantity of data across nodes remained relatively uniform.

Excluding noise scenarios, the global models' performance, measured by the weighted average F1-score, showed no decrease greater than 27.85% compared to their centralized counterparts. This level of reduction may be considered acceptable in practical applications, as it can be offset by adherence to emerging data sovereignty principles and by the improved data privacy provided to network participants. However, in situations where there is noise in the data, the performance declines were substantial, exceeding 50% in some cases. In such instances, it may be challenging to justify the use of this type of DFL system for practical applications, unless legal requirements mandate their use.

Lastly, regarding the strategy implemented to counter poisoning attacks, the results in this chapter demonstrate its success, as global models that employed these protective measures showed a significant improvement in performance compared to those without them. Furthermore, the strategy demonstrated its reliability in scenarios where no nodes were actually poisoned, accurately identifying legitimate nodes in nearly all situations. The only exception occurred with the most extreme results involving unbalanced data, where a few nodes were incorrectly classified as poisoned when they were not. However, even in this situation, fewer than 50% of the nodes were affected, and the overall results did not vary significantly, with some even showing improvement. Thus, the global model maintained its integrity in these scenarios.

6.6 Threats to Validity

Several limitations of this study should be acknowledged. First, the experiments relied on a single dataset. As a result, the generalizability of the conclusions may be limited, and the findings could differ when applied to other datasets. Additionally, each scenario was evaluated with only one test. Conducting multiple runs per scenario, such as 30 repetitions, would yield more statistically robust results.

Another limitation concerns the methodological choices made during experimentation. For example, the study exclusively examined the FedAvg aggregation method for the DFL system and employed the SGD optimizer for the evaluated CNN model. Alternative aggregation strategies and optimization algorithms, which could impact performance under varying conditions, were not explored. Moreover, only CNN models were assessed. Evaluating different types of AI models could uncover distinct behaviors and performance dynamics.

The handling of data imbalance in this work also presents some limitations. The experiments assessed data imbalance by analyzing the distribution of classes across nodes, ensuring each node contained approximately equal amounts of data. However, it did not consider scenarios in which nodes have significantly different quantities of data. This type of imbalance could greatly affect the dynamics of the DFL system and compromise the fairness of the global model aggregation.

In addition, the analysis of noise injection focused on applying noise to the training and validation datasets, while the test set remained unaffected. This design choice may limit the ability to accurately assess how noise affects model inference, as it encourages the model to learn the noise patterns during training rather than generalize to clean data.

Finally, the threshold value for classifying nodes in the poisoning attack defense mechanism was fixed. Conducting additional tests with varying thresholds could offer deeper insights into the balance between classification accuracy and system resilience.

7 Conclusion

This work designed and assessed a DFL system for audio-to-text conversion, integrating blockchain and decentralized storage to enhance privacy and trust among untrusted participants. It also examined the system's behavior under adverse conditions, including data imbalance, inference attacks, and poisoning attacks, with an emphasis on assessing its viability for practical applications.

This chapter provides an overview of this study. Section 7.1 outlines the main contributions, while Section 7.2 discusses potential future directions for researchers wishing to continue this line of research.

7.1 Key Contributions

This work makes several significant contributions to the field of decentralized, privacy-preserving machine learning. First, it introduces the design and implementation of a DFL system for audio-to-text conversion. This system integrates the Ethereum blockchain for decentralized consensus and the InterPlanetary File System for decentralized model storage. It was specifically developed to function in environments where participants cannot be managed, trusted, or predetermined.

To assess the viability of the proposed system, this study conducted a comprehensive evaluation of the system's behavior under challenging conditions, including protections against inference attacks, disproportionate class representation across nodes, and poisoning attacks. The inference protection strategies assessed included the injection of noise into the training data and a sequence of model compression techniques: standard pruning, conversion using the LiteRT tool, and quantization. The findings show that noise injection can significantly degrade model performance, with losses exceeding 50% in some scenarios. In contrast, model compression methods generally had a smaller impact. Among them, the full compression pipeline—combining pruning, LiteRT conversion, and quantization—resulted in the most pronounced performance degradation, while standard pruning and pruning with LiteRT led to comparatively minimal differences.

Moreover, this work demonstrates the DFL system's resilience to both data imbalance and poisoning attacks. In scenarios involving imbalanced data distributions, the system consistently maintained robust global model performance. The poisoning defense strategy also proved effective, significantly enhancing system integrity by accu-

rately classifying legitimate and compromised nodes in most cases, including scenarios where no actual poisoning occurred. Furthermore, the evaluation showed that, excluding scenarios involving noise injection, the performance degradation of the DFL system compared to centralized counterparts remained below 27.85%. These findings suggest that the trade-offs in performance may be acceptable when weighed against the benefits of improved data privacy and greater alignment with emerging data sovereignty principles.

To support reproducibility and further development, the complete source code for the DFL system and the experimental framework has been made available at [137]. Additionally, the results directory—containing all trained models and detailed information on the experiments conducted in this work—is outlined in Appendix C.

7.2 Future Directions

Building on the findings of this study, several promising directions can be pursued to expand and strengthen the research. To enhance the generalizability and statistical robustness of the results, upcoming studies should repeat the experiments across a wider range of datasets and conduct multiple runs for each scenario, averaging the results to minimize variance. Exploring datasets with diverse structures and domains could provide deeper insights into the model's behavior and adaptability. Additionally, it would be valuable to examine data imbalance not only in terms of class distribution across nodes but also concerning quantity imbalance, where nodes contain significantly differing amounts of data — a situation commonly encountered in real-world decentralized environments.

Future work could also explore alternative aggregation strategies beyond FedAvg, the use of different optimization algorithms beyond SGD, and variations in training parameters. Investigating the effects of these components may reveal more robust or efficient training dynamics across different DFL networks and data conditions. Similarly, experimenting with a broader range of AI models, including non-ANN architectures, could uncover distinct performance trade-offs or security implications.

In the context of protecting against inference attacks, it would be beneficial to evaluate additional techniques such as homomorphic encryption, differential privacy, and transfer learning. The presence of noise in the data resulted in performance drops of over 50% in some scenarios, while the model compression methods did not exhibit such declines. Therefore, a comparative analysis of the different defense mechanisms used against inference attacks would be valuable. This analysis could focus on their cost-effectiveness by examining the trade-off between performance degradation and privacy benefits. Such information can guide the real-world deployment of DFL systems.

In relation to protections against poisoning attacks, future research could investigate adaptive thresholding strategies. In this approach, the threshold used for classifying nodes would be dynamically adjusted based on system behavior or performance metrics. Additionally, conducting extensive experiments with various fixed thresholds could offer better insight into the trade-offs between classification accuracy, false positive rates, and overall system resilience.

An alternative line of research would involve encrypting data stored in IPFS to help ensure that access is restricted to DFL participants, thereby enhancing privacy guarantees. Exploring alternative decentralized storage solutions, like Arweave, and conducting comparative evaluations may provide valuable insights into the trade-offs between scalability and security.

Another relevant direction involves studying the effect of applying noise to the test datasets. This simulates situations where the global model performs inference on real-world data that has been pre-processed using the same noise injection techniques employed during training. This approach could potentially improve the global model's performance in scenarios that prioritize high privacy preservation.

On a final note, the results presented in this study indicate that, under specific conditions, DFL systems hold considerable promise for real-world audio-to-text applications. The findings demonstrate that, despite the inherent challenges of decentralization and adversarial conditions, a DFL system can achieve performance that is competitive with centralized alternatives, all while significantly improving data privacy and system resilience. Ultimately, this work contributes to the advancement of privacy-preserving, decentralized learning architectures — a vital step toward building resilient, secure, and user-driven AI ecosystems in the digital era.

Bibliographical references

[1] *General data protection regulation* (*gdpr*). [Online]. Available: https://gdpr-info.eu/ (visited on 04/09/2025).

[2] *California consumer privacy act* (*ccpa*). [Online]. Available: https://oag.ca.gov/privacy/ccpa (visited on 04/09/2025).

[3] B. McMahan and D. Ramage, *Federated learning: Collaborative machine learning without centralized training*, Apr. 2017. [Online]. Available: https://research.google/blog/federated-learning-collaborative-machine-learning-without-centralized-training-data/ (visited on 03/30/2025).

[4] *Learn how google improves speech models*. [Online]. Available: https://support.google.com/assistant/answer/11140942?hl (visited on 04/09/2025).

[5] A. M. Turing, "Computing machinery and intelligence," *Mind*, vol. LIX, no. 236, pp. 433–460, 1950. DOI: 10.1093/MIND/LIX.236.433. [Online]. Available: https://doi.org/10.1093/mind/LIX.236.433.

[6] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach* (*4th Edition*). Pearson, 2020, ISBN: 9780134610993. [Online]. Available: http://aima.cs.berkeley.edu/.

[7] C. Janiesch, P. Zschech, and K. Heinrich, "Machine learning and deep learning," *Electron. Mark.*, vol. 31, no. 3, pp. 685–695, 2021. DOI: 10.1007/S12525-021-00475-2. [Online]. Available: https://doi.org/10.1007/s12525-021-00475-2.

[8] Y. Bengio, A. C. Courville, and P. Vincent, "Representation learning: A review and new perspectives," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 8, pp. 1798–1828, 2013. DOI: 10.1109/TPAMI.2013.50. [Online]. Available: https://doi.org/10.1109/TPAMI.2013.50.

[9] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach* (*4th Edition*). Pearson, 2020, ISBN: 9780134610993. [Online]. Available: http://aima.cs.berkeley.edu/.

[10] B. Ghojogh and M. Crowley, "The theory behind overfitting, cross validation, regularization, bagging, and boosting: Tutorial," *CoRR*, vol. abs/1905.12787, 2019. arXiv: 1905.12787. [Online]. Available: http://arxiv.org/abs/1905.12787.

[11] A. Tharwat, "Classification assessment methods," *Applied Computing and Informatics*, vol. 17, pp. 168–192, 2018, ISSN: 22108327. DOI: 10.1016/j.aci.2018.08.003.

[12] P. J. Hardin and M. Shumway, "Statistical significance and normalized confusion matrices," *Photogrammetric Engineering & Remote Sensing*, vol. 63, no. 6, pp. 735–740, 1997.

[13] S. Mohammed, L. Budach, M. Feuerpfeil, *et al.*, "The effects of data quality on machine learning performance on tabular data," *Inf. Syst.*, vol. 132, p. 102 549, 2025. DOI: 10.1016/J.IS.2025.102549. [Online]. Available: https://doi.org/10.1016/j.is.2025.102549.

[14] W. S. McCulloch and W. H. Pitts, "A logical calculus of the ideas immanent in nervous activity," in *The Philosophy of Artificial Intelligence*, ser. Oxford readings in philosophy, M. A. Boden, Ed., Oxford University Press, 1990, pp. 22–39.

[15] I. J. Goodfellow, Y. Bengio, and A. C. Courville, *Deep Learning* (Adaptive computation and machine learning). MIT Press, 2016, ISBN: 978-0-262-03561-3. [Online]. Available: http://www.deeplearningbook.org/.

[16] J. R. Quinlan, "Induction of decision trees," *Mach. Learn.*, vol. 1, no. 1, pp. 81–106, 1986. DOI: 10.1023/A:1022643204877. [Online]. Available: https://doi.org/10.1023/A:1022643204877.

[17] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995. DOI: 10.1007/BF00994018. [Online]. Available: https://doi.org/10.1007/BF00994018.

[18] R. A. Mariette and Khanna, "Support vector machines for classification," in *Efficient Learning Machines: Theories, Concepts, and Applications for Engineers and System Designers*. Apress, 2015, pp. 39–66, ISBN: 978-1-4302-5990-9. DOI: 10.1007/978-1-4302-5990-9_3. [Online]. Available: https://doi.org/10.1007/978-1-4302-5990-9_3.

[19] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems 25: 26th Annual Conference on Neural Information Processing Systems 2012. Proceedings of a meeting held December 3-6, 2012, Lake Tahoe, Nevada, United States*, P. L. Bartlett, F. C. N. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, Eds., 2012, pp. 1106–1114. [Online]. Available: https://proceedings.neurips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html.

[20] A. M. Elbir, S. Coleri, A. K. Papazafeiropoulos, P. Kourtessis, and S. Chatzinotas, "A hybrid architecture for federated and centralized learning," *IEEE Trans. Cogn. Commun. Netw.*, vol. 8, no. 3, pp. 1529–1542, 2022. DOI: 10.1109/TCCN.2022.3181032. [Online]. Available: https://doi.org/10.1109/TCCN.2022.3181032.

[21] S. Wu, B. Yu, S. Liu, and Y. Zhu, "Autonomy 2.0: The quest for economies of scale," *CoRR*, vol. abs/2307.03973, 2023. DOI: 10.48550/ARXIV.2307.03973.

arXiv: 2307.03973. [Online]. Available: https://doi.org/10.48550/arXiv.2307.03973.

[22] O. Choudhury, A. Gkoulalas-Divanis, T. Salonidis, *et al.*, "Differential privacy-enabled federated learning for sensitive health data," *CoRR*, vol. abs/1910.02578, 2019. arXiv: 1910.02578. [Online]. Available: http://arxiv.org/abs/1910.02578.

[23] N. Truong, K. Sun, S. Wang, F. Guitton, and Y. Guo, "Privacy preservation in federated learning: An insightful survey from the GDPR perspective," *Comput. Secur.*, vol. 110, p. 102 402, 2021. DOI: 10.1016/J.COSE.2021.102402. [Online]. Available: https://doi.org/10.1016/j.cose.2021.102402.

[24] P. Schafhalter, A. Krentsel, J. E. Gonzalez, S. Ratnasamy, S. Shenker, and I. Stoica, "Bandwidth allocation for cloud-augmented autonomous driving," *CoRR*, vol. abs/2503.20127, 2025. DOI: 10.48550/ARXIV.2503.20127. arXiv: 2503.20127. [Online]. Available: https://doi.org/10.48550/arXiv.2503.20127.

[25] E. Bonabeau, M. Dorigo, and G. Theraulaz, *Swarm Intelligence - From Natural to Artificial Systems* (Santa Fe Institute Studies in the Sciences of Complexity). Oxford University Press, 1999, ISBN: 978-0-19-513159-8. [Online]. Available: http://ukcatalogue.oup.com/product/9780195131598.do.

[26] W. Shi, Z. Yu, F. Feng, X. He, and C. Xiong, "Efficient multi-agent system training with data influence-oriented tree search," *CoRR*, vol. abs/2502.00955, 2025. DOI: 10.48550/ARXIV.2502.00955. arXiv: 2502.00955. [Online]. Available: https://doi.org/10.48550/arXiv.2502.00955.

[27] S. K. Lo, Q. Lu, H.-Y. Paik, and L. Zhu, "Flra: A reference architecture for federated learning systems," *CoRR*, vol. abs/2106.11570, 2021. [Online]. Available: https://arxiv.org/abs/2106.11570 (visited on 03/31/2025).

[28] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, AISTATS 2017, 20-22 April 2017, Fort Lauderdale, FL, USA*, A. Singh and X. (Zhu, Eds., ser. Proceedings of Machine Learning Research, vol. 54, PMLR, 2017, pp. 1273–1282. [Online]. Available: http://proceedings.mlr.press/v54/mcmahan17a.html.

[29] T. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," *CoRR*, vol. abs/1909.06335, 2019. arXiv: 1909.06335. [Online]. Available: http://arxiv.org/abs/1909.06335.

[30] T. H. Hsu, H. Qi, and M. Brown, "Federated visual classification with real-world data distribution," in *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part X*, A. Vedaldi, H. Bischof, T. Brox, and J. Frahm, Eds., ser. Lecture Notes in Computer Science, vol. 12355,

Springer, 2020, pp. 76–92. DOI: 10.1007/978-3-030-58607-2_5. [Online]. Available: https://doi.org/10.1007/978-3-030-58607-2%5C_5.

[31] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proceedings of the Third Conference on Machine Learning and Systems, MLSys 2020, Austin, TX, USA, March 2-4, 2020*, I. S. Dhillon, D. S. Papailiopoulos, and V. Sze, Eds., mlsys.org, 2020. [Online]. Available: https://proceedings.mlsys.org/paper%5C_files/paper/2020/hash/1f5fe83998a09396ebe6477d9475ba0c-Abstract.html.

[32] *Health insurance portability and accountability act of 1996*. [Online]. Available: https://aspe.hhs.gov/reports/health-insurance-portability-accountability-act-1996 (visited on 04/09/2025).

[33] L. Atzori, A. Iera, and G. Morabito, "The internet of things: A survey," *Comput. Networks*, vol. 54, no. 15, pp. 2787–2805, 2010. DOI: 10.1016/J.COMNET.2010.05.010. [Online]. Available: https://doi.org/10.1016/j.comnet.2010.05.010.

[34] P. Kairouz, H. B. McMahan, B. Avent, *et al.*, "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1-2, pp. 1–210, 2021. DOI: 10.1561/2200000083. [Online]. Available: https://doi.org/10.1561/2200000083.

[35] J. Mirkovic and P. L. Reiher, "A taxonomy of ddos attack and ddos defense mechanisms," *Comput. Commun. Rev.*, vol. 34, no. 2, pp. 39–53, 2004. DOI: 10.1145/997150.997156. [Online]. Available: https://doi.org/10.1145/997150.997156.

[36] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *The 23rd International Conference on Artificial Intelligence and Statistics, AISTATS 2020, 26-28 August 2020, Online* [*Palermo, Sicily, Italy*], S. Chiappa and R. Calandra, Eds., ser. Proceedings of Machine Learning Research, vol. 108, PMLR, 2020, pp. 2938–2948. [Online]. Available: http://proceedings.mlr.press/v108/bagdasaryan20a.html.

[37] R. Gosselin, L. Vieu, F. Loukil, and A. Benoit, "Privacy and security in federated learning: A survey," *Applied Sciences*, vol. 12, 19 2022, ISSN: 2076-3417. DOI: 10.3390/app12199901. [Online]. Available: https://www.mdpi.com/2076-3417/12/19/9901.

[38] T. Krauß, J. König, A. Dmitrienko, and C. Kanzow, "Automatic adversarial adaption for stealthy poisoning attacks in federated learning," in *31st Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26 - March 1, 2024*, The Internet Society, 2024. [Online]. Available: https://www.ndss-symposium.org/ndss-paper/automatic-adversarial-adaption-for-stealthy-poisoning-attacks-in-federated-learning/.

[39] Y. Huang, S. Gupta, Z. Song, K. Li, and S. Arora, "Evaluating gradient inversion attacks and defenses in federated learning," in *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, M. Ranzato, A. Beygelzimer, Y. N. Dauphin, P. Liang, and J. W. Vaughan, Eds., 2021, pp. 7232–7241. [Online]. Available: https://proceedings.neurips.cc/paper/2021/hash/3b3fff6463464959dcd1b68d0320f781-Abstract.html.

[40] Z. Li, A. Lowy, J. Liu, *et al.*, "Analyzing inference privacy risks through gradients in machine learning," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie, Eds., ACM, 2024, pp. 3466–3480. DOI: 10.1145/3658644.3690304. [Online]. Available: https://doi.org/10.1145/3658644.3690304.

[41] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information and basic countermeasures," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, Denver, CO, USA, October 12-16, 2015*, I. Ray, N. Li, and C. Kruegel, Eds., ACM, 2015, pp. 1322–1333. DOI: 10.1145/2810103.2813677. [Online]. Available: https://doi.org/10.1145/2810103.2813677.

[42] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proceedings - IEEE Symposium on Security and Privacy*, Institute of Electrical and Electronics Engineers Inc., Jun. 2017, pp. 3–18. DOI: 10.1109/SP.2017.41.

[43] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, H. M. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. B. Fox, and R. Garnett, Eds., 2019, pp. 14747–14756. [Online]. Available: https://proceedings.neurips.cc/paper/2019/hash/60a6c4002cc7b29142def8871531281a-Abstract.html.

[44] N. Carlini, F. Tramèr, E. Wallace, *et al.*, "Extracting training data from large language models," in *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, M. D. Bailey and R. Greenstadt, Eds., USENIX Association, 2021, pp. 2633–2650. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/carlini-extracting.

[45] Q. Tan, Q. Li, Y. Zhao, Z. Liu, X. Guo, and K. Xu, "Defending against data reconstruction attacks in federated learning: An information theory approach," in *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*, D. Balzarotti and W. Xu, Eds., USENIX Associa-

tion, 2024. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/tan.

[46] R. Soltani, M. Zaman, R. Joshi, and S. Sampalli, "Distributed ledger technologies and their applications: A review," *Applied Sciences*, vol. 12, 15 2022, ISSN: 2076-3417. DOI: 10.3390/app12157898. [Online]. Available: https://www.mdpi.com/2076-3417/12/15/7898.

[47] C. Antal, T. Cioara, I. Anghel, M. Antal, and I. Salomie, "Distributed ledger technology review and decentralized applications development guidelines," *Future Internet*, vol. 13, 3 2021, ISSN: 1999-5903. DOI: 10.3390/fi13030062. [Online]. Available: https://www.mdpi.com/1999-5903/13/3/62.

[48] M. Haque, K. Upreti, A. Ohol, G. Pisolkar, P. Vats, and N. Kumar, "A comprehensive study of blockchain technology based decentralised ledger implementations," in *2023 10th IEEE Uttar Pradesh Section International Conference on Electrical, Electronics and Computer Engineering (UPCON)*, vol. 10, 2023, pp. 134–140. DOI: 10.1109/UPCON59197.2023.10434407.

[49] S. Nakamoto, *Bitcoin: A peer-to-peer electronic cash system*, 2008. [Online]. Available: https://dx.doi.org/10.2139/ssrn.3440802 (visited on 04/23/2025).

[50] V. Buterin *et al.*, *Ethereum: A next-generation smart contract and decentralized application platform*, 2014. [Online]. Available: https://ethereum.org/en/whitepaper/ (visited on 04/17/2025).

[51] M. Attaran and A. Gunasekaran, "Blockchain-enabled technology: The emerging technology set to reshape and decentralise many industries," *Int. J. Appl. Decis. Sci.*, vol. 12, no. 4, pp. 424–444, 2019. DOI: 10.1504/IJADS.2019.102642. [Online]. Available: https://doi.org/10.1504/IJADS.2019.102642.

[52] M. Loporchio, D. D. F. Maesa, A. Bernasconi, and L. Ricci, "Comparing ethereum fungible and non-fungible tokens: An analysis of transfer networks," *Appl. Netw. Sci.*, vol. 9, no. 1, p. 72, 2024. DOI: 10.1007/S41109-024-00682-8. [Online]. Available: https://doi.org/10.1007/s41109-024-00682-8.

[53] R. Asif and S. R. Hassan, "Shaping the future of ethereum: Exploring energy consumption in proof-of-work and proof-of-stake consensus," *Frontiers in Blockchain*, vol. 6, 2023, ISSN: 2624-7852. DOI: 10.3389/fbloc.2023.1151724. [Online]. Available: https://www.frontiersin.org/journals/blockchain/articles/10.3389/fbloc.2023.1151724 (visited on 03/30/2025).

[54] B. Liu, T. Prodromou, S. Suardi, and C. Xu, "Ethereum's merge: Market liquidity, efficiency and volatility in the proof of stake era," *Economics Letters*, vol. 247, p. 112 202, 2025, ISSN: 0165-1765. DOI: https://doi.org/10.1016/j.econlet.2025.112202. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0165176525000394.

[55] B. Lashkari and P. Musílek, "A comprehensive review of blockchain consensus mechanisms," *IEEE Access*, vol. 9, pp. 43 620–43 652, 2021. DOI: 10.1109/ACCESS.

2021.3065880. [Online]. Available: https://doi.org/10.1109/ACCESS.2021.3065880.

[56] G. Tripathi, M. A. Ahad, and G. Casalino, "A comprehensive review of blockchain technology: Underlying principles and historical background with future challenges," *Decision Analytics Journal*, vol. 9, p. 100 344, 2023, ISSN: 2772-6622. DOI: https://doi.org/10.1016/j.dajour.2023.100344. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2772662223001844.

[57] G. Huberman, J. D. Leshno, and C. Moallemi, "Monopoly without a monopolist: An economic analysis of the bitcoin payment system," *The Review of Economic Studies*, vol. 88, pp. 3011–3040, 6 Nov. 2021, ISSN: 0034-6527. DOI: 10.1093/restud/rdab014. [Online]. Available: https://doi.org/10.1093/restud/rdab014.

[58] A. Donmez and A. Karaivanov, "Transaction fee economics in the ethereum blockchain," *Economic Inquiry*, vol. 60, pp. 265–292, 1 2022. DOI: https://doi.org/10.1111/ecin.13025. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1111/ecin.13025.

[59] J. P. D. Richard and Disso, "An analysis into the scalability of bitcoin and ethereum," in *Third International Congress on Information and Communication Technology*, Simon, D. Nilanjan, J. A. Y. Xin-She, and Sherratt, Eds., Springer Singapore, 2019, pp. 619–627, ISBN: 978-981-13-1165-9.

[60] L. He, "A comparative examination of network and contract-based blockchain storage solutions for decentralized applications," *CoRR*, vol. abs/2401.07417, 2024. DOI: 10.48550/ARXIV.2401.07417. arXiv: 2401.07417. [Online]. Available: https://doi.org/10.48550/arXiv.2401.07417.

[61] J. Benet, "IPFS - content addressed, versioned, P2P file system," *CoRR*, vol. abs/1407.3561, 2014. arXiv: 1407.3561. [Online]. Available: http://arxiv.org/abs/1407.3561.

[62] D. Trautwein, A. Raman, G. Tyson, *et al.*, "Design and evaluation of IPFS: a storage layer for the decentralized web," in *SIGCOMM '22: ACM SIGCOMM 2022 Conference, Amsterdam, The Netherlands, August 22 - 26, 2022*, F. Kuipers and A. Orda, Eds., ACM, 2022, pp. 739–752. DOI: 10.1145/3544216.3544232. [Online]. Available: https://doi.org/10.1145/3544216.3544232.

[63] N. Sangeeta and S. Y. Nam, "Blockchain and interplanetary file system (ipfs)-based data storage system for vehicular networks with keyword search capability," *Electronics*, vol. 12, 7 2023, ISSN: 2079-9292. DOI: 10.3390/electronics12071545. [Online]. Available: https://www.mdpi.com/2079-9292/12/7/1545.

[64] J. Benet, *Filecoin: A decentralized storage network*, 2017. [Online]. Available: https://filecoin.io/filecoin.pdf (visited on 04/24/2025).

[65] *Filecoin mainnet is live*, Oct. 2020. [Online]. Available: https://filecoin.io/blog/posts/filecoin-mainnet-is-live/ (visited on 04/24/2025).

[66] S. Williams and W. Jones, *Archain: An open, irrevocable, unforgeable and uncensorable archive for the internet*, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:53050295 (visited on 04/24/2025).

[67] S. Williams, A. Kedia, L. Berman, and S. Campos-Groth, *Arweave: The permanent information storage protocol*, 2023. [Online]. Available: https://www.arweave.org/files/arweave-lightpaper.pdf (visited on 04/24/2025).

[68] *Arweave network launch report*, Jun. 2018. [Online]. Available: https://arweave.medium.com/arweave-network-launch-report-b7e7ffac0f75 (visited on 04/24/2025).

[69] *Welcome to arfleet documentation!* [Online]. Available: https://docs.arfleet.io/docs/home (visited on 04/24/2025).

[70] L. Yuan, Z. Wang, L. Sun, P. S. Yu, and C. G. Brinton, "Decentralized federated learning: A survey and perspective," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34 617–34 638, 2024. DOI: 10.1109/JIOT.2024.3407584. [Online]. Available: https://doi.org/10.1109/JIOT.2024.3407584.

[71] Y. Qu, L. Gao, T. H. Luan, *et al.*, "Decentralized privacy using blockchain-enabled federated learning in fog computing," *IEEE Internet of Things Journal*, vol. 7, pp. 5171–5183, Jun. 2020, ISSN: 23274662. DOI: 10.1109/JIOT.2020.2977383.

[72] D. Pasquini, M. Raynal, and C. Troncoso, "On the (in)security of peer-to-peer decentralized machine learning," in *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*, IEEE, 2023, pp. 418–436. DOI: 10.1109/SP46215.2023.10179291. [Online]. Available: https://doi.org/10.1109/SP46215.2023.10179291.

[73] M. Fang, Z. Zhang, Hairi, *et al.*, "Byzantine-robust decentralized federated learning," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie, Eds., ACM, 2024, pp. 2874–2888. DOI: 10.1145/3658644.3670307. [Online]. Available: https://doi.org/10.1145/3658644.3670307.

[74] R. Zheng, L. Qu, T. Chen, K. Zheng, Y. Shi, and H. Yin, "Poisoning decentralized collaborative recommender system and its countermeasures," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2024, Washington DC, USA, July 14-18, 2024*, G. H. Yang, H. Wang, S. Han, C. Hauff, G. Zuccon, and Y. Zhang, Eds., ACM, 2024, pp. 1712–1721. DOI: 10.1145/3626772.3657814. [Online]. Available: https://doi.org/10.1145/3626772.3657814.

[75] B. Li, X. Miao, Y. Shang, X. Zhao, S. Deng, and J. Yin, "Gradient purification: Defense against poisoning attack in decentralized federated learning," *CoRR*, vol. abs/2501.04453, 2025. DOI: 10.48550/ARXIV.2501.04453. arXiv: 2501.04453. [Online]. Available: https://doi.org/10.48550/arXiv.2501.04453.

[76] G. Xia, J. Chen, C. Yu, and J. Ma, "Poisoning attacks in federated learning: A survey," *IEEE Access*, vol. 11, pp. 10 708–10 722, 2023. DOI: 10.1109/ACCESS.2023.3238823. [Online]. Available: https://doi.org/10.1109/ACCESS.2023.3238823.

[77] R. Thennakoon, A. Wanigasundara, S. Weerasinghe, C. Seneviratne, Y. Siriwardhana, and M. Liyanage, "Decentralized defense: Leveraging blockchain against poisoning attacks in federated learning systems," in *2024 IEEE 21st Consumer Communications & Networking Conference (CCNC)*, 2024, pp. 950–955. DOI: 10.1109/CCNC51664.2024.10454688.

[78] Y. Ren, M. Hu, Z. Yang, G. Feng, and X. Zhang, "Bpfl: Blockchain-based privacy-preserving federated learning against poisoning attack," *Information Sciences*, vol. 665, p. 120 377, 2024, ISSN: 0020-0255. DOI: https://doi.org/10.1016/j.ins.2024.120377. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0020025524002901.

[79] N. Dong, Z. Wang, J. Sun, M. Kampffmeyer, W. J. Knottenbelt, and E. P. Xing, "Defending against poisoning attacks in federated learning with blockchain," *IEEE Trans. Artif. Intell.*, vol. 5, no. 7, pp. 3743–3756, 2024. DOI: 10.1109/TAI.2024.3376651. [Online]. Available: https://doi.org/10.1109/TAI.2024.3376651.

[80] H. Zhu, R. Wang, Y. Jin, K. Liang, and J. Ning, "Distributed additive encryption and quantization for privacy preserving federated deep learning," *Neurocomputing*, vol. 463, pp. 309–327, 2020. [Online]. Available: https://api.semanticscholar.org/CorpusID:227162344.

[81] D. Stripelis, H. Saleem, T. Ghai, *et al.*, "Secure neuroimaging analysis using federated learning with homomorphic encryption," in *Symposium on Medical Information Processing and Analysis*, 2021. [Online]. Available: https://api.semanticscholar.org/CorpusID:236956474.

[82] G. D. Németh, M. A. Lozano, N. Quadrianto, and N. Oliver, "Addressing membership inference attack in federated learning with model compression," *CoRR*, vol. abs/2311.17750, 2023. DOI: 10.48550/ARXIV.2311.17750. arXiv: 2311.17750. [Online]. Available: https://doi.org/10.48550/arXiv.2311.17750.

[83] M. J. Mia and M. H. Amini, "Quancrypt-fl: Quantized homomorphic encryption with pruning for secure federated learning," *CoRR*, vol. abs/2411.05260, 2024. DOI: 10.48550/ARXIV.2411.05260. arXiv: 2411.05260. [Online]. Available: https://doi.org/10.48550/arXiv.2411.05260.

[84] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, no. 3-4, pp. 211–407, 2014. DOI: 10.1561/0400000042. [Online]. Available: https://doi.org/10.1561/0400000042.

[85] P. V. Dantas, W. S. da Silva Jr., L. C. Cordeiro, and C. B. Carvalho, "A comprehensive review of model compression techniques in machine learning," *Appl. Intell.*, vol. 54, no. 22, pp. 11 804–11 844, 2024. DOI: 10.1007/S10489-024-05747-W. [Online]. Available: https://doi.org/10.1007/s10489-024-05747-w.

[86] Y. LeCun, J. S. Denker, and S. A. Solla, "Optimal brain damage," in *Advances in Neural Information Processing Systems 2, [NIPS Conference, Denver, Colorado, USA, November 27-30, 1989]*, D. S. Touretzky, Ed., Morgan Kaufmann, 1989, pp. 598–605. [Online]. Available: http://papers.nips.cc/paper/250-optimal-brain-damage.

[87] A. Finlinson and S. Moschoyiannis, "Synthesis and pruning as a dynamic compression strategy for efficient deep neural networks," in *From Data to Models and Back - 9th International Symposium, DataMod 2020, Virtual Event, October 20, 2020, Revised Selected Papers*, J. Bowles, G. Broccia, and M. Nanni, Eds., ser. Lecture Notes in Computer Science, vol. 12611, Springer, 2020, pp. 3–17. DOI: 10.1007/978-3-030-70650-0_1. [Online]. Available: https://doi.org/10.1007/978-3-030-70650-0%5C_1.

[88] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," *CoRR*, vol. abs/1806.08342, 2018. arXiv: 1806.08342. [Online]. Available: http://arxiv.org/abs/1806.08342.

[89] X. Zhao, R. Xu, and X. Guo, "Post-training quantization or quantization-aware training? that is the question," in *2023 China Semiconductor Technology International Conference (CSTIC)*, 2023, pp. 1–3. DOI: 10.1109/CSTIC58779.2023.10219214.

[90] G. E. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *CoRR*, vol. abs/1503.02531, 2015. arXiv: 1503.02531. [Online]. Available: http://arxiv.org/abs/1503.02531.

[91] C. Gentry, "A fully homomorphic encryption scheme," Ph.D. dissertation, Stanford University, USA, 2009. [Online]. Available: https://searchworks.stanford.edu/view/8493082.

[92] N. Dowlin, R. Gilad-Bachrach, K. Laine, K. E. Lauter, M. Naehrig, and J. R. Wernsing, "Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proceedings of the 33rd International Conference on International Conference on Machine Learning - Volume 48*, ser. ICML'16, New York, NY, USA: JMLR.org, 2016, pp. 201–210.

[93] C. Korkmaz, H. E. Kocas, A. Uysal, A. Masry, O. Ozkasap, and B. Akgun, "Chain fl: Decentralized federated machine learning via blockchain," in *2020 2nd International Conference on Blockchain Computing and Applications, BCCA*

2020, Institute of Electrical and Electronics Engineers Inc., Nov. 2020, pp. 140–146. DOI: 10.1109/BCCA50787.2020.9274451.

[94] P. Ramanan and K. Nakayama, "Baffle : Blockchain based aggregator free federated learning," in *Proceedings - 2020 IEEE International Conference on Blockchain, Blockchain 2020*, Institute of Electrical and Electronics Engineers Inc., Nov. 2020, pp. 72–81. DOI: 10.1109/Blockchain50366.2020.00017.

[95] Ravi, K. L. M. Vaikkunth, and Rahman, "Blockflow: Decentralized, privacy-preserving, and accountable federated machine learning," in *Blockchain and Applications*, Alberto, L. Paulo, P. A. P. Javier, and Partida, Eds., Springer International Publishing, 2022, pp. 233–242, ISBN: 978-3-030-86162-9.

[96] S. Teerapittayanon and H. T. Kung, "Daimon: A decentralized artificial intelligence model network," in *Proceedings - 2019 2nd IEEE International Conference on Blockchain, Blockchain 2019*, Institute of Electrical and Electronics Engineers Inc., Jul. 2019, pp. 132–139. DOI: 10.1109/Blockchain.2019.00026.

[97] Y. Zhao, J. Zhao, L. Jiang, *et al.*, *Privacy-preserving blockchain-based federated learning for iot devices*, 2021. arXiv: 1906.10893 [cs.CR]. [Online]. Available: https://arxiv.org/abs/1906.10893.

[98] Y. Lu, X. Huang, Y. Dai, S. Maharjan, and Y. Zhang, "Blockchain and federated learning for privacy-preserved data sharing in industrial iot," *IEEE Transactions on Industrial Informatics*, vol. 16, pp. 4177–4186, Jun. 2020, ISSN: 19410050. DOI: 10.1109/TII.2019.2942190.

[99] Y. Hu, Y. Zhou, J. Xiao, and C. Wu, *Gfl: A decentralized federated learning framework based on blockchain*, 2021. arXiv: 2010.10996 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2010.10996.

[100] Y. Lu, X. Huang, K. Zhang, S. Maharjan, and Y. Zhang, "Blockchain empowered asynchronous federated learning for secure data sharing in internet of vehicles," *IEEE Transactions on Vehicular Technology*, vol. 69, pp. 4298–4311, Apr. 2020, ISSN: 19399359. DOI: 10.1109/TVT.2020.2973651.

[101] N. Q. Hieu, T. T. Anh, N. C. Luong, D. Niyato, D. I. Kim, and E. Elmroth, *Resource management for blockchain-enabled federated learning: A deep reinforcement learning approach*, 2020. arXiv: 2004.04104 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2004.04104.

[102] Zehui, J. Chunxiao, L. Yi, *et al.*, "Scalable and communication-efficient decentralized federated edge learning with multi-blockchain framework," in *Blockchain and Trustworthy Systems*, Hong-Ning, F. Xiaodong, C. B. Z. Zibin, and Dai, Eds., Springer Singapore, 2020, pp. 152–165, ISBN: 978-981-15-9213-3.

[103] Y. Qu, L. Gao, T. H. Luan, *et al.*, "Decentralized privacy using blockchain-enabled federated learning in fog computing," *IEEE Internet of Things Journal*, vol. 7, pp. 5171–5183, Jun. 2020, ISSN: 23274662. DOI: 10.1109/JIOT.2020.2977383.

[104] A. Mandelbaum and A. Shalev, *Word embeddings and their use in sentence classification tasks*, 2016. arXiv: 1610.08229 [cs.LG]. [Online]. Available: https://arxiv.org/abs/1610.08229.

[105] H. Kim, J. Park, M. Bennis, and S. L. Kim, "Blockchained on-device federated learning," *IEEE Communications Letters*, vol. 24, pp. 1279–1283, Jun. 2020, ISSN: 15582558. DOI: 10.1109/LCOMM.2019.2921755.

[106] J. Weng, J. Weng, J. Zhang, M. Li, Y. Zhang, and W. Luo, "Deepchain: Auditable and privacy-preserving deep learning with blockchain-based incentive," *IEEE Transactions on Dependable and Secure Computing*, pp. 1–1, Nov. 2019, ISSN: 1545-5971. DOI: 10.1109/tdsc.2019.2952332.

[107] U. Majeed and C. S. Hong, "Flchain: Federated learning via mec-enabled blockchain network," in *2019 20th Asia-Pacific Network Operations and Management Symposium: Management in a Cyber-Physical World, APNOMS 2019*, Institute of Electrical and Electronics Engineers Inc., Sep. 2019. DOI: 10.23919/APNOMS.2019.8892848.

[108] J. Kang, Z. Xiong, D. Niyato, S. Xie, and J. Zhang, "Incentive mechanism for reliable federated learning: A joint optimization approach to combining reputation and contract theory," *IEEE Internet of Things Journal*, vol. 6, pp. 10 700–10 714, Dec. 2019, ISSN: 23274662. DOI: 10.1109/JIOT.2019.2940820.

[109] J. Li, Y. Shao, K. Wei, *et al.*, "Blockchain assisted decentralized federated learning (blade-fl): Performance analysis and resource allocation," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, pp. 2401–2415, Oct. 2022, ISSN: 15582183. DOI: 10.1109/TPDS.2021.3138848.

[110] Y. Li, C. Chen, N. Liu, H. Huang, Z. Zheng, and Q. Yan, "A blockchain-based decentralized federated learning framework with committee consensus," *IEEE Network*, vol. 35, pp. 234–241, Mar. 2021, ISSN: 1558156X. DOI: 10.1109/MNET.011.2000263.

[111] H. Chen, S. A. Asif, J. Park, C.-C. Shen, and M. Bennis, *Robust blockchained federated learning with model validation and proof-of-stake inspired consensus*, 2021. arXiv: 2101.03300 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2101.03300.

[112] S. R. Pokhrel and J. Choi, "Federated learning with blockchain for autonomous vehicles: Analysis and design challenges," *IEEE Transactions on Communications*, vol. 68, pp. 4734–4746, Aug. 2020, ISSN: 15580857. DOI: 10.1109/TCOMM.2020.2990686.

[113] H. B. Desai, M. S. Ozdayi, and M. Kantarcioglu, "Blockfla: Accountable federated learning via hybrid blockchain architecture," in *Proceedings of the Eleventh ACM Conference on Data and Application Security and Privacy*, ser. CODASPY '21, Virtual Event, USA: Association for Computing Machinery, 2021, pp. 101–

112, ISBN: 9781450381437. DOI: 10.1145/3422337.3447837. [Online]. Available: https://doi.org/10.1145/3422337.3447837.

[114] J. Poushter, *Smartphone ownership and internet usage continues to climb in emerging economies, pew research center's global attitudes project*, Feb. 2016. [Online]. Available: https://www.pewresearch.org/global/2016/02/22/smartphone-ownership-and-internet-usage-continues-to-climb-in-emerging-economies/ (visited on 04/16/2025).

[115] C. Pappas, D. Chatzopoulos, S. Lalis, and M. Vavalis, "Ipls: A framework for decentralized federated learning," in *2021 IFIP Networking Conference, IFIP Networking 2021*, Institute of Electrical and Electronics Engineers Inc., Jun. 2021. DOI: 10.23919/IFIPNetworking52078.2021.9472790.

[116] A. Fadaeddini, B. Majidi, and M. Eshghi, "Secure decentralized peer-to-peer training of deep neural networks based on distributed ledger technology," *The Journal of Supercomputing*, vol. 76, pp. 10 354–10 368, 12 2020, ISSN: 1573-0484. DOI: 10.1007/s11227-020-03251-9. [Online]. Available: https://doi.org/10.1007/s11227-020-03251-9.

[117] A. Nagar, *Privacy-preserving blockchain based federated learning with differential data sharing*, 2019. arXiv: 1912.04859 [cs.CR]. [Online]. Available: https://arxiv.org/abs/1912.04859.

[118] P. Rimba, A. B. Tran, I. Weber, M. Staples, A. Ponomarev, and X. Xu, "Comparing blockchain and cloud services for business process execution," in *Proceedings - 2017 IEEE International Conference on Software Architecture, ICSA 2017*, Institute of Electrical and Electronics Engineers Inc., May 2017, pp. 257–260. DOI: 10.1109/ICSA.2017.44.

[119] M. Shayan, C. Fung, C. J. M. Yoon, and I. Beschastnikh, *Biscotti: A ledger for private and secure peer-to-peer machine learning*, 2019. arXiv: 1811.09904 [cs.LG]. [Online]. Available: https://arxiv.org/abs/1811.09904.

[120] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," *CoRR*, vol. abs/1804.03209, 2018. [Online]. Available: http://arxiv.org/abs/1804.03209.

[121] D. Amodei, S. Ananthanarayanan, R. Anubhai, *et al.*, "Deep speech 2 : End-to-end speech recognition in english and mandarin," in *Proceedings of the 33nd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, M. Balcan and K. Q. Weinberger, Eds., ser. JMLR Workshop and Conference Proceedings, vol. 48, JMLR.org, 2016, pp. 173–182. [Online]. Available: http://proceedings.mlr.press/v48/amodei16.html.

[122] R. Collobert, C. Puhrsch, and G. Synnaeve, "Wav2letter: An end-to-end convnet-based speech recognition system," *CoRR*, vol. abs/1609.03193, 2016. arXiv: 1609.03193. [Online]. Available: http://arxiv.org/abs/1609.03193.

[123] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, AISTATS 2017, 20-22 April 2017, Fort Lauderdale, FL, USA*, A. Singh and X. (Zhu, Eds., ser. Proceedings of Machine Learning Research, vol. 54, PMLR, 2017, pp. 1273–1282. [Online]. Available: http://proceedings.mlr.press/v54/mcmahan17a.html.

[124] Z. Zhao, C. Feng, W. Hong, *et al.*, "Federated learning with non-iid data in wireless networks," *IEEE Trans. Wirel. Commun.*, vol. 21, no. 3, pp. 1927–1942, 2022. DOI: 10.1109/TWC.2021.3108197. [Online]. Available: https://doi.org/10.1109/TWC.2021.3108197.

[125] C. Connors and D. Sarkar, "Review of most popular open-source platforms for developing blockchains," in *2022 Fourth International Conference on Blockchain Computing and Applications (BCCA)*, 2022, pp. 20–26. DOI: 10.1109/BCCA55292.2022.9922396.

[126] *Ethereum leads in dapp fee revenue in q1 2025*, Apr. 2025. [Online]. Available: https://beincrypto.com/ethereum-leads-in-dapp-fee-revenue-in-q1-2025/ (visited on 04/17/2025).

[127] *Ethereum's energy expenditure*, Oct. 2023. [Online]. Available: https://ethereum.org/en/energy-consumption/ (visited on 04/17/2025).

[128] C. Connors and D. Sarkar, "Review of most popular open-source platforms for developing blockchains," in *2022 4th International Conference on Blockchain Computing and Applications, BCCA 2022*, Institute of Electrical and Electronics Engineers Inc., 2022, pp. 20–26. DOI: 10.1109/BCCA55292.2022.9922396.

[129] *2024 developer report*. [Online]. Available: https://www.developerreport.com/ (visited on 04/17/2025).

[130] *Pension funds dabble in crypto after massive bitcoin rally*, Jan. 2025. [Online]. Available: https://www.ft.com/content/4146fbca-2930-41ef-965d-9cc8ea1d9aaf/ (visited on 04/17/2025).

[131] A. Fadaeddini, B. Majidi, and M. Eshghi, "Secure decentralized peer-to-peer training of deep neural networks based on distributed ledger technology," *The Journal of Supercomputing*, vol. 76, pp. 10 354–10 368, 12 2020, ISSN: 1573-0484. DOI: 10.1007/s11227-020-03251-9. [Online]. Available: https://doi.org/10.1007/s11227-020-03251-9.

[132] P. Kostamis, A. Sendros, and P. S. Efraimidis, "Data management in ethereum dapps: A cost and performance analysis," *Future Generation Computer Systems*, vol. 153, pp. 193–205, 2024, ISSN: 0167-739X. DOI: https://doi.org/10.1016/j.future.2023.11.026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167739X23004351.

[133] *Case study: Snapshot*. [Online]. Available: https://docs.ipfs.tech/case-studies/snapshot/#overview (visited on 03/30/2025).

[134] B. Hájek and F. Masár, *Federated learning token*, Nov. 2021. [Online]. Available: https://devpost.com/software/federated-learning-token (visited on 03/30/2025).

[135] B. Hájek and F. Masár, *Felt - federated learning token*, Nov. 2021. [Online]. Available: https://github.com/FELT-Labs/federated-learning-token (visited on 03/30/2025).

[136] B. Hájek and F. Masár, *Felt architecture | felt labs*. [Online]. Available: https://docs.feltlabs.ai/fundamentals/felt-architecture (visited on 03/30/2025).

[137] T. Ferreira, *Decentralized federated learning study based on distributed ledgers*, 2025. [Online]. Available: https://github.com/PTiagorio/dfl-study-based-on-dl/ (visited on 04/30/2025).

[138] N. Papernot, S. Song, I. Mironov, A. Raghunathan, K. Talwar, and Ú. Erlingsson, "Scalable private learning with PATE," in *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*, OpenReview.net, 2018. [Online]. Available: https://openreview.net/forum?id=rkZB1XbRZ.

[139] *Google colaboratory*. [Online]. Available: https://colab.google/ (visited on 03/30/2025).

[140] *Speech commands example*. [Online]. Available: https://github.com/tensorflow/tensorflow/tree/master/tensorflow/examples/speech_commands (visited on 03/30/2025).

[141] M. Abadi, A. Agarwal, P. Barham, *et al.*, "Tensorflow: Large-scale machine learning on heterogeneous distributed systems," *CoRR*, vol. abs/1603.04467, 2016. arXiv: 1603.04467. [Online]. Available: http://arxiv.org/abs/1603.04467.

[142] *An end-to-end platform for machine learning*. [Online]. Available: https://www.tensorflow.org/ (visited on 04/26/2025).

[143] *Simple audio recognition: Recognizing keywords*. [Online]. Available: https://www.tensorflow.org/tutorials/audio/simple_audio (visited on 03/30/2025).

[144] L. Bottou, F. E. Curtis, and J. Nocedal, "Optimization methods for large-scale machine learning," *SIAM Review*, vol. 60, pp. 223–311, 2 2018. DOI: 10.1137/16M1080173. [Online]. Available: https://doi.org/10.1137/16M1080173.

[145] *Speech command recognizer*, 2021. [Online]. Available: https://www.npmjs.com/package/@tensorflow-models/speech-commands/ (visited on 04/26/2025).

[146] *Speech command recognizer*. [Online]. Available: https://github.com/tensorflow/tfjs-models/tree/master/speech-commands (visited on 03/30/2025).

[147] *Converting a tensorflow.js speech-commands model to python and tflite formats*. [Online]. Available: https://github.com/tensorflow/tfjs-models/blob/master/speech-commands/training/browser-fft/tflite_conversion.ipynb (visited on 04/27/2025).

[148] *Keras*. [Online]. Available: https://github.com/tensorflow/tensorflow/tree/master/tensorflow/examples/speech_commands (visited on 03/30/2025).

[149] *Python*. [Online]. Available: https://www.python.org/ (visited on 03/30/2025).

[150] *Numpy*. [Online]. Available: https://numpy.org/ (visited on 03/30/2025).

[151] *Pruning in keras example*. [Online]. Available: https://www.tensorflow.org/model_optimization/guide/pruning/pruning_with_keras (visited on 03/30/2025).

[152] *Litert*. [Online]. Available: https://ai.google.dev/edge/litert (visited on 03/30/2025).

[153] *Numpy*. [Online]. Available: https://github.com/google-ai-edge/LiteRT (visited on 03/30/2025).

[154] *Onnx runtime*. [Online]. Available: https://onnxruntime.ai/ (visited on 04/17/2025).

[155] *Nvidia tensorrt*. [Online]. Available: https://developer.nvidia.com/tensorrt (visited on 04/17/2025).

[156] *Quantization aware training*. [Online]. Available: https://www.tensorflow.org/model_optimization/guide/quantization/training.md (visited on 03/30/2025).

[157] C. M. Bishop, "Training with noise is equivalent to tikhonov regularization," *Neural Comput.*, vol. 7, no. 1, pp. 108–116, 1995. DOI: 10.1162/NECO.1995.7.1.108. [Online]. Available: https://doi.org/10.1162/neco.1995.7.1.108.

[158] T. Ferreira, *Dfl study based on dl results*, 2025. [Online]. Available: https://bit.ly/DFL_Study_Based_on_DL (visited on 04/30/2025).

[159] T. Ferreira, *Dfl study based on dl results*, 2025. [Online]. Available: https://linktr.ee/DFL_Study_Based_on_DL (visited on 04/30/2025).

Appendices

Appendix A - Complementary Results to Chapter 6

This appendix provides additional results that complement the tables and figures presented in Chapter 6. While Chapter 6 focuses on the weighted averaging F1-score results, this section includes the results from the same experiments for cross-entropy loss, weighted average precision, weighted average recall, and accuracy.

The appendix is organized into 20 sections, each corresponding to a specific table or figure from Chapter 6. Consistent with the rest of the document, captions for the tables are placed above them, while captions for the figures are positioned below.

A.1 Complements to Table 6.1 and Figure 6.1

Table A.1: Standard Model - Cross-entropy loss considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.336	0.864	0.876	1.090	0.912
0.20	0.617	1.057	1.364	1.475	1.553
0.40	0.743	1.508	1.536	1.664	1.752
0.60	0.877	1.709	1.789	1.799	1.889
0.80	1.017	1.764	1.912	1.946	2.066
1.00	1.072	1.687	1.969	2.071	2.173

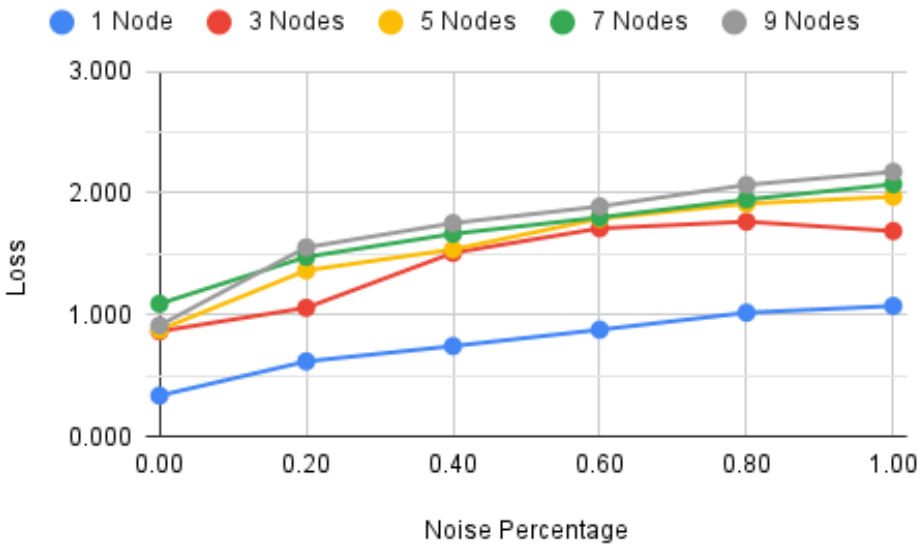


Figure A.1: Standard Model - Cross-entropy loss considering noise in the data.

Table A.2: Standard Model - Weighted average precision considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.897	0.806	0.786	0.754	0.766
0.20	0.832	0.759	0.718	0.696	0.669
0.40	0.813	0.694	0.681	0.658	0.626
0.60	0.787	0.652	0.634	0.614	0.593
0.80	0.758	0.646	0.616	0.580	0.569
1.00	0.737	0.635	0.592	0.566	0.540

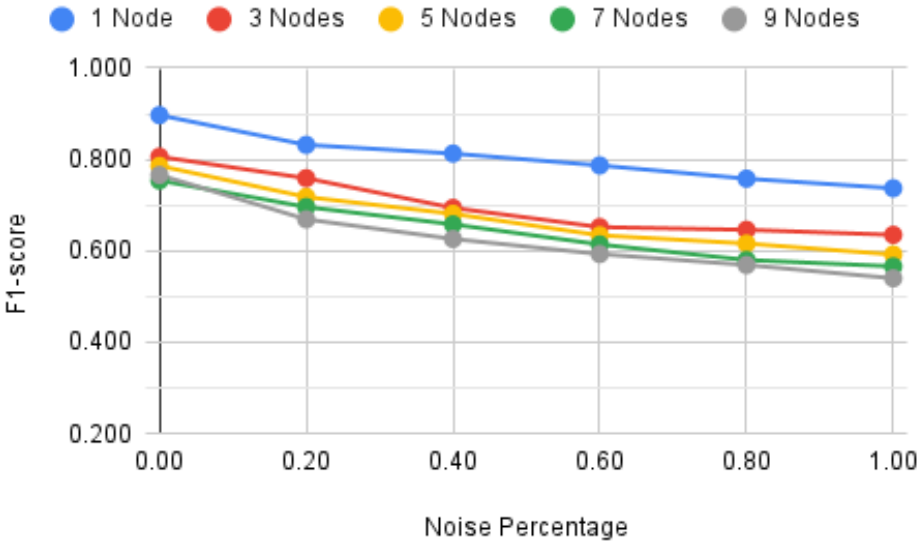


Figure A.2: Standard Model - Weighted average precision considering noise in the data.

Table A.3: Standard Model - Weighted average recall considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.896	0.793	0.772	0.738	0.762
0.20	0.821	0.742	0.678	0.648	0.595
0.40	0.791	0.632	0.623	0.555	0.517
0.60	0.746	0.557	0.509	0.513	0.471
0.80	0.707	0.494	0.469	0.449	0.398
1.00	0.698	0.507	0.450	0.392	0.358

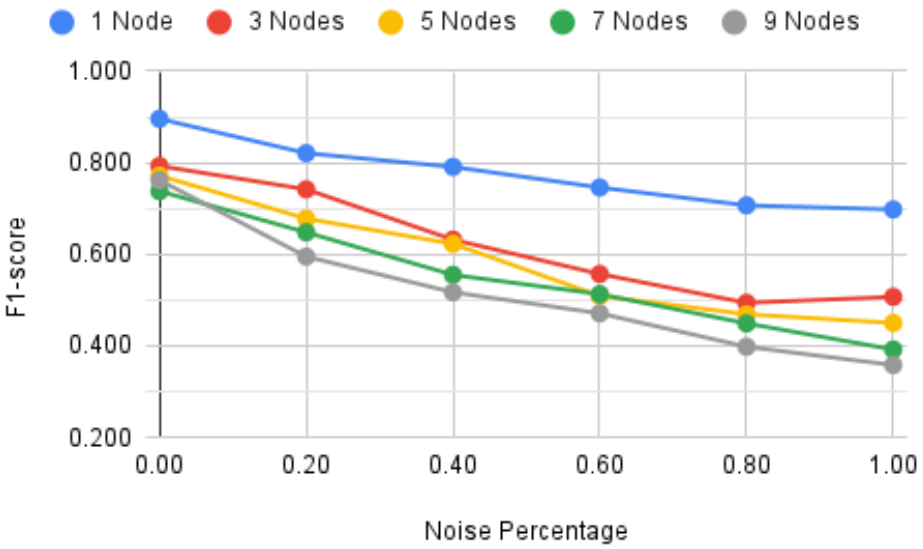


Figure A.3: Standard Model - Weighted average recall considering noise in the data.

Table A.4: Standard Model - Accuracy considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.896	0.793	0.772	0.738	0.762
0.20	0.821	0.742	0.678	0.648	0.595
0.40	0.791	0.632	0.623	0.555	0.517
0.60	0.746	0.557	0.509	0.513	0.471
0.80	0.707	0.494	0.469	0.449	0.398
1.00	0.698	0.507	0.450	0.392	0.358

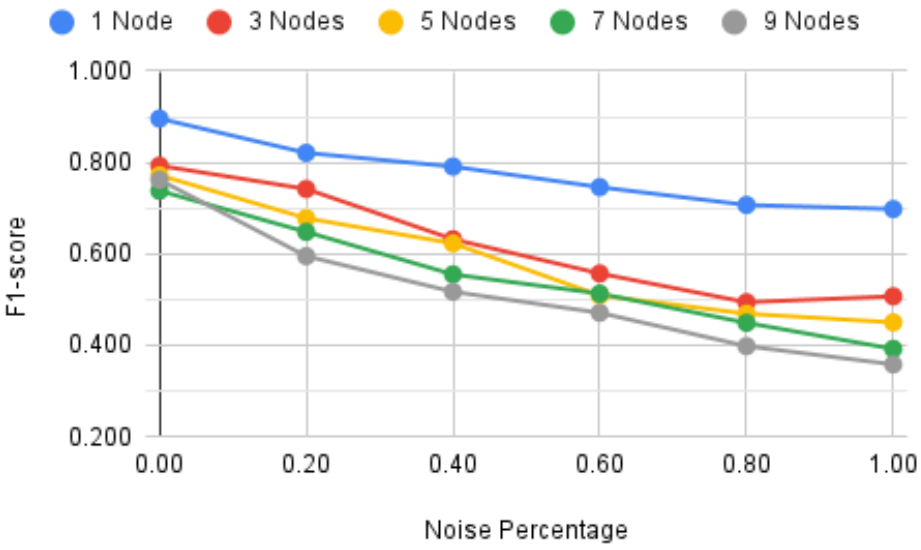


Figure A.4: Standard Model - Accuracy considering noise in the data.

A.2 Complements to Table 6.2 and Figure 6.2

Table A.5: Pruning - Cross-entropy loss considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.308	0.807	0.834	1.042	0.878
0.20	0.573	0.972	1.321	1.416	1.506
0.40	0.728	1.465	1.475	1.608	1.703
0.60	0.857	1.651	1.710	1.747	1.834
0.80	0.982	1.742	1.852	1.931	2.018
1.00	1.088	1.692	1.953	2.053	2.136

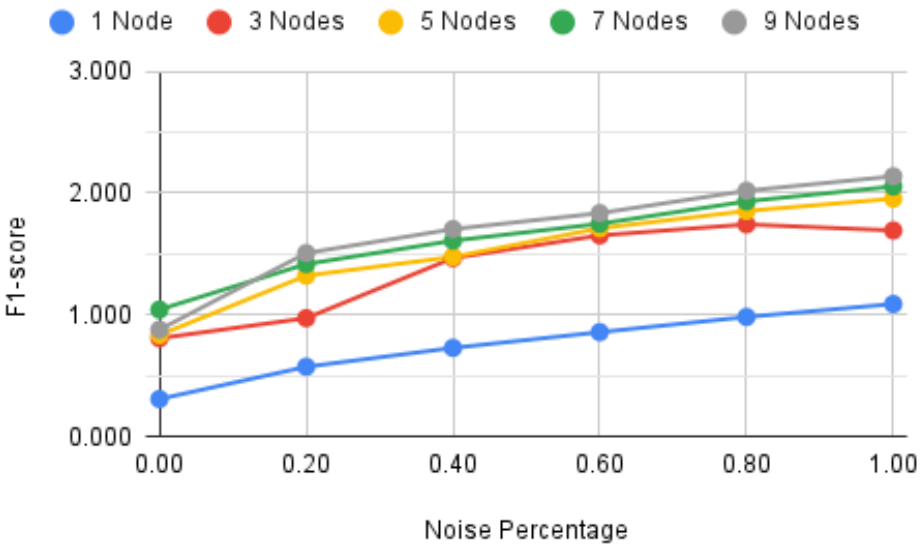


Figure A.5: Pruning - Cross-entropy loss considering noise in the data.

Table A.6: Pruning - Weighted average precision considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.907	0.817	0.793	0.754	0.770
0.20	0.842	0.783	0.718	0.699	0.670
0.40	0.815	0.691	0.690	0.658	0.629
0.60	0.784	0.657	0.638	0.619	0.596
0.80	0.762	0.652	0.618	0.584	0.573
1.00	0.735	0.634	0.592	0.563	0.544

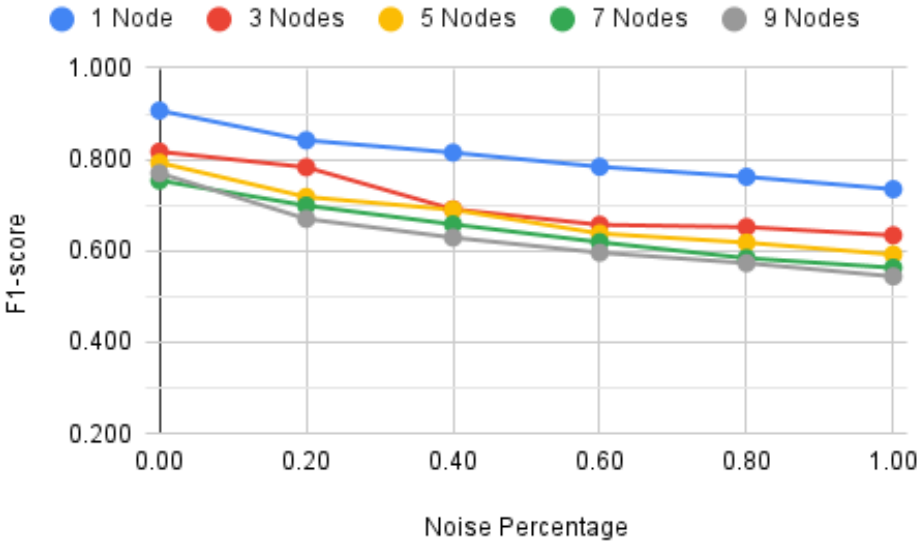


Figure A.6: Pruning - Weighted average precision considering noise in the data.

Table A.7: Pruning - Weighted average recall considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.906	0.806	0.782	0.739	0.765
0.20	0.832	0.772	0.677	0.649	0.601
0.40	0.793	0.633	0.633	0.571	0.525
0.60	0.753	0.562	0.536	0.519	0.474
0.80	0.713	0.493	0.486	0.443	0.406
1.00	0.683	0.496	0.445	0.394	0.366

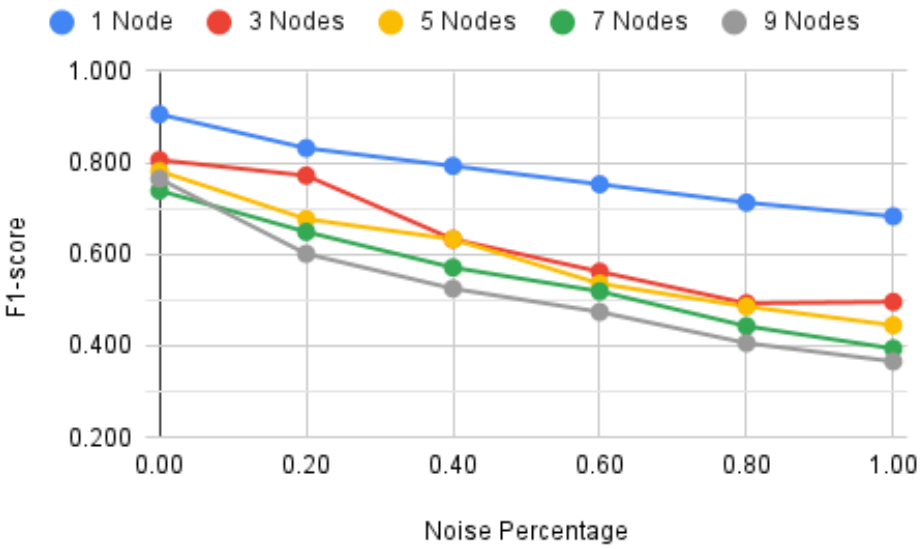


Figure A.7: Pruning - Weighted average recall considering noise in the data.

Table A.8: Pruning - Accuracy considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.906	0.806	0.782	0.739	0.765
0.20	0.832	0.772	0.677	0.649	0.601
0.40	0.793	0.633	0.633	0.571	0.525
0.60	0.753	0.562	0.536	0.519	0.474
0.80	0.713	0.493	0.486	0.443	0.406
1.00	0.683	0.496	0.445	0.394	0.366

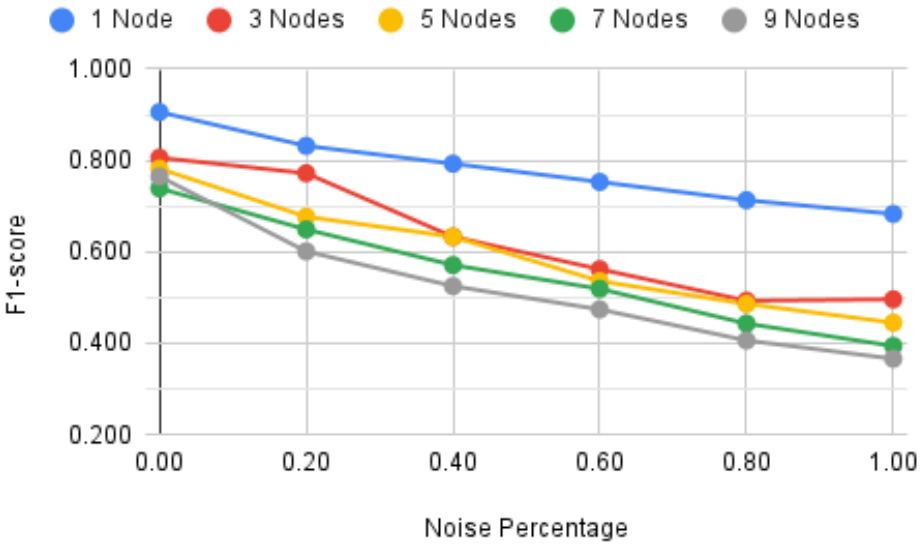


Figure A.8: Pruning - Accuracy considering noise in the data.

A.3 Complements to Table 6.3 and Figure 6.3

Table A.9: LiteRT - Cross-entropy loss considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.308	0.807	0.834	1.042	0.878
0.20	0.573	0.972	1.321	1.416	1.506
0.40	0.728	1.465	1.475	1.608	1.703
0.60	0.857	1.651	1.710	1.747	1.834
0.80	0.982	1.742	1.852	1.931	2.018
1.00	1.088	1.692	1.953	2.053	2.136

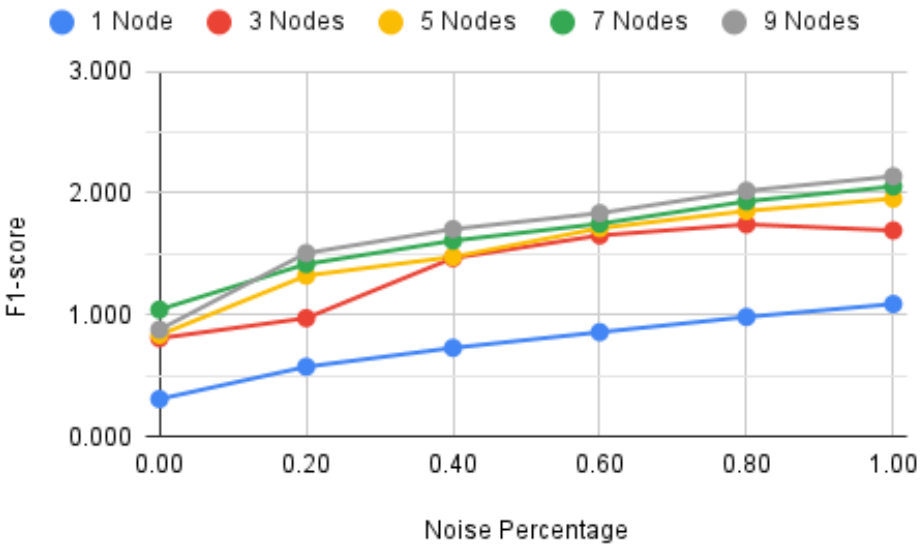


Figure A.9: LiteRT - Cross-entropy loss considering noise in the data.

Table A.10: LiteRT - Weighted average precision considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.907	0.817	0.793	0.754	0.770
0.20	0.842	0.783	0.718	0.699	0.670
0.40	0.815	0.691	0.690	0.658	0.629
0.60	0.784	0.657	0.638	0.619	0.596
0.80	0.762	0.652	0.618	0.584	0.573
1.00	0.735	0.634	0.592	0.563	0.544

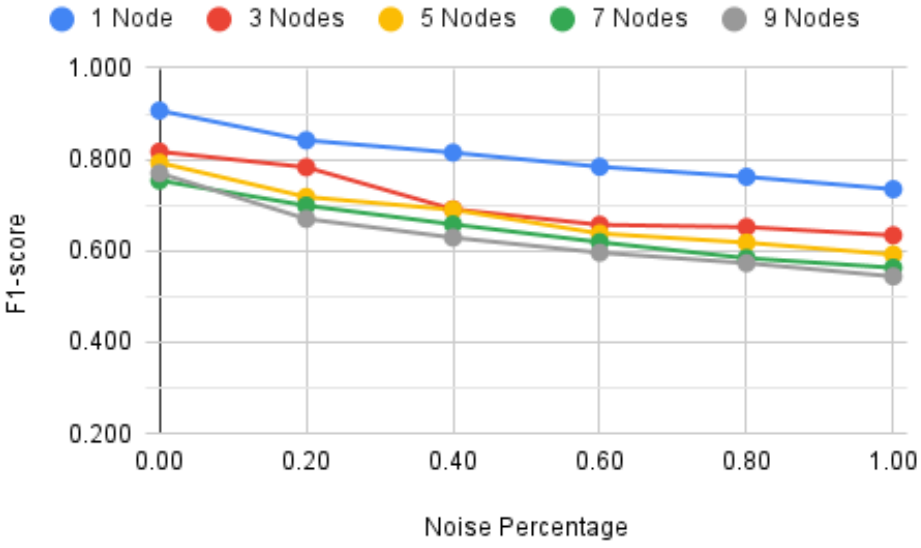


Figure A.10: LiteRT - Weighted average precision considering noise in the data.

Table A.11: LiteRT - Weighted average recall considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.906	0.806	0.782	0.739	0.765
0.20	0.832	0.772	0.677	0.649	0.601
0.40	0.793	0.633	0.633	0.571	0.525
0.60	0.753	0.562	0.536	0.519	0.474
0.80	0.713	0.493	0.486	0.443	0.406
1.00	0.683	0.496	0.445	0.394	0.366

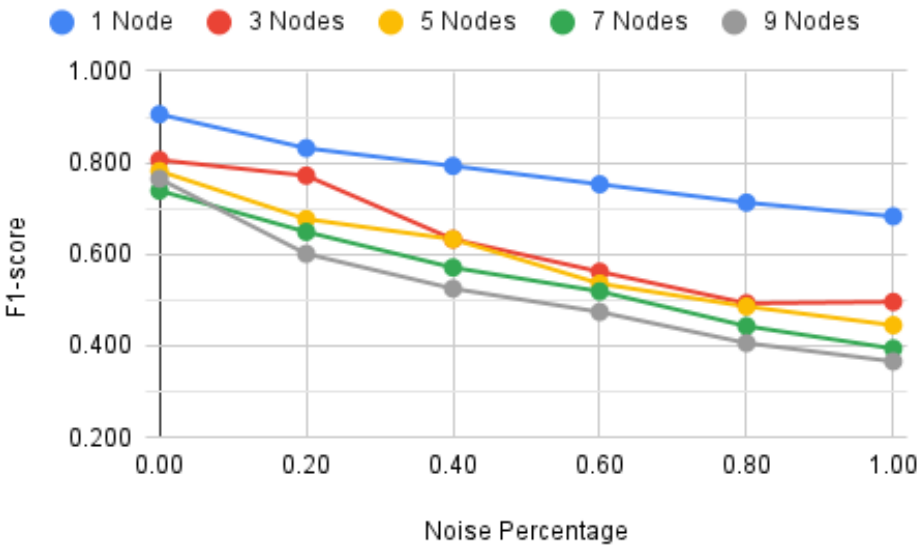


Figure A.11: LiteRT - Weighted average recall considering noise in the data.

Table A.12: LiteRT - Accuracy considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.906	0.806	0.782	0.739	0.765
0.20	0.832	0.772	0.677	0.649	0.601
0.40	0.793	0.633	0.633	0.571	0.525
0.60	0.753	0.562	0.536	0.519	0.474
0.80	0.713	0.493	0.486	0.443	0.406
1.00	0.683	0.496	0.445	0.394	0.366

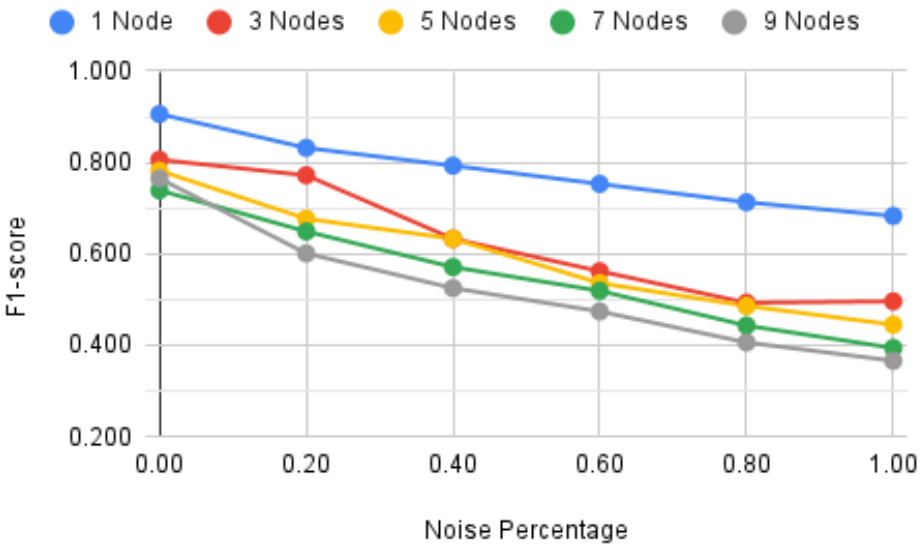


Figure A.12: LiteRT - Accuracy considering noise in the data.

A.4 Complements to Table 6.4 and Figure 6.4

Table A.13: Quantization - Cross-entropy loss considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.309	0.843	0.863	1.187	0.902
0.20	0.573	1.152	1.372	1.382	1.788
0.40	0.729	1.516	1.529	1.774	1.723
0.60	0.856	1.605	1.763	1.948	1.889
0.80	0.984	1.779	1.797	2.041	2.090
1.00	1.085	1.631	1.985	2.151	2.254

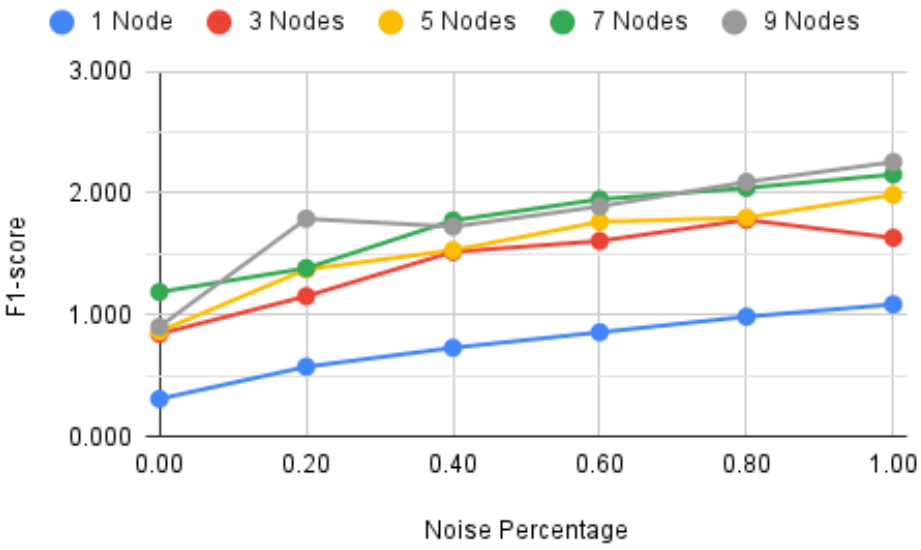


Figure A.13: Quantization - Cross-entropy loss considering noise in the data.

Table A.14: Quantization - Weighted average precision considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.906	0.804	0.781	0.721	0.758
0.20	0.842	0.768	0.714	0.688	0.640
0.40	0.815	0.669	0.683	0.643	0.625
0.60	0.784	0.648	0.613	0.613	0.591
0.80	0.761	0.634	0.611	0.583	0.559
1.00	0.735	0.627	0.573	0.552	0.528

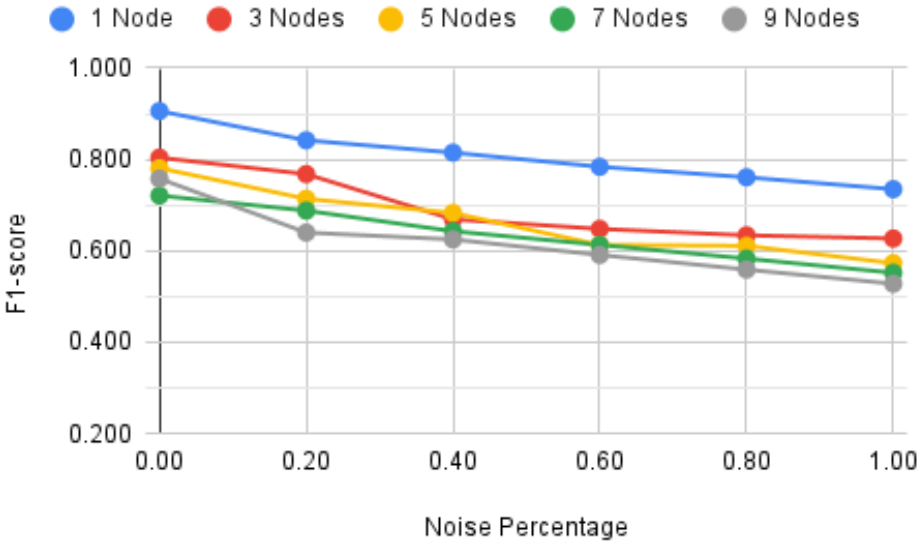


Figure A.14: Quantization - Weighted average precision considering noise in the data.

Table A.15: Quantization - Weighted average recall considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.906	0.786	0.767	0.691	0.752
0.20	0.833	0.755	0.659	0.627	0.486
0.40	0.792	0.595	0.620	0.528	0.513
0.60	0.754	0.535	0.497	0.484	0.460
0.80	0.712	0.482	0.487	0.409	0.381
1.00	0.685	0.522	0.451	0.366	0.336

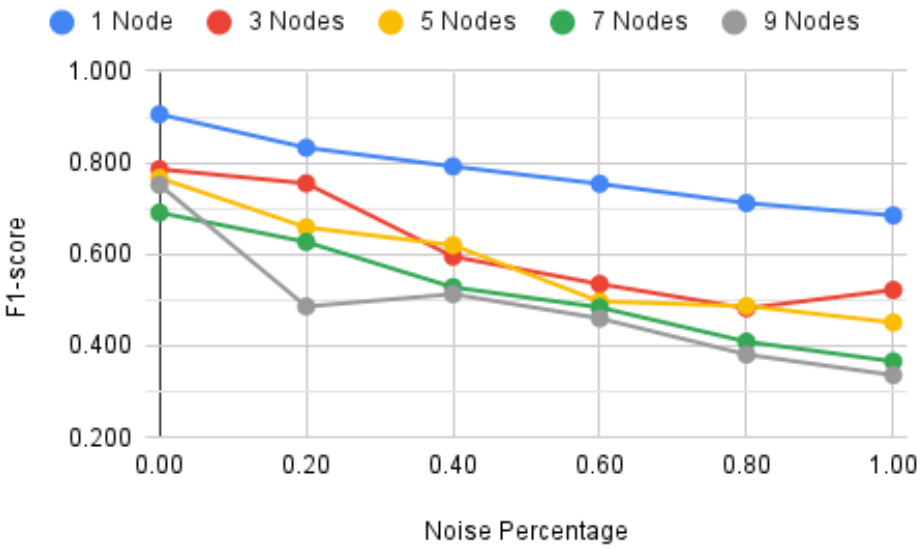


Figure A.15: Quantization - Weighted average recall considering noise in the data.

Table A.16: Quantization - Accuracy considering noise in the data.

Noise Percentage Over Data	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
0.00	0.906	0.786	0.767	0.691	0.752
0.20	0.833	0.755	0.659	0.627	0.486
0.40	0.792	0.595	0.620	0.528	0.513
0.60	0.754	0.535	0.497	0.484	0.460
0.80	0.712	0.482	0.487	0.409	0.381
1.00	0.685	0.522	0.451	0.366	0.336

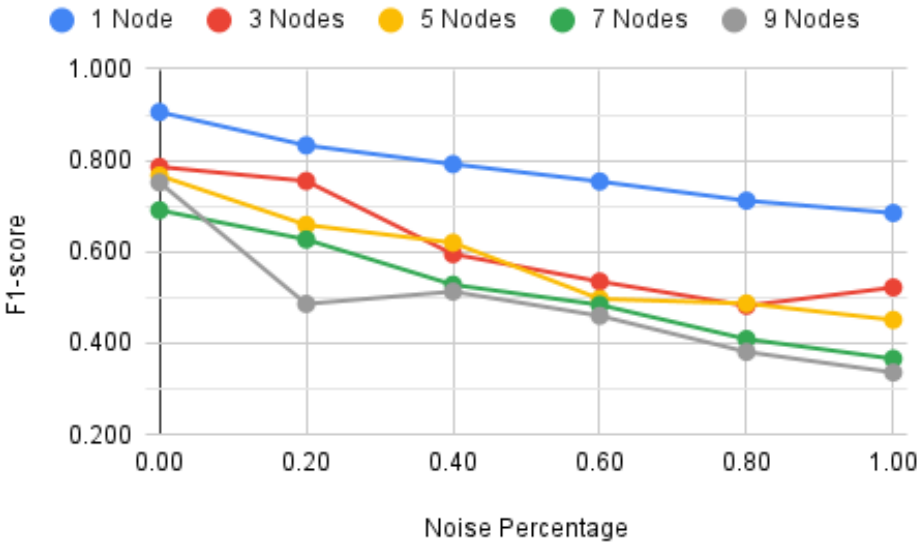


Figure A.16: Quantization - Accuracy considering noise in the data.

A.5 Complements to Table 6.5 and Figure 6.5

Table A.17: Standard Model - Cross-entropy loss considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.336	0.864	0.876	1.090	0.912
2	-	0.647	0.939	0.836	1.139
3	-	0.836	1.037	1.032	1.164
4	-	0.986	0.988	1.068	0.977
5	-	0.941	1.109	1.085	1.020
6	-	1.170	1.164	1.142	1.147

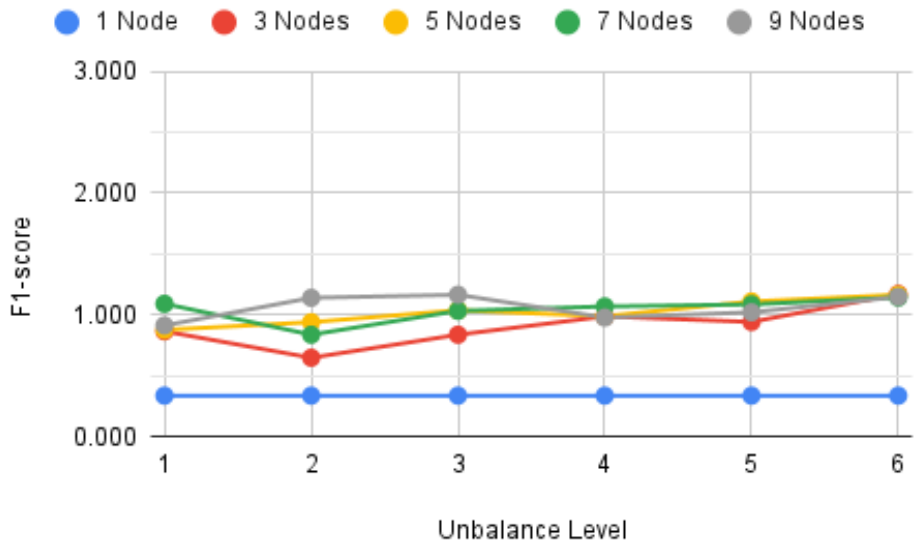


Figure A.17: Standard Model - Cross-entropy loss considering unbalanced data.

Table A.18: Standard Model - Weighted average precision considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.897	0.806	0.786	0.754	0.766
2	-	0.825	0.780	0.785	0.731
3	-	0.799	0.775	0.748	0.733
4	-	0.779	0.772	0.749	0.755
5	-	0.782	0.769	0.744	0.744
6	-	0.777	0.760	0.744	0.718

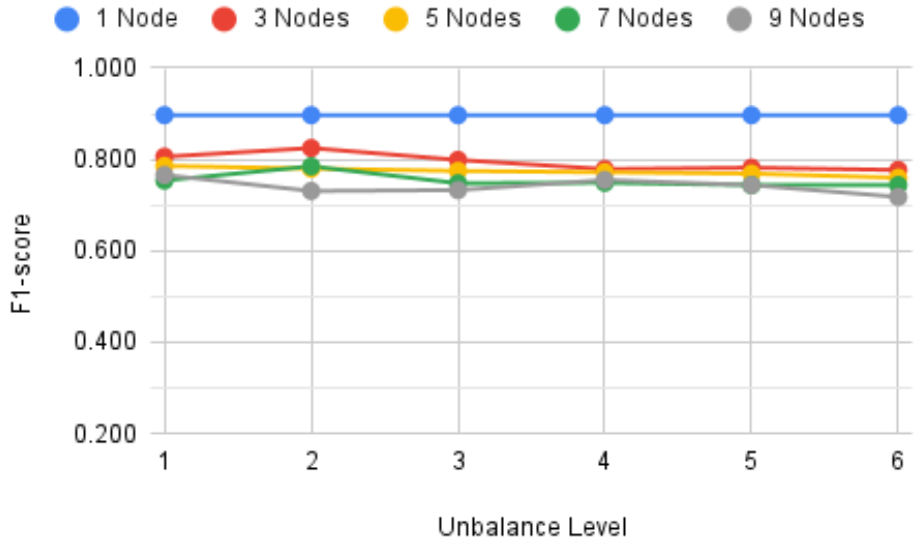


Figure A.18: Standard Model - Weighted average precision considering unbalanced data.

Table A.19: Standard Model - Weighted average recall considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.896	0.793	0.772	0.738	0.762
2	-	0.819	0.771	0.777	0.719
3	-	0.779	0.771	0.734	0.726
4	-	0.760	0.764	0.739	0.727
5	-	0.772	0.760	0.727	0.702
6	-	0.751	0.751	0.706	0.670

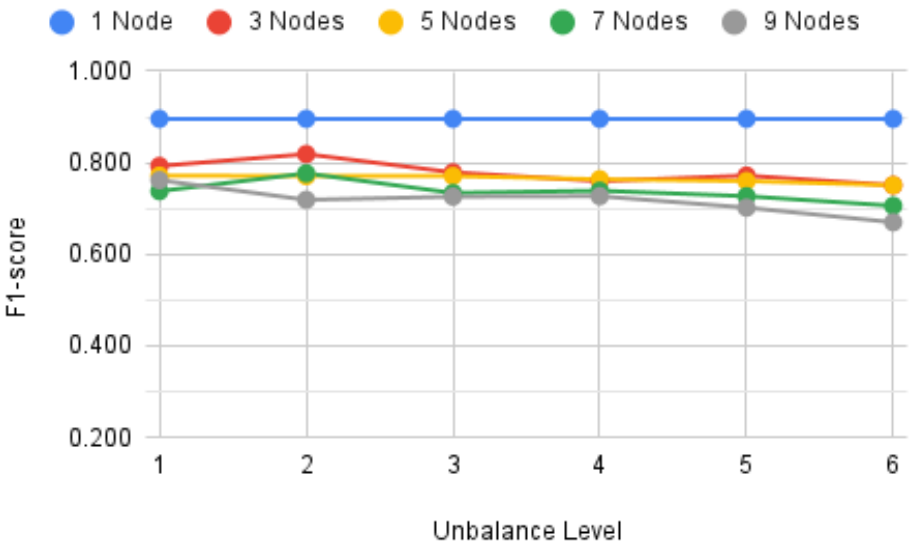


Figure A.19: Standard Model - Weighted average recall considering unbalanced data.

Table A.20: Standard Model - Accuracy considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.896	0.793	0.772	0.738	0.762
2	-	0.819	0.771	0.777	0.719
3	-	0.779	0.771	0.734	0.726
4	-	0.760	0.764	0.739	0.727
5	-	0.772	0.760	0.727	0.702
6	-	0.751	0.751	0.706	0.670

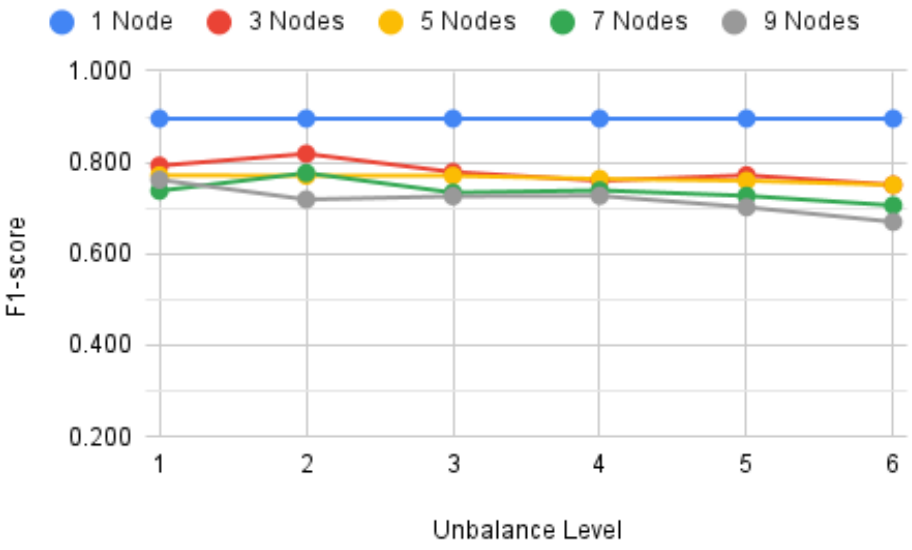


Figure A.20: Standard Model - Accuracy considering unbalanced data.

A.6 Complements to Table 6.6 and Figure 6.6

Table A.21: Pruning - Cross-entropy loss considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.308	0.807	0.834	1.042	0.878
2	-	0.641	0.900	0.813	1.087
3	-	0.762	1.003	1.014	1.156
4	-	0.929	0.961	1.032	1.004
5	-	0.923	1.065	1.147	0.998
6	-	1.106	1.133	1.306	1.131

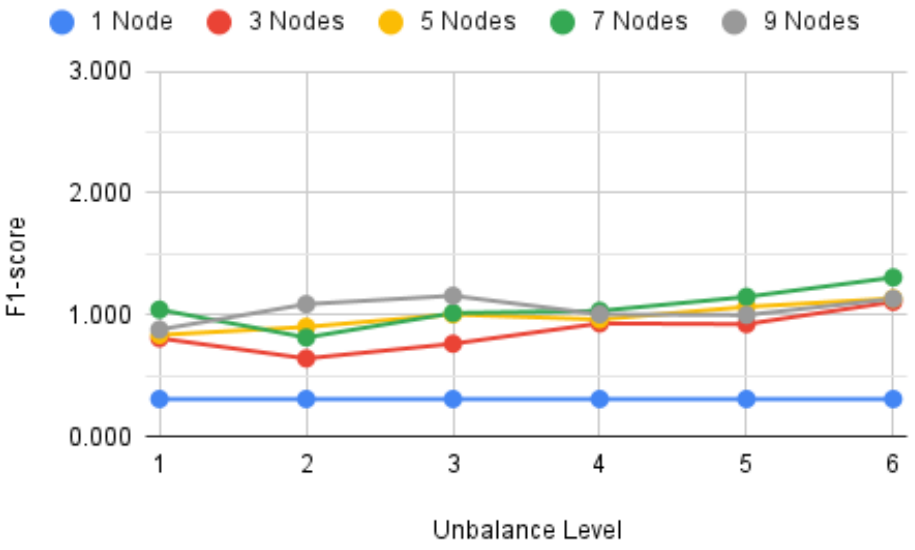


Figure A.21: Pruning - Cross-entropy loss considering unbalanced data.

Table A.22: Pruning - Weighted average precision considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.907	0.817	0.793	0.754	0.770
2	-	0.823	0.784	0.786	0.734
3	-	0.818	0.779	0.748	0.732
4	-	0.788	0.773	0.751	0.746
5	-	0.787	0.770	0.744	0.746
6	-	0.783	0.762	0.724	0.721

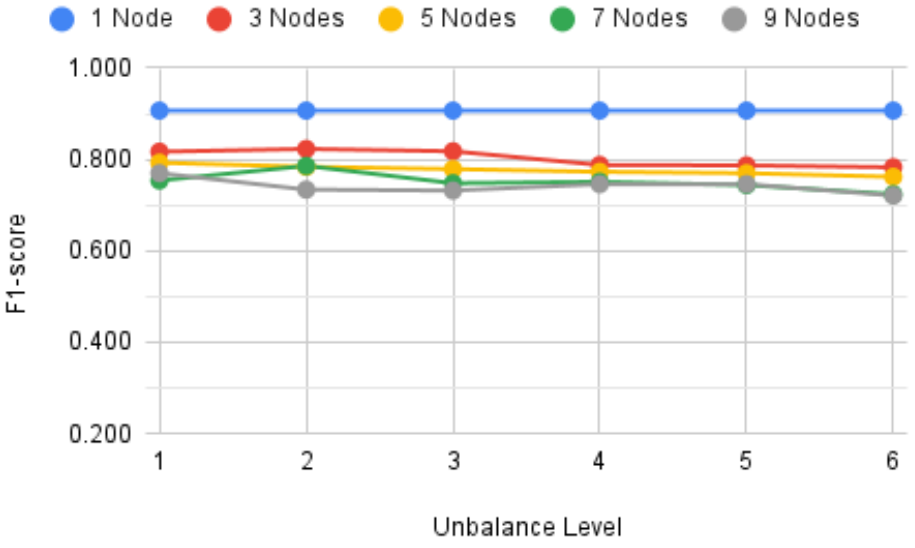


Figure A.22: Pruning - Weighted average precision considering unbalanced data.

Table A.23: Pruning - Weighted average recall considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.906	0.806	0.782	0.739	0.765
2	-	0.813	0.776	0.779	0.723
3	-	0.811	0.773	0.732	0.723
4	-	0.772	0.764	0.740	0.731
5	-	0.777	0.762	0.728	0.706
6	-	0.760	0.754	0.693	0.672

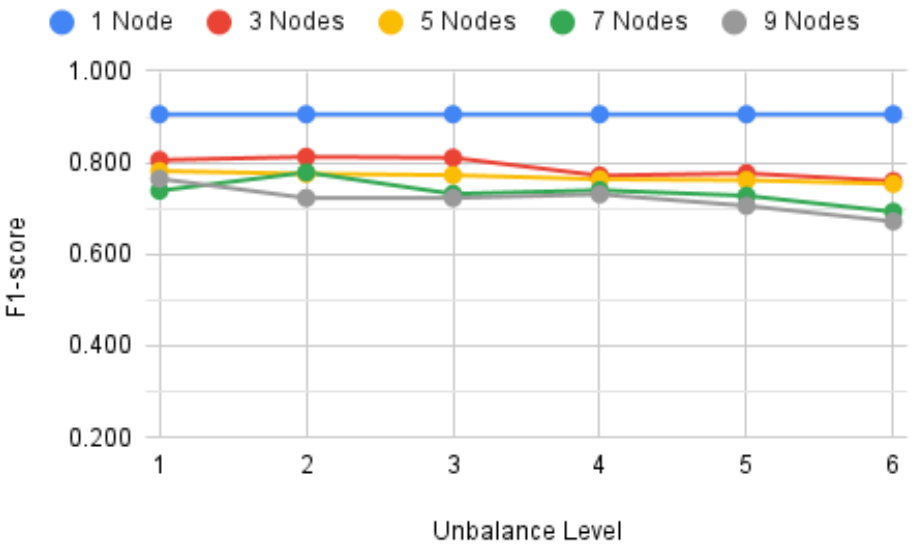


Figure A.23: Pruning - Weighted average recall considering unbalanced data.

Table A.24: Pruning - Accuracy considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.906	0.806	0.782	0.739	0.765
2	-	0.813	0.776	0.779	0.723
3	-	0.811	0.773	0.732	0.723
4	-	0.772	0.764	0.740	0.731
5	-	0.777	0.762	0.728	0.706
6	-	0.760	0.754	0.693	0.672

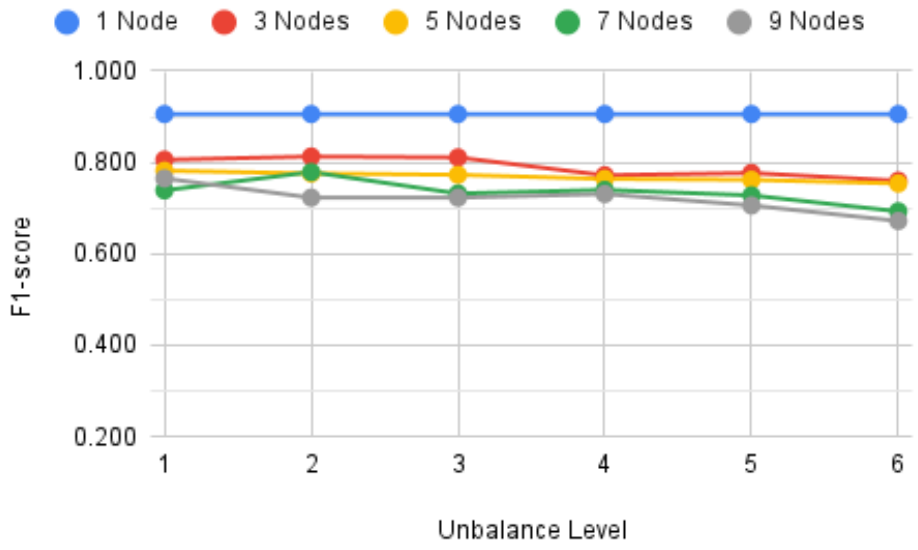


Figure A.24: Pruning - Accuracy considering unbalanced data.

A.7 Complements to Table 6.7 and Figure 6.7

Table A.25: LiteRT - Cross-entropy loss considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.308	0.807	0.834	1.042	0.878
2	-	0.641	0.900	0.813	1.087
3	-	0.762	1.003	1.014	1.156
4	-	0.929	0.961	1.032	1.004
5	-	0.923	1.065	1.147	0.998
6	-	1.106	1.133	1.306	1.131

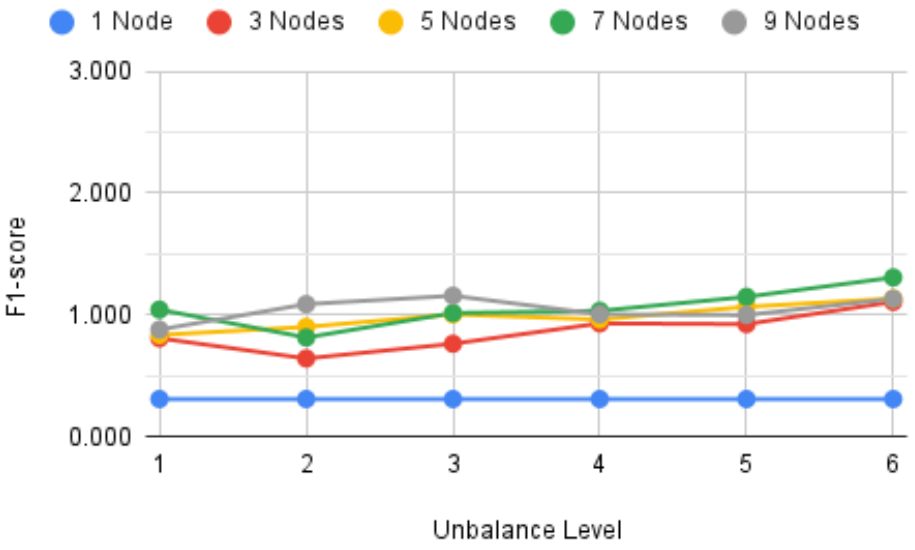


Figure A.25: LiteRT - Cross-entropy loss considering unbalanced data.

Table A.26: LiteRT - Weighted average precision considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.907	0.817	0.793	0.754	0.770
2	-	0.823	0.784	0.786	0.734
3	-	0.818	0.779	0.748	0.732
4	-	0.788	0.773	0.751	0.746
5	-	0.787	0.770	0.744	0.746
6	-	0.783	0.762	0.724	0.721

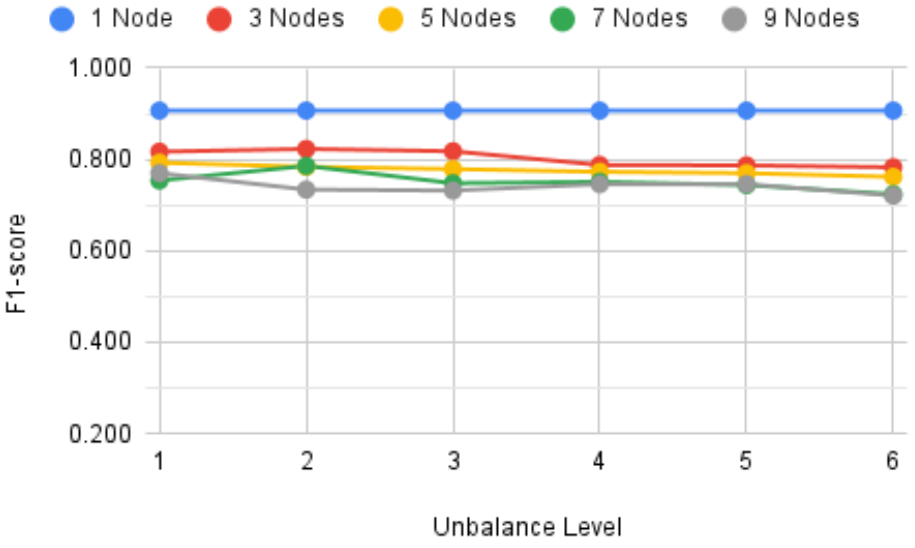


Figure A.26: LiteRT - Weighted average precision considering unbalanced data.

Table A.27: LiteRT - Weighted average recall considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.906	0.806	0.782	0.739	0.765
2	-	0.813	0.776	0.779	0.723
3	-	0.811	0.773	0.732	0.723
4	-	0.772	0.764	0.740	0.731
5	-	0.777	0.762	0.728	0.706
6	-	0.760	0.754	0.693	0.672

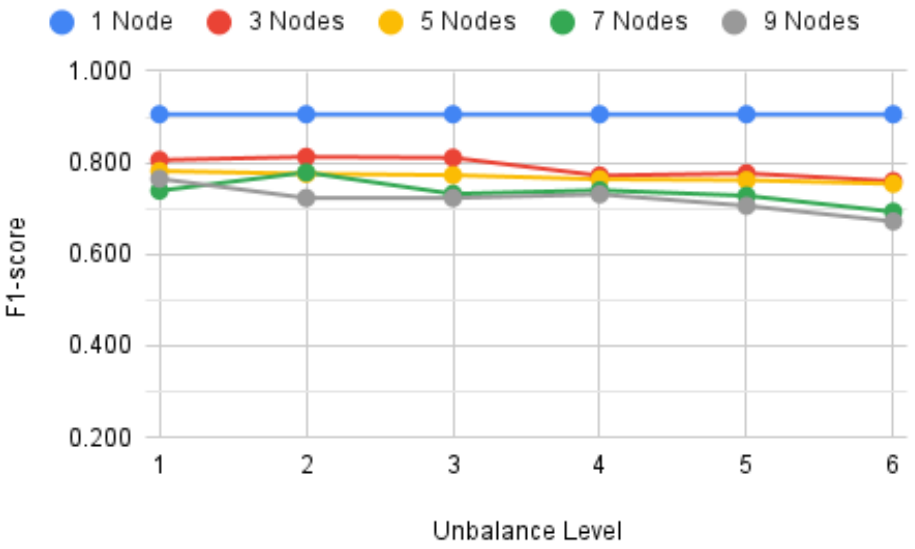


Figure A.27: LiteRT - Weighted average recall considering unbalanced data.

Table A.28: LiteRT - Accuracy considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.906	0.806	0.782	0.739	0.765
2	-	0.813	0.776	0.779	0.723
3	-	0.811	0.773	0.732	0.723
4	-	0.772	0.764	0.740	0.731
5	-	0.777	0.762	0.728	0.706
6	-	0.760	0.754	0.693	0.672

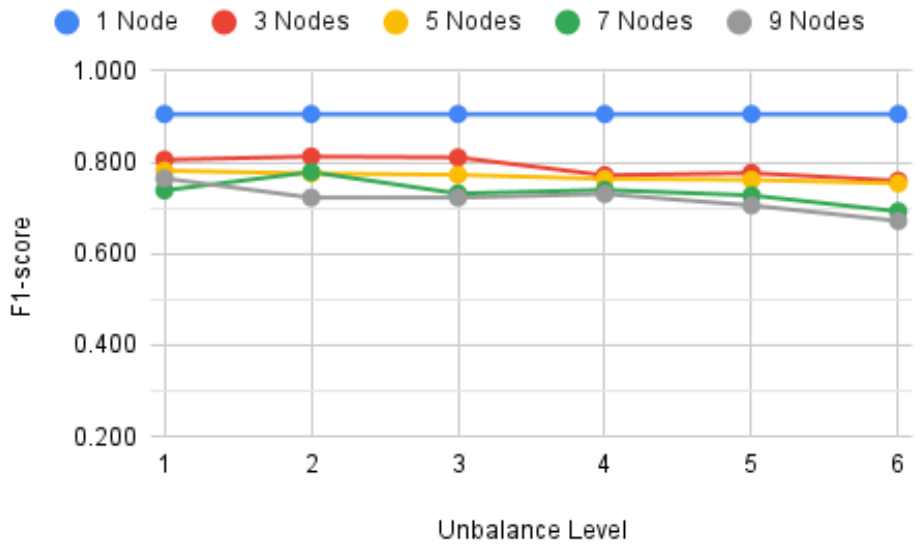


Figure A.28: LiteRT - Accuracy considering unbalanced data.

A.8 Complements to Table 6.8 and Figure 6.8

Table A.29: Quantization - Cross-entropy loss considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.309	0.843	0.863	1.187	0.902
2	-	0.699	0.970	0.876	1.248
3	-	0.873	1.037	1.060	1.235
4	-	0.971	1.065	1.175	1.025
5	-	1.034	1.229	1.310	1.029
6	-	1.322	1.072	1.358	1.194

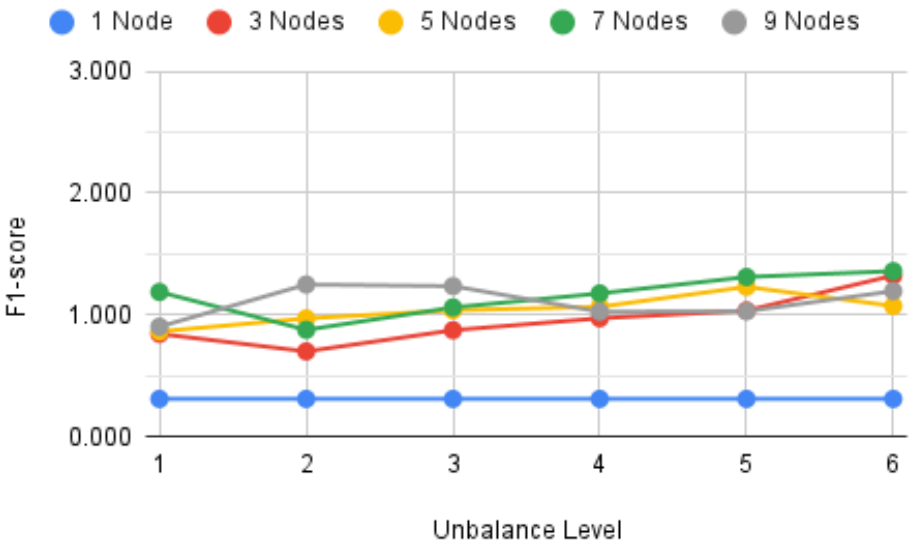


Figure A.29: Quantization - Cross-entropy loss considering unbalanced data.

Table A.30: Quantization - Weighted average precision considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.906	0.804	0.781	0.721	0.758
2	-	0.814	0.773	0.782	0.711
3	-	0.803	0.758	0.732	0.715
4	-	0.758	0.765	0.735	0.735
5	-	0.765	0.749	0.702	0.741
6	-	0.764	0.752	0.716	0.716

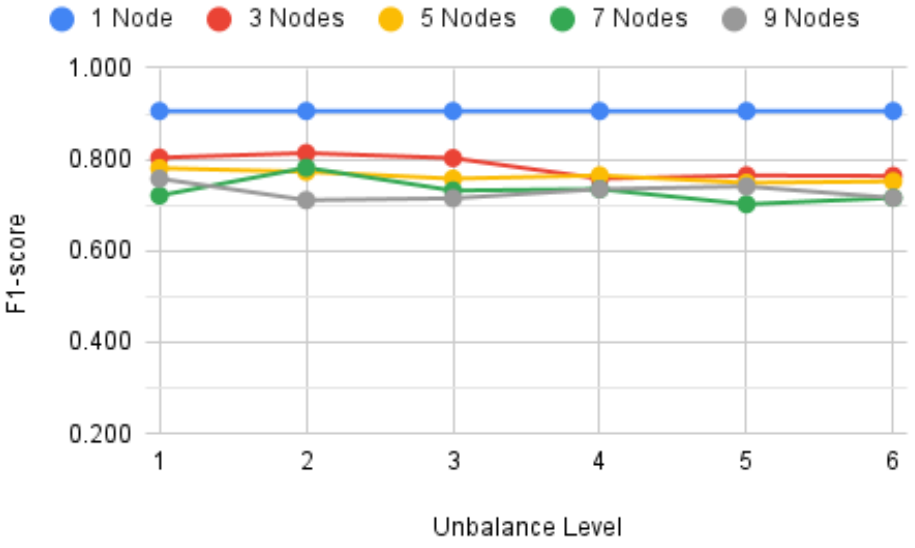


Figure A.30: Quantization - Weighted average precision considering unbalanced data.

Table A.31: Quantization - Weighted average recall considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.906	0.786	0.767	0.691	0.752
2	-	0.799	0.768	0.771	0.691
3	-	0.791	0.742	0.700	0.701
4	-	0.730	0.750	0.721	0.708
5	-	0.738	0.729	0.653	0.695
6	-	0.725	0.737	0.688	0.666

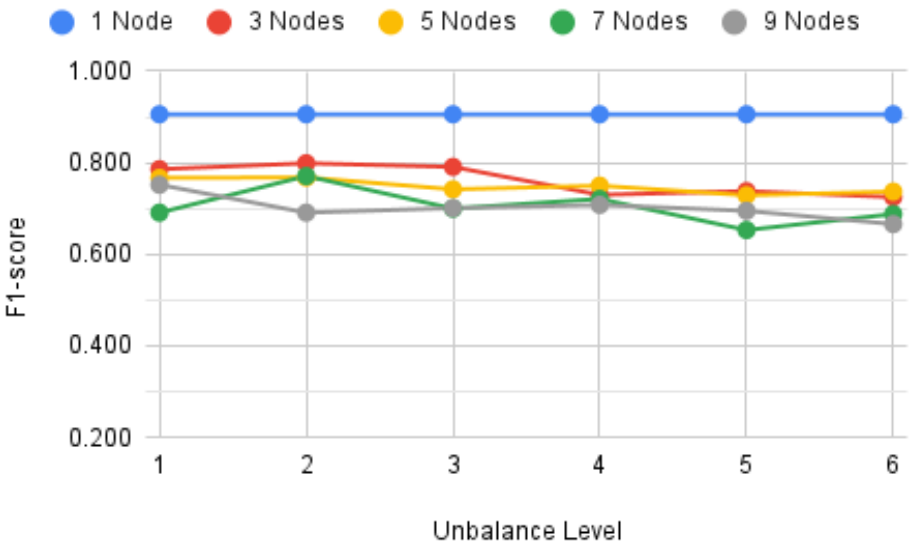


Figure A.31: Quantization - Weighted average recall considering unbalanced data.

Table A.32: Quantization - Accuracy considering unbalanced data.

Unbalance Level	1 Node	3 Nodes	5 Nodes	7 Nodes	9 Nodes
1	0.906	0.786	0.767	0.691	0.752
2	-	0.799	0.768	0.771	0.691
3	-	0.791	0.742	0.700	0.701
4	-	0.730	0.750	0.721	0.708
5	-	0.738	0.729	0.653	0.695
6	-	0.725	0.737	0.688	0.666

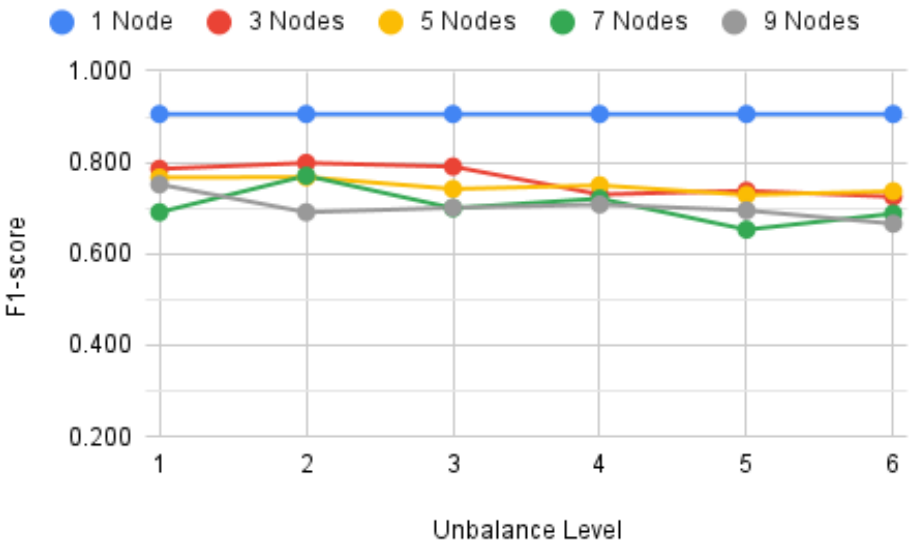


Figure A.32: Quantization - Accuracy considering unbalanced data.

A.9 Complements to Table 6.9 and Figure 6.9

Table A.33: Standard Model - Cross-entropy loss considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.912			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.991	1.153	1.194	1.106
0.40	1.068	1.162	1.257	1.207
0.60	1.001	1.219	1.216	1.254
0.80	1.074	1.186	1.162	1.436
1.00	1.121	1.168	1.213	1.434

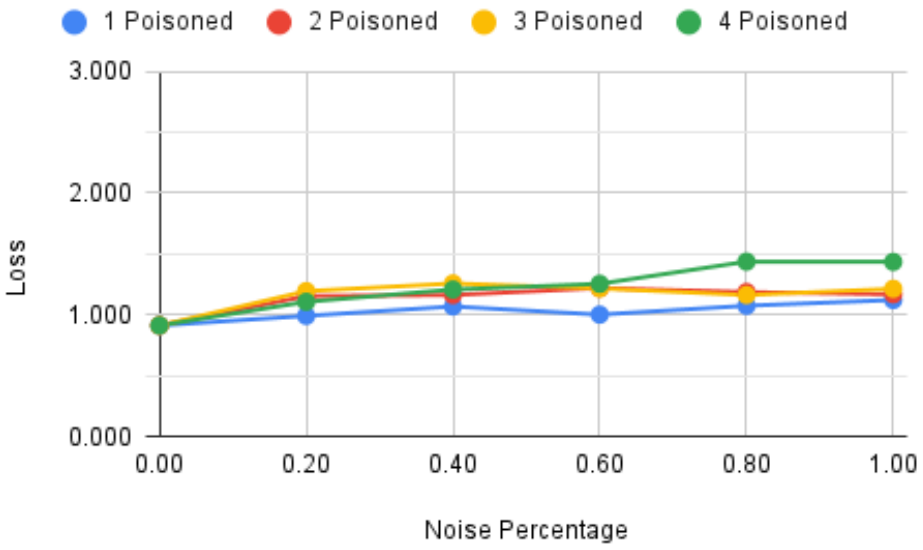


Figure A.33: Standard Model - Cross-entropy loss considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table A.34: Standard Model - Weighted average precision considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.766			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.749	0.733	0.731	0.734
0.40	0.743	0.729	0.717	0.713
0.60	0.747	0.721	0.714	0.702
0.80	0.739	0.723	0.719	0.668
1.00	0.735	0.720	0.711	0.672

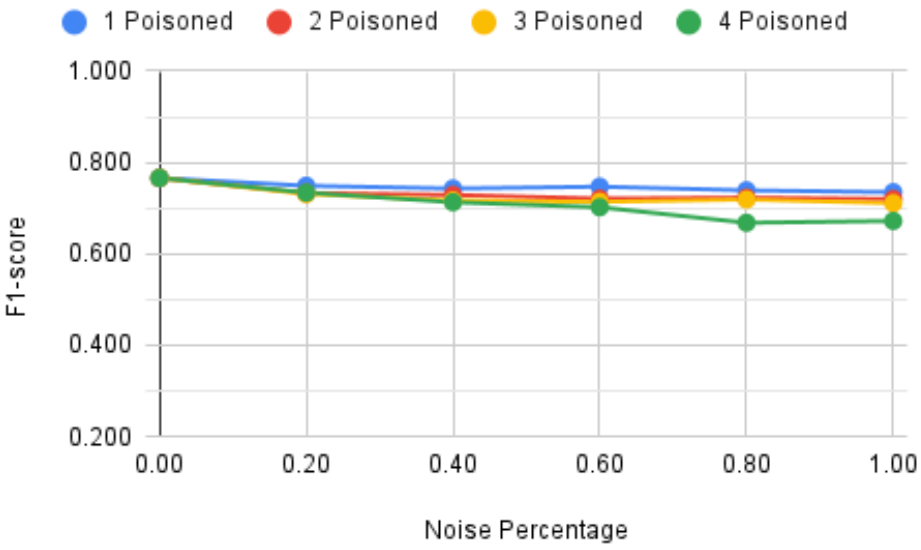


Figure A.34: Standard Model - Weighted average precision considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table A.35: Standard Model - Weighted average recall considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.762			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.743	0.724	0.714	0.719
0.40	0.724	0.717	0.689	0.691
0.60	0.733	0.706	0.694	0.672
0.80	0.726	0.700	0.697	0.613
1.00	0.725	0.698	0.677	0.619

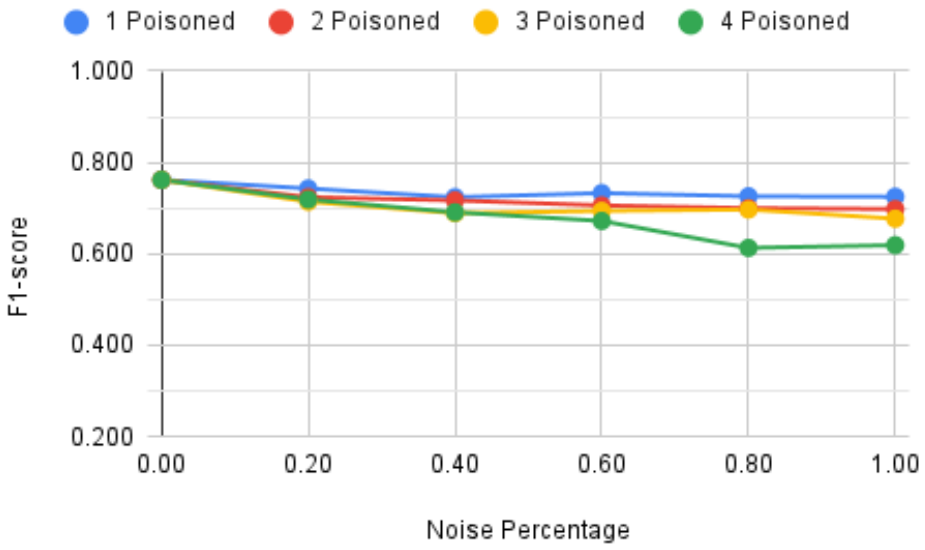


Figure A.35: Standard Model - Weighted average recall considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table A.36: Standard Model - Accuracy considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.762			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.743	0.724	0.714	0.719
0.40	0.724	0.717	0.689	0.691
0.60	0.733	0.706	0.694	0.672
0.80	0.726	0.700	0.697	0.613
1.00	0.725	0.698	0.677	0.619

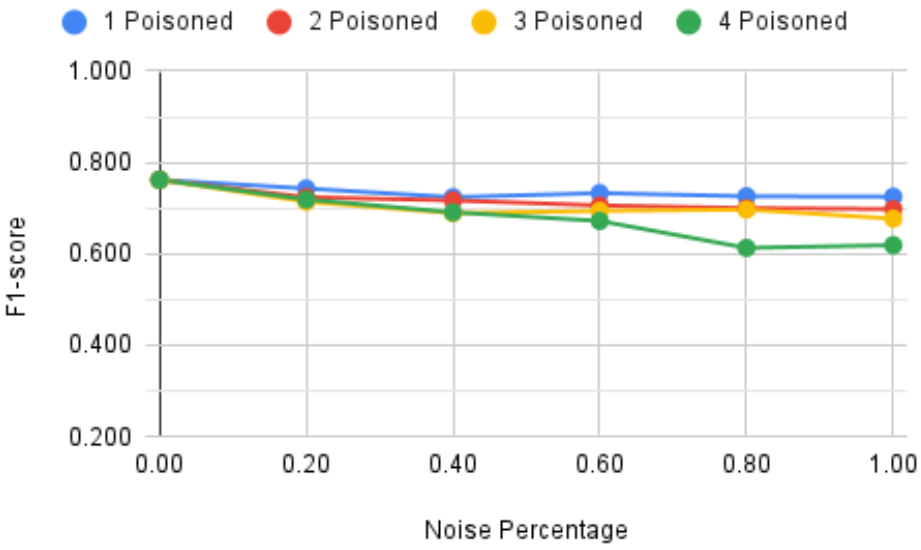


Figure A.36: Standard Model - Accuracy considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

A.10 Complements to Table 6.10 and Figure 6.10

Table A.37: Pruning - Cross-entropy loss considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.878			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.967	1.098	1.139	1.069
0.40	1.035	1.106	1.200	1.175
0.60	0.957	1.162	1.187	1.214
0.80	1.009	1.125	1.134	1.414
1.00	1.063	1.109	1.176	1.387

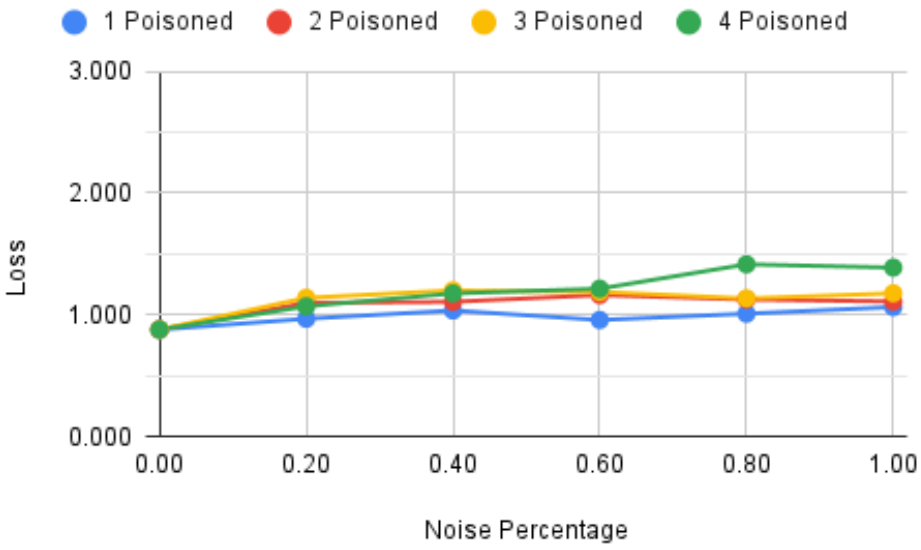


Figure A.37: Pruning - Cross-entropy loss considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table A.38: Pruning - Weighted average precision considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.770			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.750	0.736	0.735	0.735
0.40	0.747	0.733	0.720	0.713
0.60	0.752	0.723	0.715	0.703
0.80	0.746	0.728	0.719	0.667
1.00	0.736	0.726	0.710	0.673

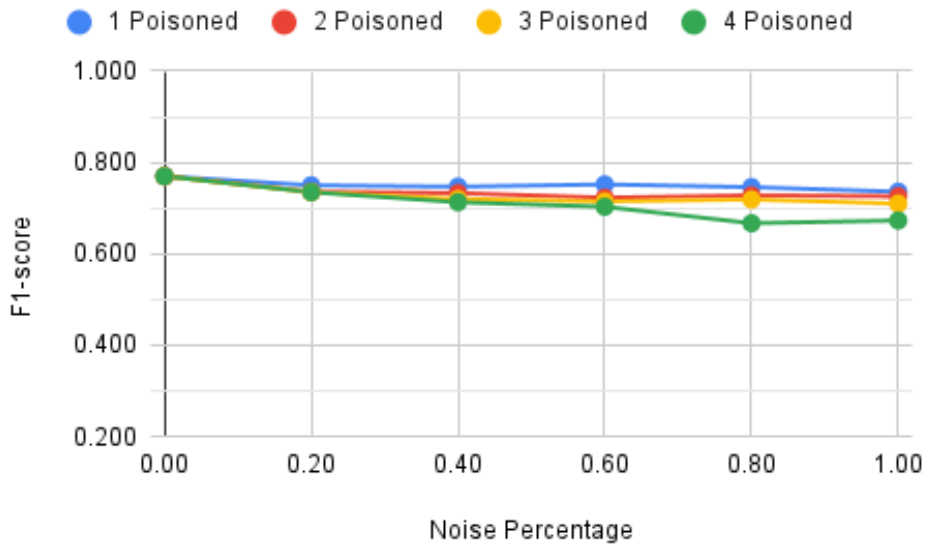


Figure A.38: Pruning - Weighted average precision considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table A.39: Pruning - Weighted average recall considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.743	0.728	0.721	0.722
0.40	0.727	0.724	0.695	0.688
0.60	0.739	0.711	0.695	0.675
0.80	0.736	0.709	0.696	0.611
1.00	0.728	0.709	0.679	0.630

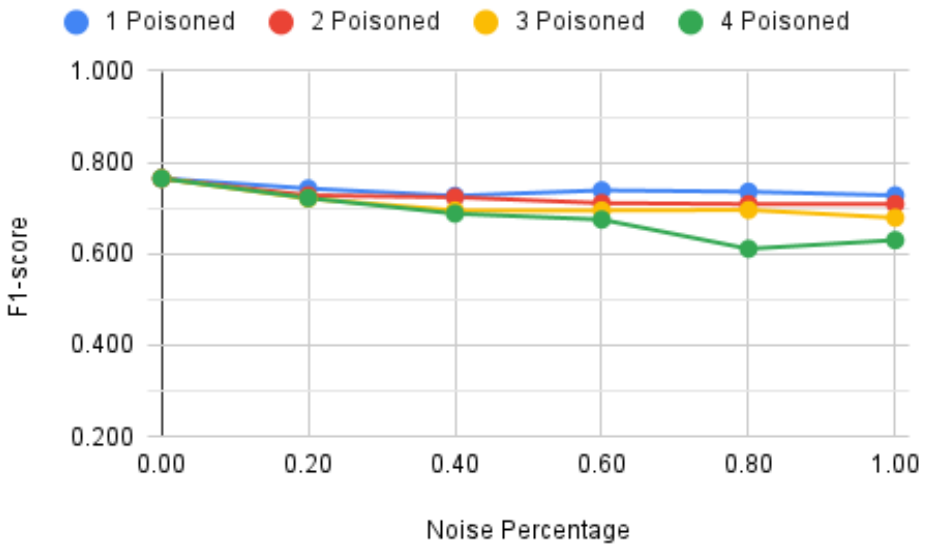


Figure A.39: Pruning - Weighted average recall considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table A.40: Pruning - Accuracy considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.743	0.728	0.721	0.722
0.40	0.727	0.724	0.695	0.688
0.60	0.739	0.711	0.695	0.675
0.80	0.736	0.709	0.696	0.611
1.00	0.728	0.709	0.679	0.630

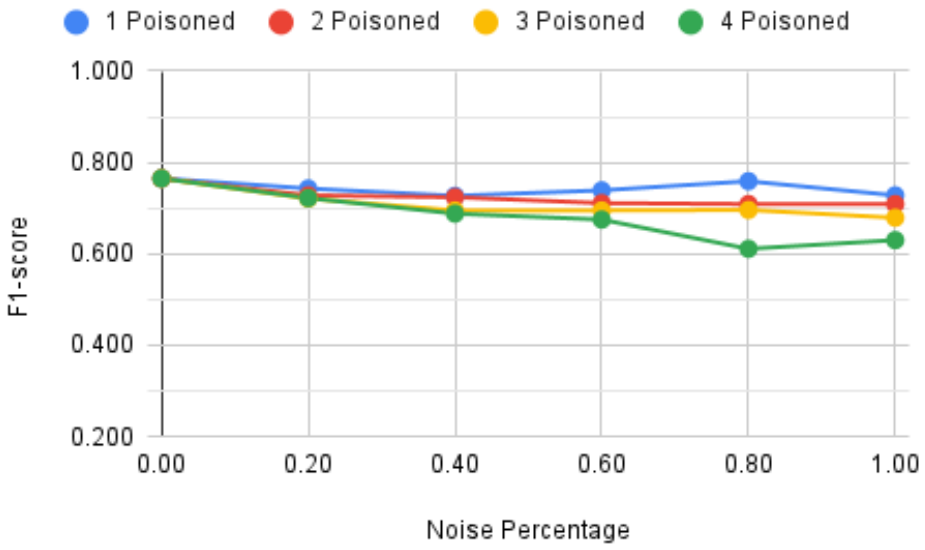


Figure A.40: Pruning - Accuracy considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

A.11 Complements to Table 6.11 and Figure 6.11

Table A.41: LiteRT - Cross-entropy loss considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.878			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.967	1.098	0.967	1.069
0.40	0.949	1.437	2.879	4.268
0.60	0.922	1.469	2.904	4.487
0.80	0.912	1.512	2.987	5.379
1.00	0.938	1.556	3.162	5.097

Please note: The figure below employs a different scale compared to most of the graphs in this report. While the Y-axis of most graphs ranges from 0 to 3, this specific graph has been adjusted to a scale of 0 to 6 to account for the presence of outlier results.

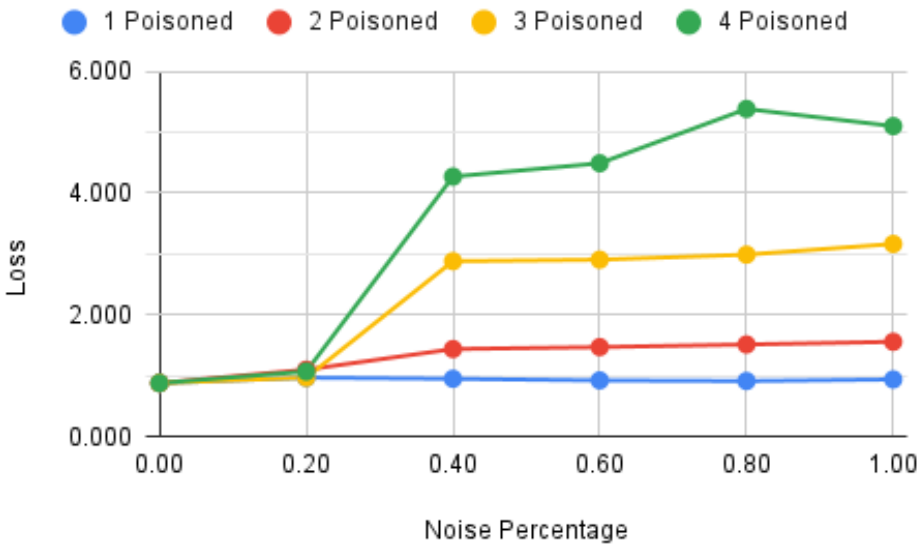


Figure A.41: LiteRT - Cross-entropy loss considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table A.42: LiteRT - Weighted average precision considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.770			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.750	0.736	0.734	0.735
0.40	0.747	0.731	0.719	0.710
0.60	0.754	0.723	0.715	0.703
0.80	0.747	0.729	0.717	0.669
1.00	0.736	0.726	0.710	0.669

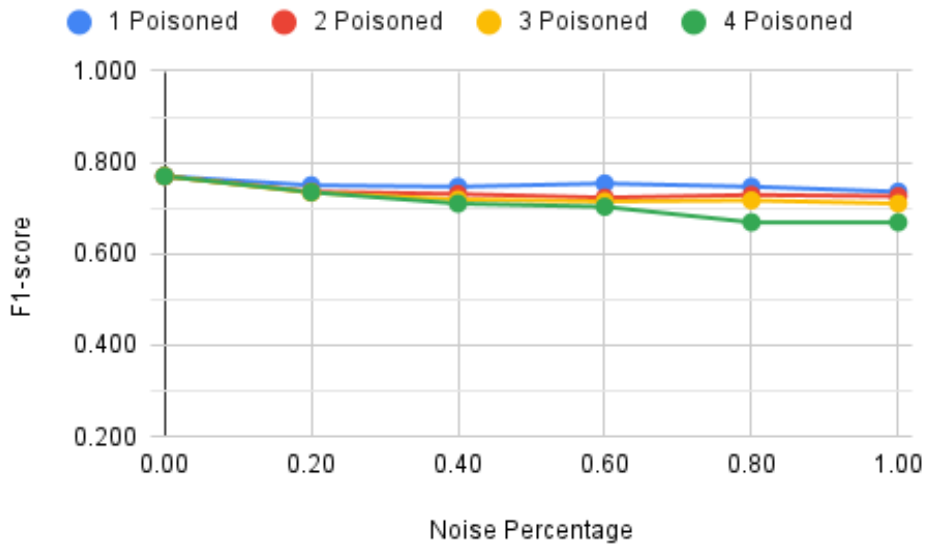


Figure A.42: LiteRT - Weighted average precision considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table A.43: LiteRT - Weighted average recall considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.743	0.728	0.721	0.722
0.40	0.729	0.722	0.695	0.686
0.60	0.742	0.711	0.695	0.675
0.80	0.736	0.709	0.695	0.610
1.00	0.729	0.709	0.678	0.625

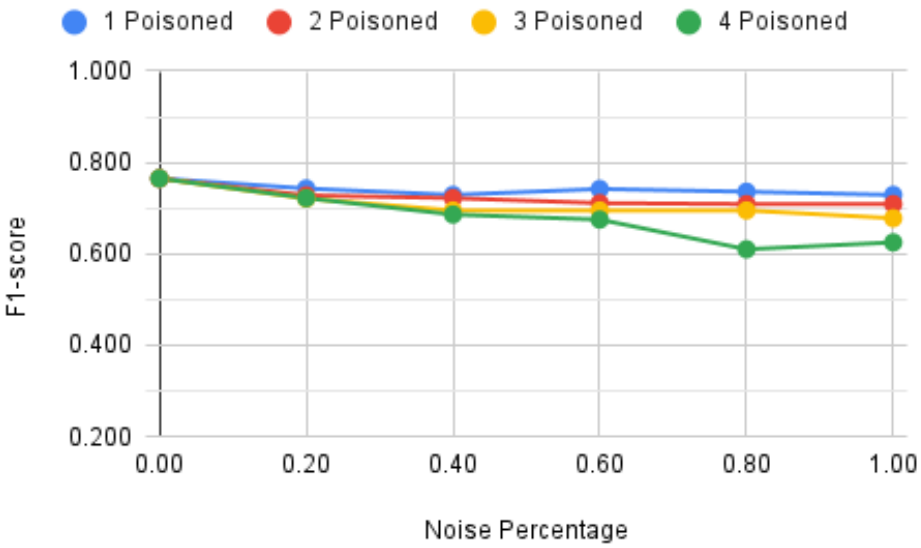


Figure A.43: LiteRT - Weighted average recall considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table A.44: LiteRT - Accuracy considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.743	0.728	0.721	0.722
0.40	0.729	0.722	0.695	0.686
0.60	0.742	0.711	0.695	0.675
0.80	0.736	0.709	0.695	0.610
1.00	0.729	0.709	0.678	0.625

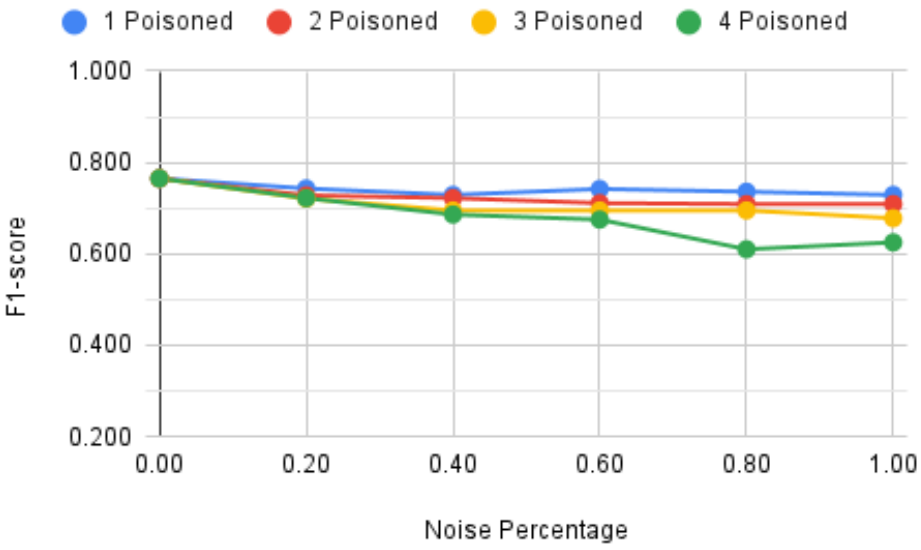


Figure A.44: LiteRT - Accuracy considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

A.12 Complements to Table 6.12 and Figure 6.12

Table A.45: Quantization - Cross-entropy loss considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.902			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.958	1.124	1.131	1.170
0.40	1.124	1.150	1.202	1.328
0.60	1.068	1.210	1.229	1.327
0.80	1.006	1.252	1.326	1.484
1.00	1.130	1.215	1.349	1.534

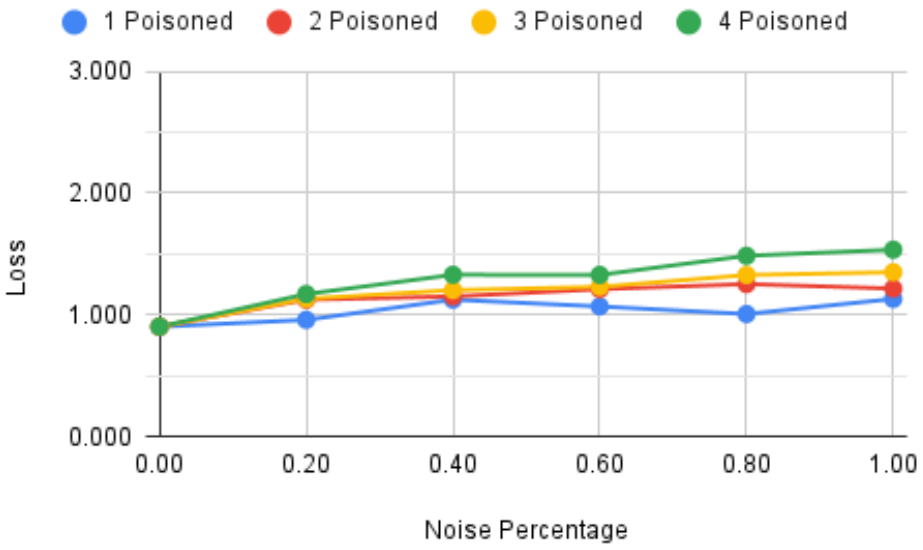


Figure A.45: Quantization - Cross-entropy loss considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table A.46: Quantization - Weighted average precision considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.758			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.743	0.722	0.708	0.699
0.40	0.734	0.718	0.693	0.673
0.60	0.744	0.708	0.687	0.668
0.80	0.730	0.708	0.675	0.656
1.00	0.720	0.706	0.675	0.653

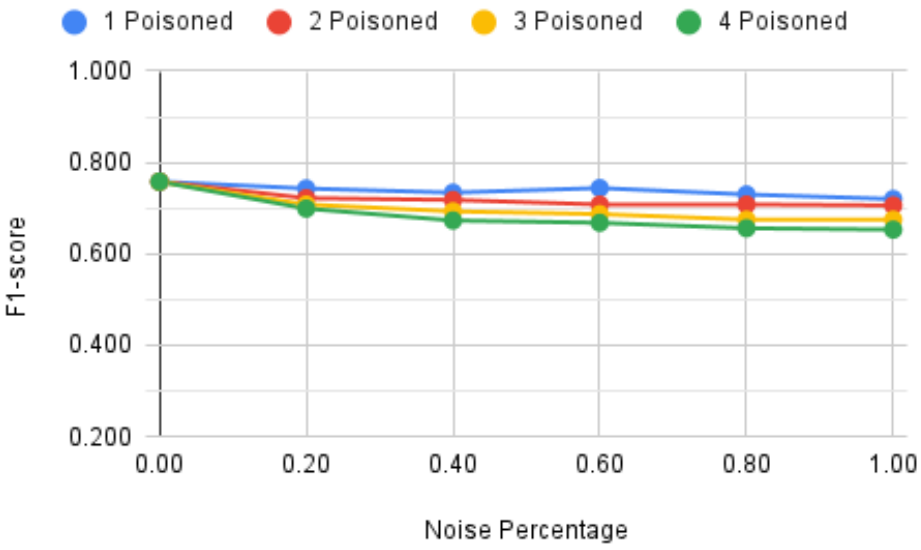


Figure A.46: Quantization - Weighted average precision considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table A.47: Quantization - Weighted average recall considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.752			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.731	0.707	0.689	0.652
0.40	0.705	0.698	0.669	0.607
0.60	0.730	0.685	0.654	0.605
0.80	0.714	0.681	0.605	0.590
1.00	0.699	0.685	0.596	0.596

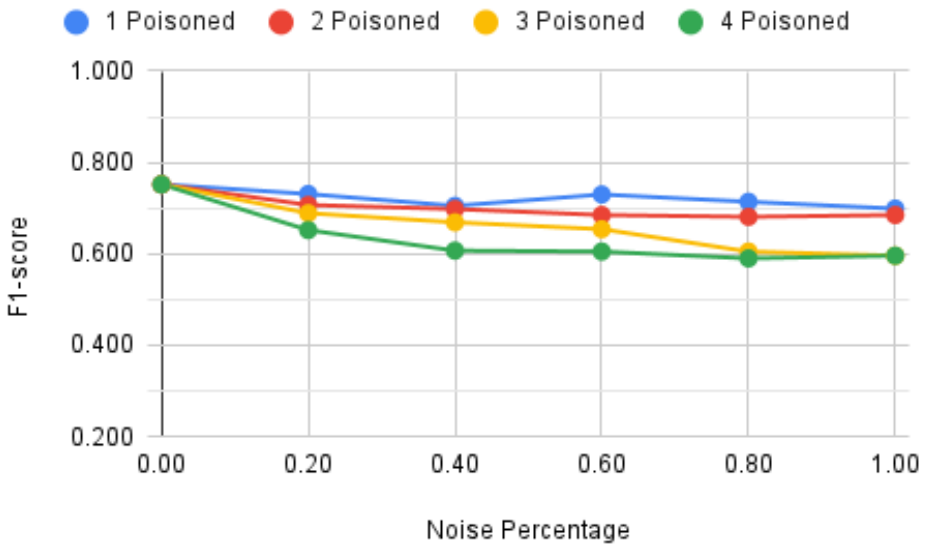


Figure A.47: Quantization - Weighted average recall considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table A.48: Quantization - Accuracy considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.752			
Noise in Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.731	0.707	0.689	0.652
0.40	0.705	0.698	0.669	0.607
0.60	0.730	0.685	0.654	0.605
0.80	0.714	0.681	0.605	0.590
1.00	0.699	0.685	0.596	0.596

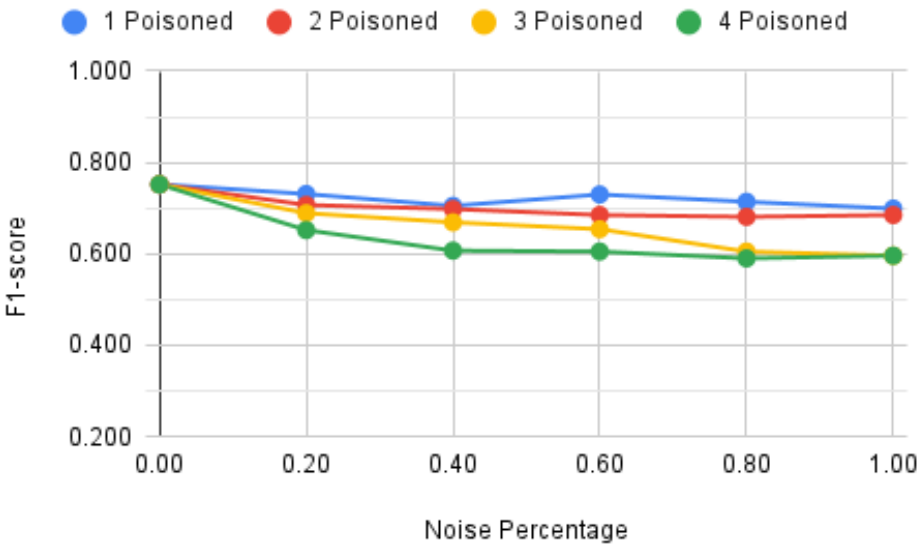


Figure A.48: Quantization - Accuracy considering a minority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

A.13 Complements to Table 6.13 and Figure 6.13

Table A.49: Standard Model - Cross-entropy loss considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.912			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.968	1.077	1.016	0.916
0.40	0.976	1.003	1.016	0.843
0.60	0.909	1.003	0.946	0.843
0.80	0.965	0.906	0.838	0.906
1.00	1.001	0.906	0.838	0.920
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	1	1	3	3
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	1	0	0
1.00	0	1	0	0

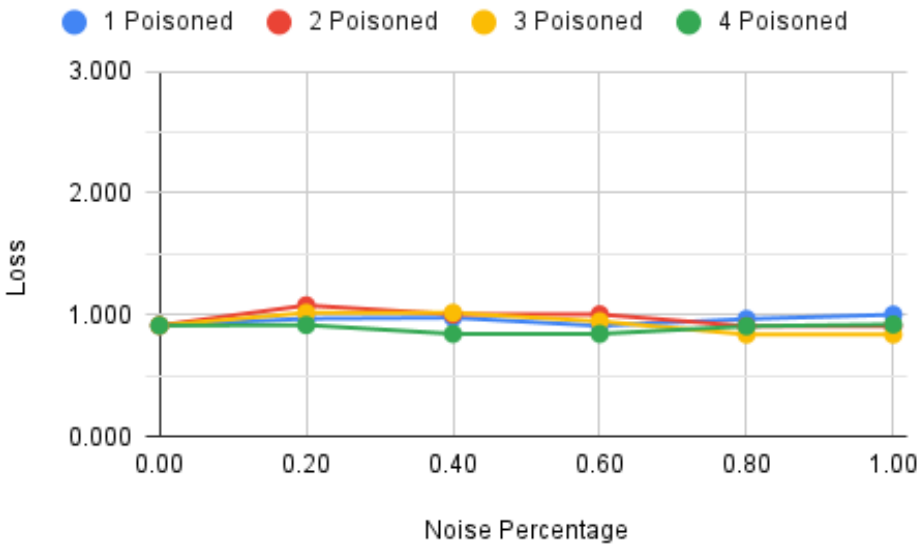


Figure A.49: Standard Model - Cross-entropy loss considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table A.50: Standard Model - Weighted average precision considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.766			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.749	0.742	0.751	0.759
0.40	0.755	0.750	0.751	0.770
0.60	0.760	0.750	0.760	0.770
0.80	0.757	0.766	0.772	0.760
1.00	0.752	0.766	0.772	0.757
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	1	1	3	3
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	1	0	0
1.00	0	1	0	0

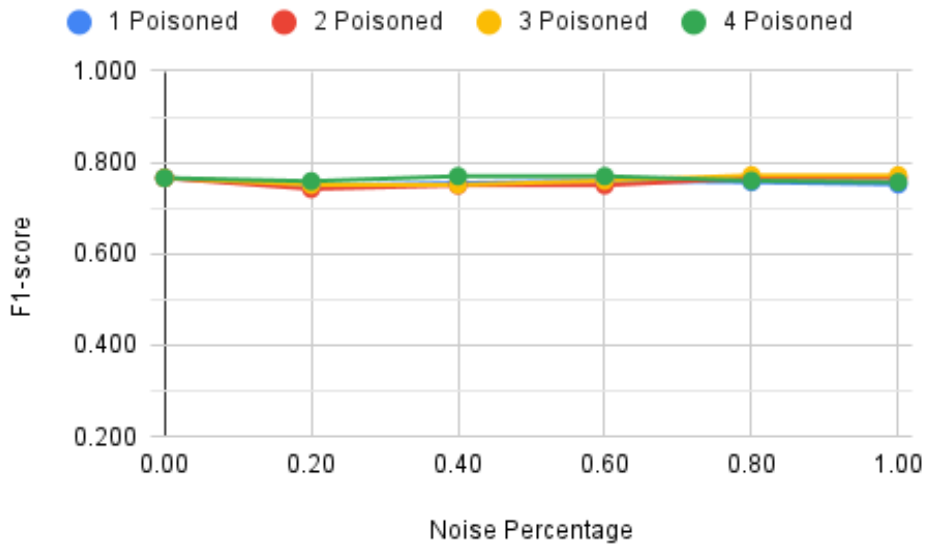


Figure A.50: Standard Model - Weighted average precision considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table A.51: Standard Model - Weighted average recall considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.762			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.743	0.737	0.742	0.753
0.40	0.745	0.747	0.742	0.766
0.60	0.753	0.747	0.754	0.766
0.80	0.749	0.761	0.768	0.756
1.00	0.748	0.761	0.768	0.749
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	1	1	3	3
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	1	0	0
1.00	0	1	0	0

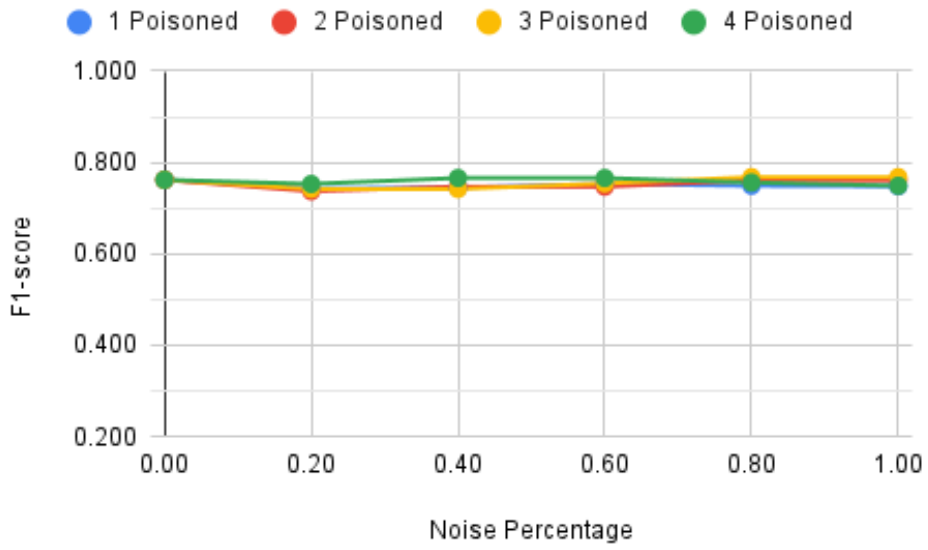


Figure A.51: Standard Model - Weighted average recall considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table A.52: Standard Model - Accuracy considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.762			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.743	0.737	0.742	0.753
0.40	0.745	0.747	0.742	0.766
0.60	0.753	0.747	0.754	0.766
0.80	0.749	0.761	0.768	0.756
1.00	0.748	0.761	0.768	0.749
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	1	1	3	3
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	1	0	0
1.00	0	1	0	0

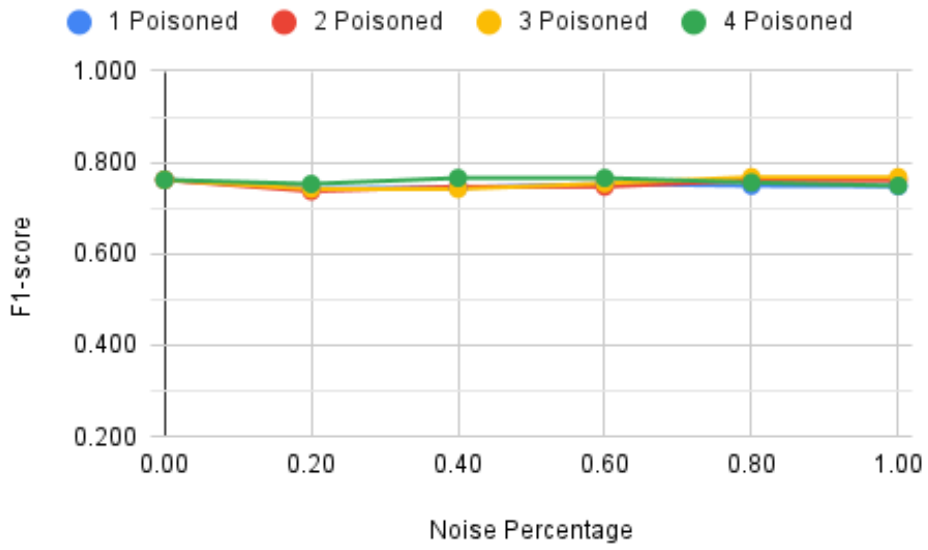


Figure A.52: Standard Model - Accuracy considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

A.14 Complements to Table 6.14 and Figure 6.14

Table A.53: Pruning - Cross-entropy loss considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.878			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.967	1.098	1.093	1.069
0.40	0.949	0.957	0.968	0.814
0.60	0.880	0.957	0.927	0.814
0.80	0.911	0.893	0.813	0.920
1.00	0.950	0.893	0.813	0.875
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	1	0
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	0	0	0
1.00	0	0	0	0

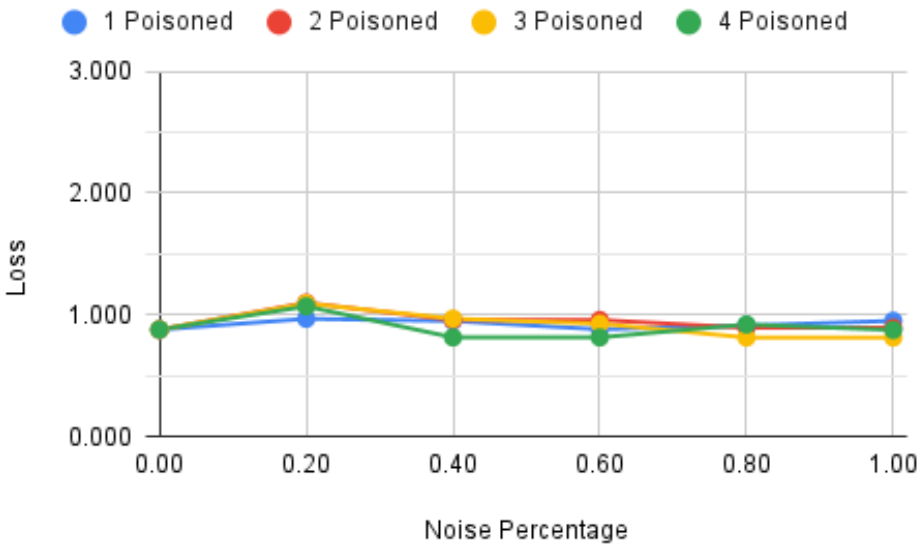


Figure A.53: Pruning - Cross-entropy loss considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table A.54: Pruning - Weighted average precision considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.770			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.750	0.736	0.739	0.735
0.40	0.755	0.754	0.756	0.772
0.60	0.764	0.754	0.759	0.772
0.80	0.765	0.766	0.775	0.758
1.00	0.756	0.766	0.775	0.763
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	1	0
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	0	0	0
1.00	0	0	0	0

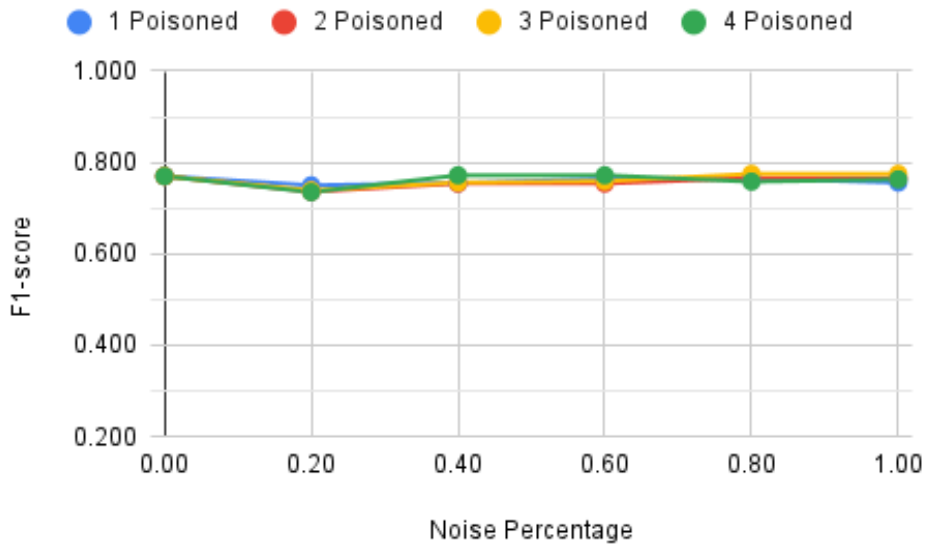


Figure A.54: Pruning - Weighted average precision considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table A.55: Pruning - Weighted average recall considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.743	0.728	0.725	0.722
0.40	0.744	0.749	0.746	0.769
0.60	0.757	0.749	0.754	0.769
0.80	0.759	0.762	0.771	0.751
1.00	0.752	0.762	0.771	0.759
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	1	0
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	0	0	0
1.00	0	0	0	0

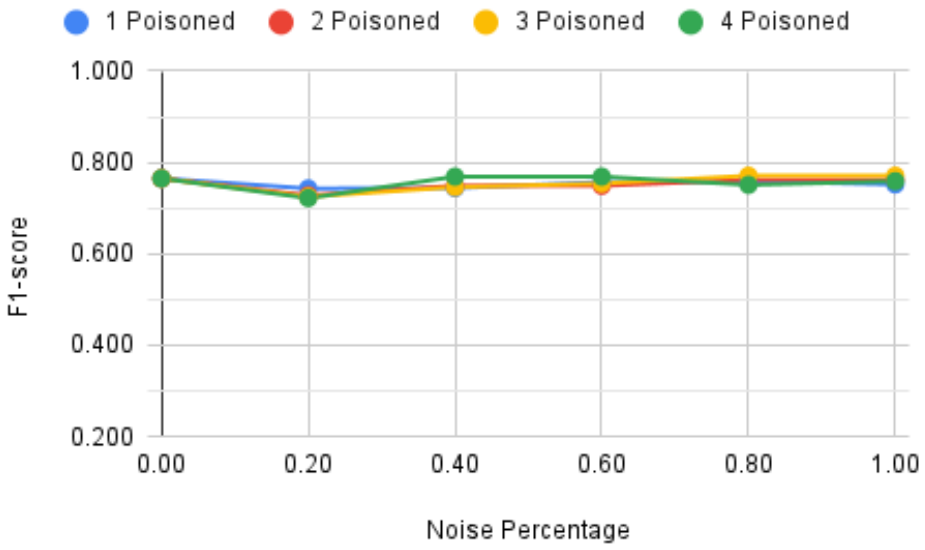


Figure A.55: Pruning - Weighted average recall considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table A.56: Pruning - Accuracy considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.743	0.728	0.725	0.722
0.40	0.744	0.749	0.746	0.769
0.60	0.757	0.749	0.754	0.769
0.80	0.759	0.762	0.771	0.751
1.00	0.752	0.762	0.771	0.759
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	1	0
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	0	0	0
1.00	0	0	0	0

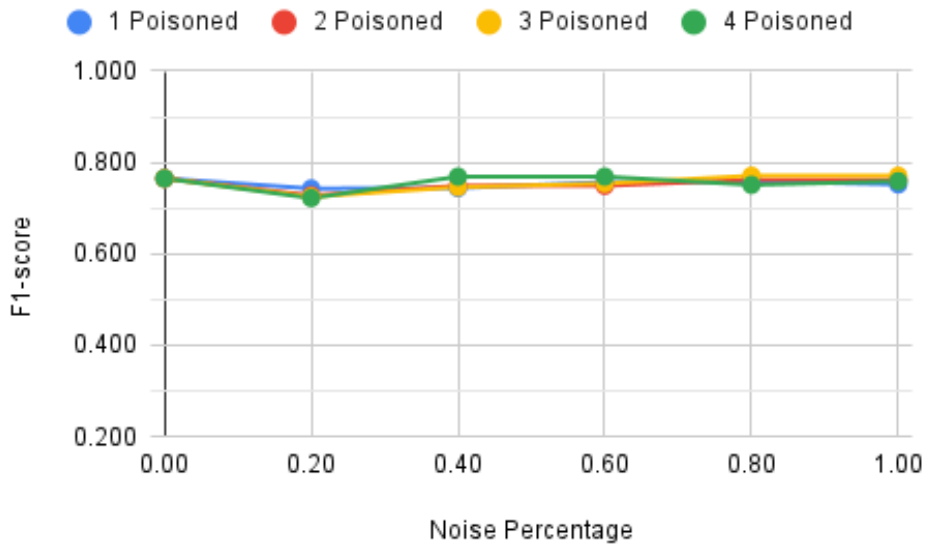


Figure A.56: Pruning - Accuracy considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

A.15 Complements to Table 6.15 and Figure 6.15

Table A.57: LiteRT - Cross-entropy loss considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.878			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.967	1.098	1.093	1.069
0.40	0.949	0.957	0.968	0.814
0.60	0.880	0.957	0.927	0.814
0.80	0.911	0.893	0.813	0.920
1.00	0.950	0.893	0.813	0.875
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	1	0
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	0	0	0
1.00	0	0	0	0

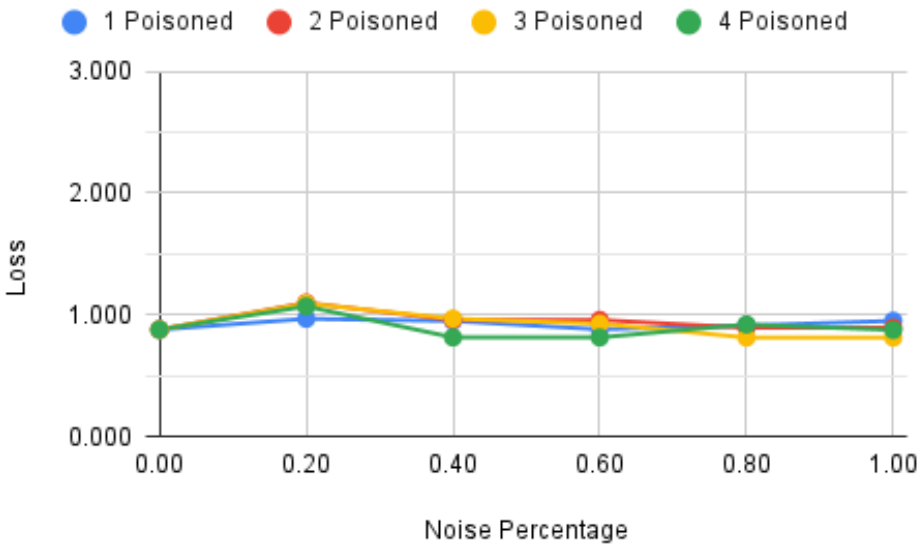


Figure A.57: LiteRT - Cross-entropy loss considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table A.58: LiteRT - Weighted average precision considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.770			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.750	0.736	0.739	0.735
0.40	0.755	0.754	0.756	0.772
0.60	0.764	0.754	0.759	0.772
0.80	0.765	0.766	0.775	0.758
1.00	0.756	0.766	0.775	0.763
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	1	0
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	0	0	0
1.00	0	0	0	0

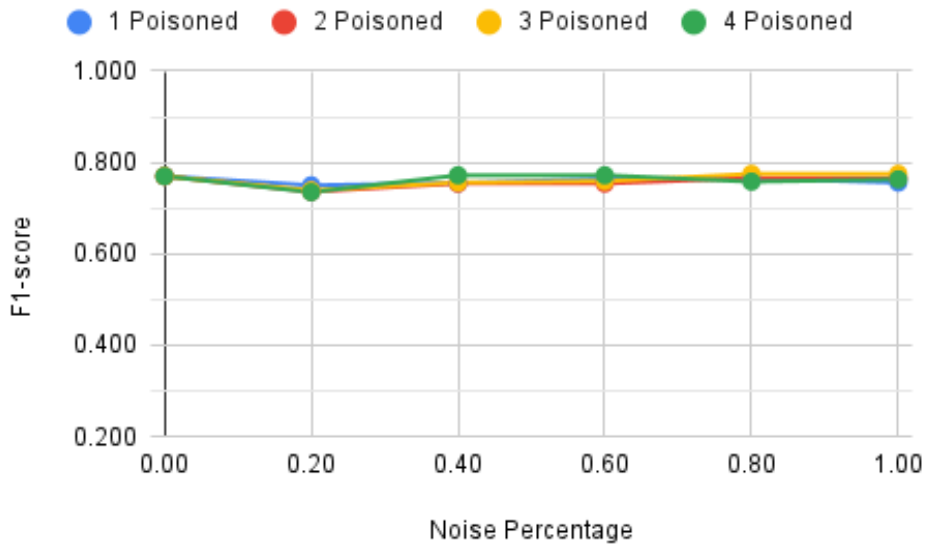


Figure A.58: LiteRT - Weighted average precision considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table A.59: LiteRT - Weighted average recall considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.743	0.728	0.725	0.722
0.40	0.744	0.749	0.746	0.769
0.60	0.757	0.749	0.754	0.769
0.80	0.759	0.762	0.771	0.751
1.00	0.752	0.762	0.771	0.759
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	1	0
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	0	0	0
1.00	0	0	0	0

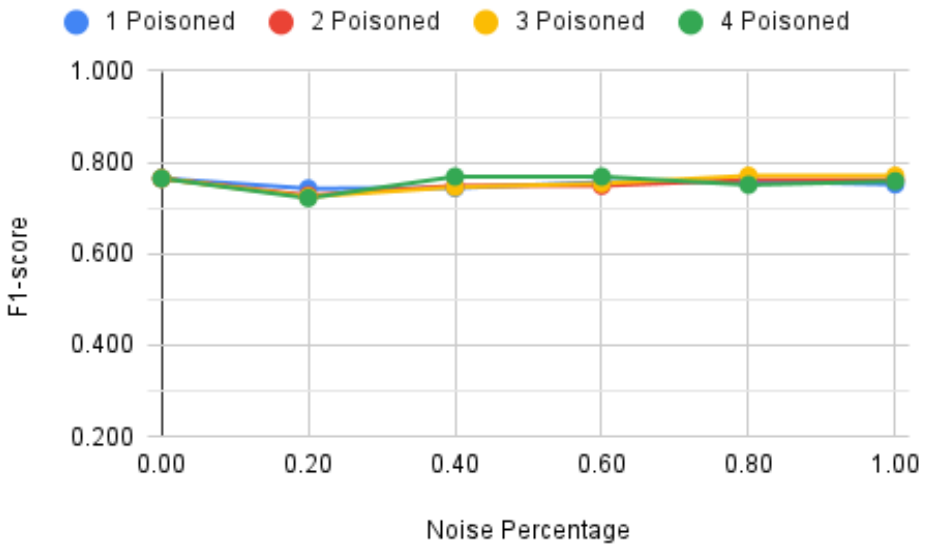


Figure A.59: LiteRT - Weighted average recall considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table A.60: LiteRT - Accuracy considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.743	0.728	0.725	0.722
0.40	0.744	0.749	0.746	0.769
0.60	0.757	0.749	0.754	0.769
0.80	0.759	0.762	0.771	0.751
1.00	0.752	0.762	0.771	0.759
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	1	0
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	0	0	0
1.00	0	0	0	0

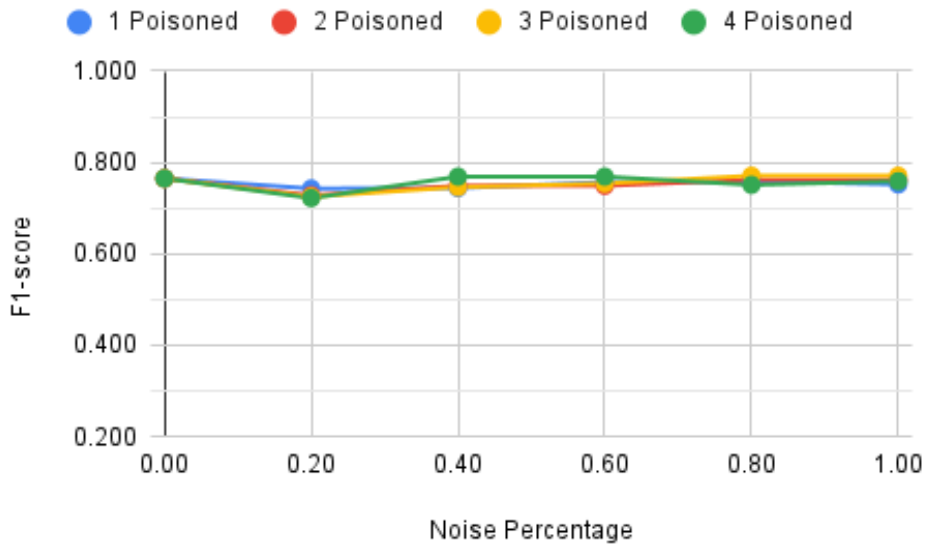


Figure A.60: LiteRT - Accuracy considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

A.16 Complements to Table 6.16 and Figure 6.16

Table A.61: Quantization - Cross-entropy loss considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.902			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.958	1.124	1.083	1.170
0.40	1.053	1.022	0.965	1.058
0.60	0.976	1.022	0.944	1.058
0.80	0.932	0.950	1.009	0.945
1.00	1.013	0.950	1.009	0.956
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	1	0
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	0	0	0
1.00	0	0	0	0

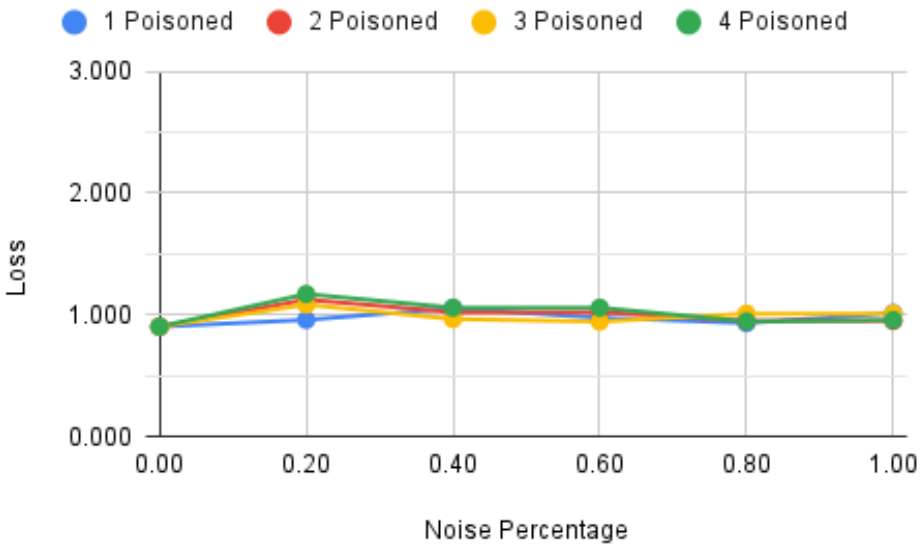


Figure A.61: Quantization - Cross-entropy loss considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table A.62: Quantization - Weighted average precision considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.758			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.743	0.722	0.716	0.699
0.40	0.744	0.740	0.739	0.726
0.60	0.757	0.740	0.743	0.726
0.80	0.745	0.751	0.735	0.746
1.00	0.739	0.751	0.735	0.754
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	1	0
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	0	0	0
1.00	0	0	0	0

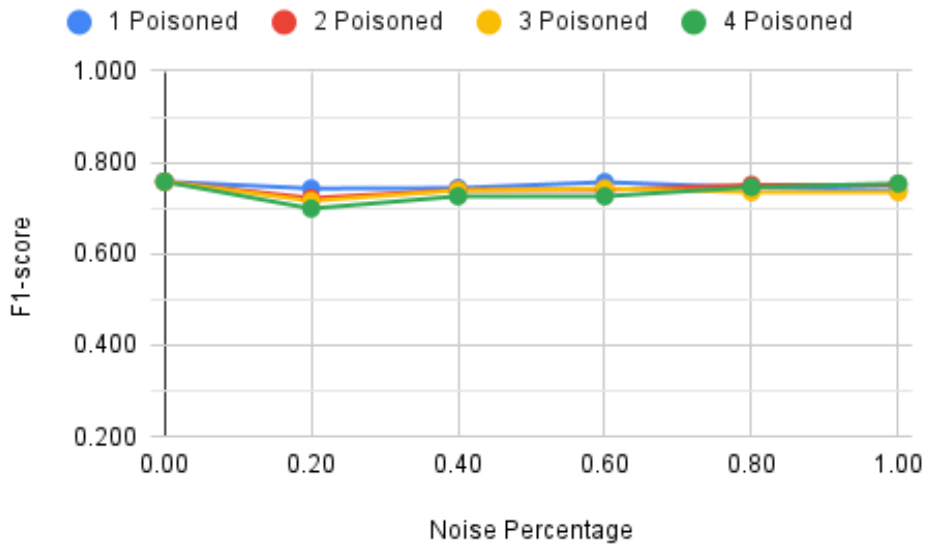


Figure A.62: Quantization - Weighted average precision considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table A.63: Quantization - Weighted average recall considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.752			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.731	0.707	0.701	0.652
0.40	0.724	0.725	0.731	0.692
0.60	0.749	0.725	0.733	0.692
0.80	0.733	0.746	0.706	0.743
1.00	0.725	0.746	0.706	0.747
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	1	0
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	0	0	0
1.00	0	0	0	0

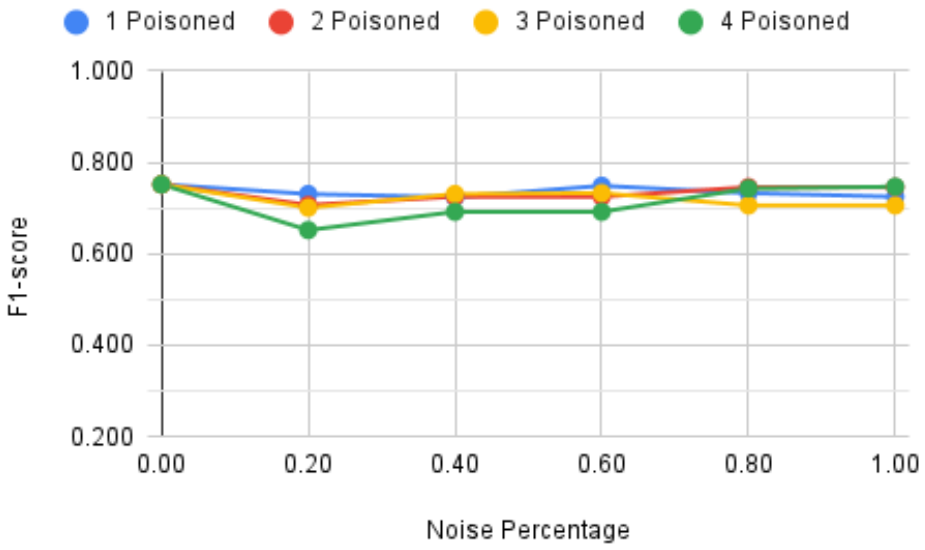


Figure A.63: Quantization - Weighted average recall considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table A.64: Quantization - Accuracy considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.752			
Noise over poisoned nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0.731	0.707	0.701	0.652
0.40	0.724	0.725	0.731	0.692
0.60	0.749	0.725	0.733	0.692
0.80	0.733	0.746	0.706	0.743
1.00	0.725	0.746	0.706	0.747
Discovered Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	1	0
0.40	1	2	3	4
0.60	1	2	3	4
0.80	1	2	3	4
1.00	1	2	3	4
Mismatched Poisoned Nodes	1 Poisoned	2 Poisoned	3 Poisoned	4 Poisoned
0.20	0	0	0	0
0.40	0	0	0	0
0.60	0	0	0	0
0.80	0	0	0	0
1.00	0	0	0	0

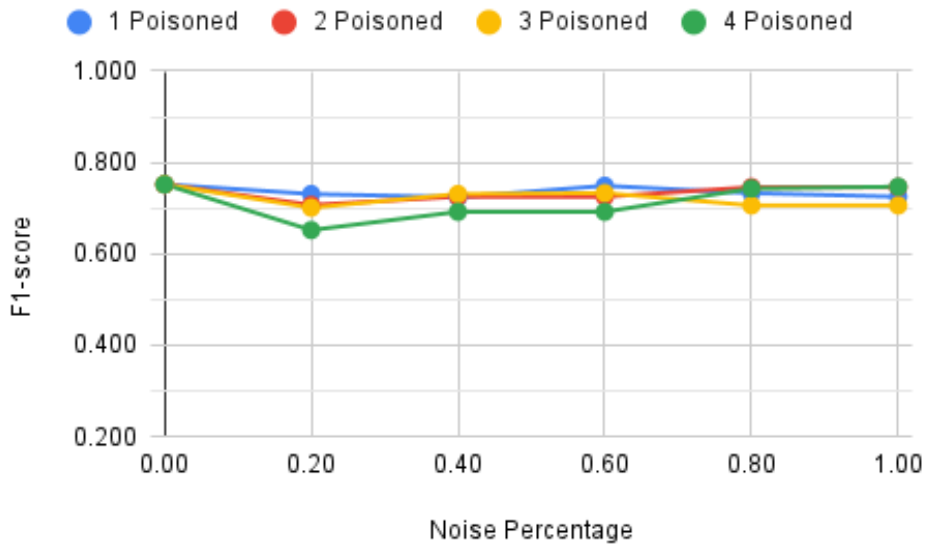


Figure A.64: Quantization - Accuracy considering a minority of poisoned nodes and setting a minimum threshold of 70% F1-score.

A.17 Complements to Table 6.17

Table A.65: Standard Model - Cross-entropy loss and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	0.977 (4)
5	1.085 (1)	1.020 (4)
6	1.142 (2)	1.147 (4)

Table A.66: Standard Model - Weighted average precision and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	0.755 (4)
5	0.744 (1)	0.744 (4)
6	0.744 (2)	0.718 (4)

Table A.67: Standard Model - Weighted average recall and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	0.727 (4)
5	0.727 (1)	0.702 (4)
6	0.706 (2)	0.670 (4)

Table A.68: Standard Model - Accuracy and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	0.727 (4)
5	0.727 (1)	0.702 (4)
6	0.706 (2)	0.670 (4)

A.18 Complements to Table 6.18

Table A.69: Pruning - Cross-entropy loss and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	1.004 (3)
5	-	0.998 (4)
6	1.306 (1)	1.131 (4)

Table A.70: Pruning - Weighted average precision and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	0.746 (3)
5	-	0.746 (4)
6	0.724 (1)	0.721 (4)

Table A.71: Pruning - Weighted average recall and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	0.731 (3)
5	-	0.706 (4)
6	0.693 (1)	0.672 (4)

Table A.72: Pruning - Accuracy and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	0.731 (3)
5	-	0.706 (4)
6	0.693 (1)	0.672 (4)

A.19 Complements to Table 6.19

Table A.73: LiteRT - Cross-entropy loss and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	1.004 (3)
5	-	0.998 (4)
6	1.306 (1)	1.131 (4)

Table A.74: LiteRT - Weighted average precision and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	0.746 (3)
5	-	0.746 (4)
6	0.724 (1)	0.721 (4)

Table A.75: LiteRT - Weighted average recall and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	0.731 (3)
5	-	0.706 (4)
6	0.693 (1)	0.672 (4)

Table A.76: LiteRT - Accuracy and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	0.731 (3)
5	-	0.706 (4)
6	0.693 (1)	0.672 (4)

A.20 Complements to Table 6.20

Table A.77: Quantization - Cross-entropy loss and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	1.025 (3)
5	-	1.029 (4)
6	1.358 (1)	1.194 (4)

Table A.78: Quantization - Weighted average precision and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	0.735 (3)
5	-	0.741 (4)
6	0.716 (1)	0.716 (4)

Table A.79: Quantization - Weighted average recall and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	0.708 (3)
5	-	0.695 (4)
6	0.688 (1)	0.666 (4)

Table A.80: Quantization - Accuracy and corresponding number of mismatched nodes considering unbalanced data, with safeguards against poisoning attacks.

Unbalance Level	7 Nodes (Mismatched)	9 Nodes (Mismatched)
4	-	0.708 (3)
5	-	0.695 (4)
6	0.688 (1)	0.666 (4)

Appendix B - Majority Poisoned Nodes Results

This appendix presents additional results that complement the experiments described in Subsections 6.4.1 and 6.4.2. While those subsections focus on scenarios where only a minority of nodes are compromised within the DFL system, this appendix provides corresponding results for cases where the majority of nodes are affected under the same experimental conditions.

The appendix is organized into eight sections. The first four sections correspond to the experiments outlined in Subsection 6.4.1, where no protective measures against poisoning attacks were employed. The first section presents results related to the standard model, the second for the pruned model, the third for the standard LiteRT-converted model, and the fourth related to the quantized model.

The remaining four sections pertain to the experiments detailed in Subsection 6.4.2, where protective measures against poisoning attacks were implemented. Following the same order as the previous group, the fifth section presents results related to the standard model, the sixth for the pruned model, the seventh for the standard LiteRT-converted model, and the eight related to the quantized model. Consistent with the rest of the document, captions for the tables are placed above them, while captions for the figures are positioned below.

B.1 Standard Model - Majority of Poisoned Nodes Without Threshold

Table B.1: Standard Model - Weighted average F1-score considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.762				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.699	0.671	0.646	0.637	0.649
0.40	0.597	0.596	0.597	0.556	0.556
0.60	0.596	0.572	0.553	0.524	0.463
0.80	0.539	0.521	0.443	0.474	0.375
1.00	0.528	0.497	0.389	0.421	0.357

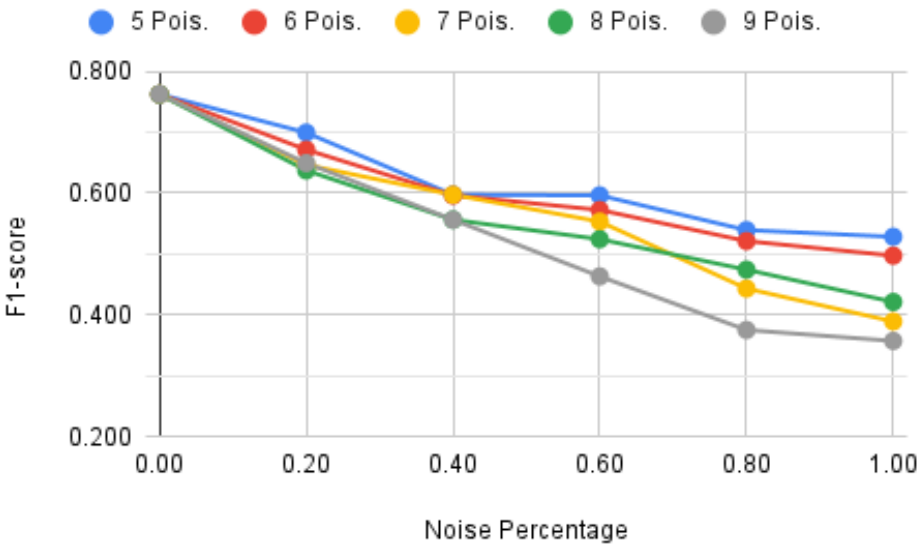


Figure B.1: Standard Model - Weighted average F1-score considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table B.2: Standard Model - Cross-entropy loss considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.912				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	1.230	1.367	1.405	1.456	1.423
0.40	1.487	1.630	1.586	1.731	1.664
0.60	1.518	1.591	1.698	1.736	1.920
0.80	1.650	1.690	1.941	1.855	2.114
1.00	1.665	1.773	2.064	1.997	2.163

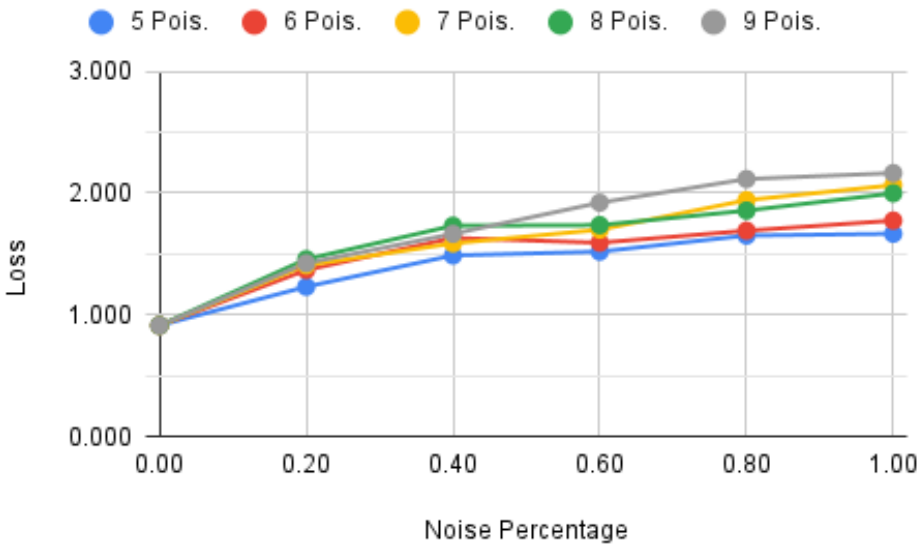


Figure B.2: Standard Model - Cross-entropy loss considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table B.3: Standard Model - Weighted average precision considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.766				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.720	0.701	0.688	0.679	0.683
0.40	0.660	0.658	0.656	0.646	0.638
0.60	0.658	0.645	0.637	0.620	0.598
0.80	0.633	0.625	0.588	0.598	0.572
1.00	0.622	0.607	0.571	0.573	0.554

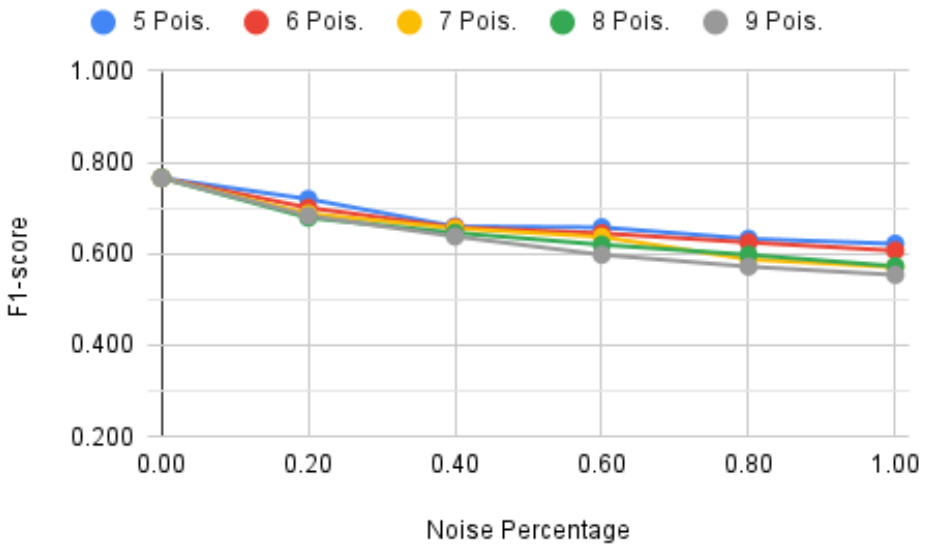


Figure B.3: Standard Model - Weighted average precision considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table B.4: Standard Model - Weighted average recall considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.762				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.700	0.671	0.650	0.638	0.651
0.40	0.599	0.598	0.599	0.553	0.557
0.60	0.597	0.572	0.554	0.526	0.467
0.80	0.544	0.522	0.445	0.475	0.384
1.00	0.535	0.499	0.397	0.423	0.369

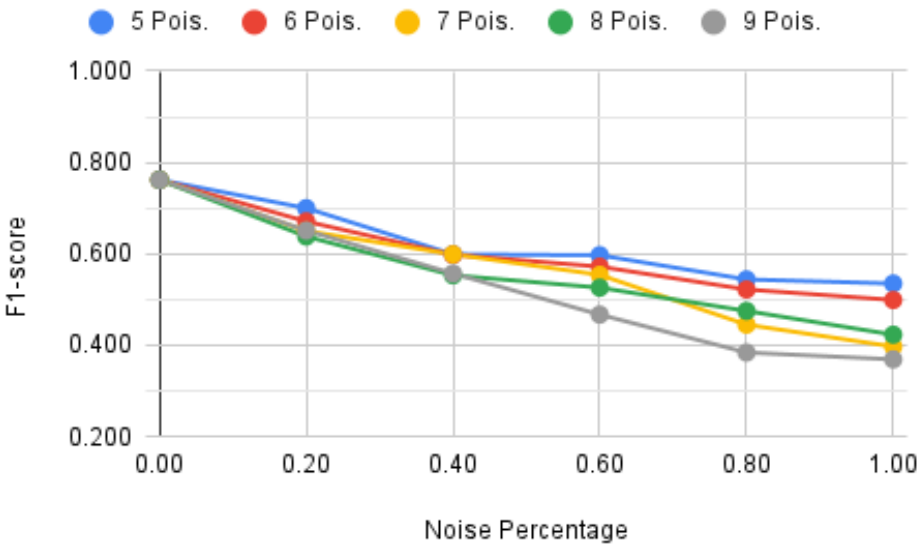


Figure B.4: Standard Model - Weighted average recall considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table B.5: Standard Model - Accuracy considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.762				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.700	0.671	0.650	0.638	0.651
0.40	0.599	0.598	0.599	0.553	0.557
0.60	0.597	0.572	0.554	0.526	0.467
0.80	0.544	0.522	0.445	0.475	0.384
1.00	0.535	0.499	0.397	0.423	0.369

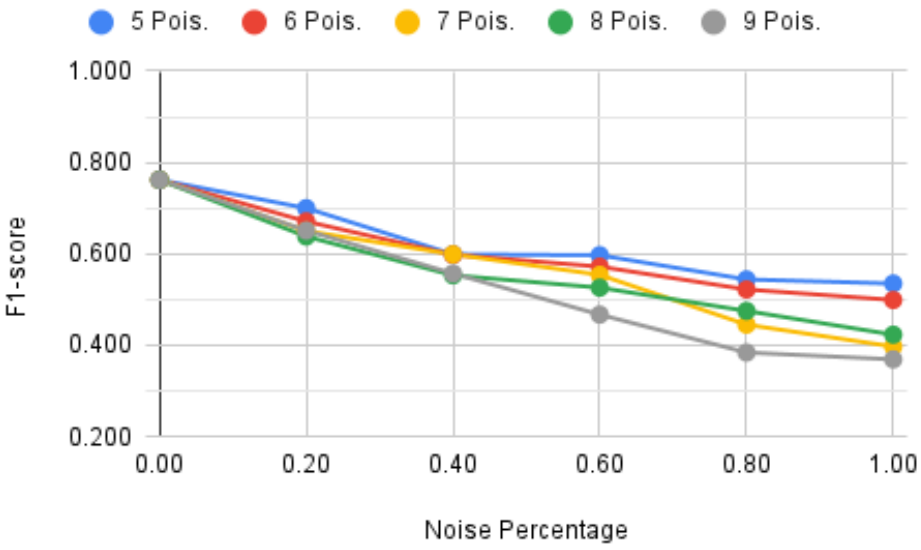


Figure B.5: Standard Model - Accuracy considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

B.2 Pruning - Majority of Poisoned Nodes Without Threshold

Table B.6: Pruning - Weighted average F1-score considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.695	0.678	0.655	0.636	0.647
0.40	0.599	0.626	0.599	0.560	0.553
0.60	0.601	0.570	0.559	0.520	0.478
0.80	0.572	0.518	0.459	0.458	0.395
1.00	0.554	0.480	0.408	0.405	0.363

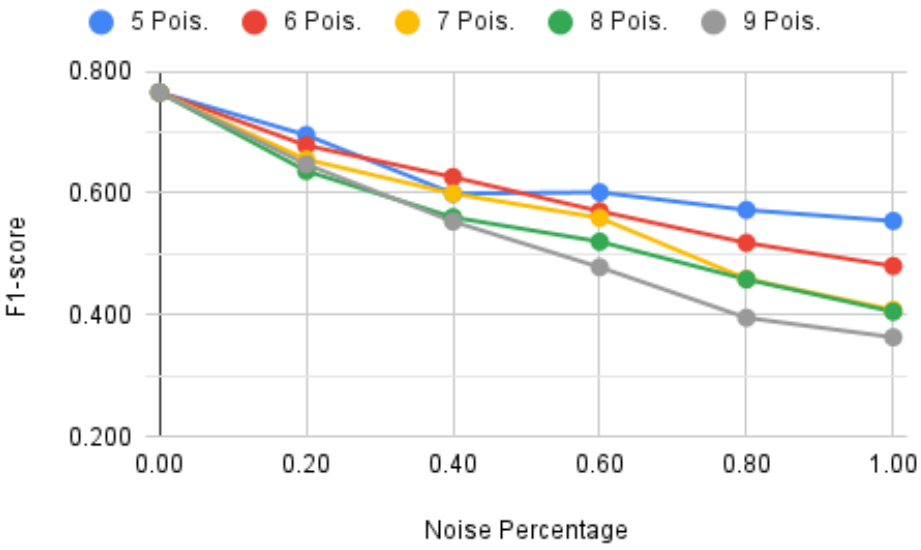


Figure B.6: Pruning - Weighted average F1-score considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table B.7: Pruning - Cross-entropy loss considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.878				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	1.184	1.315	1.352	1.413	1.400
0.40	1.468	1.524	1.528	1.675	1.639
0.60	1.484	1.562	1.650	1.721	1.856
0.80	1.568	1.676	1.892	1.875	2.060
1.00	1.594	1.785	2.013	2.022	2.130

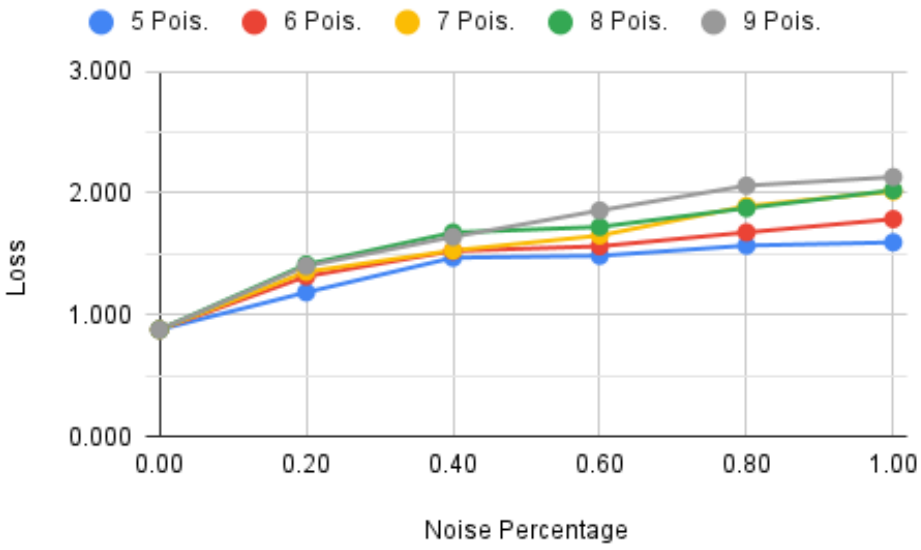


Figure B.7: Pruning - Cross-entropy loss considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table B.8: Pruning - Weighted average precision considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.770				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.714	0.703	0.693	0.677	0.683
0.40	0.661	0.665	0.657	0.646	0.636
0.60	0.658	0.648	0.638	0.617	0.603
0.80	0.644	0.626	0.592	0.592	0.568
1.00	0.629	0.605	0.576	0.575	0.549

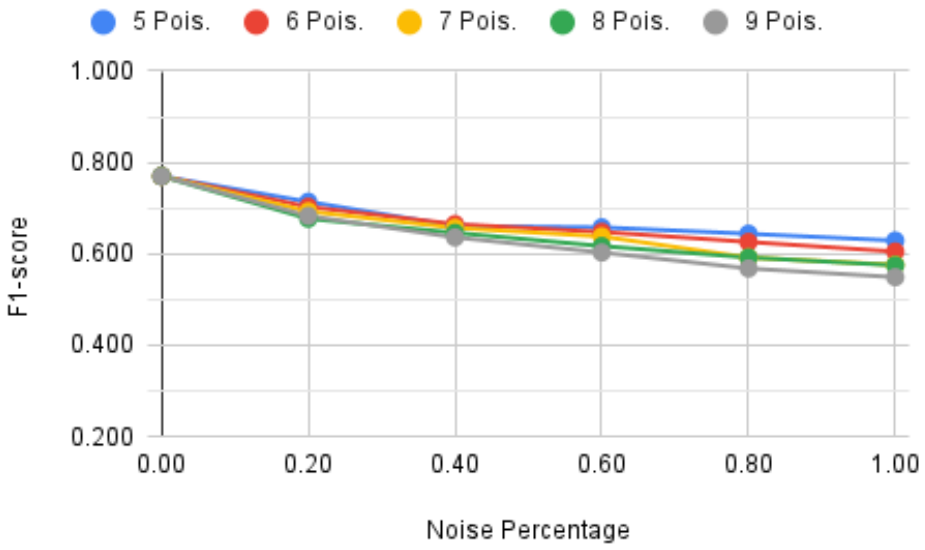


Figure B.8: Pruning - Weighted average precision considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table B.9: Pruning - Weighted average recall considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.698	0.678	0.658	0.639	0.649
0.40	0.600	0.629	0.602	0.558	0.555
0.60	0.602	0.569	0.559	0.524	0.483
0.80	0.574	0.519	0.459	0.462	0.397
1.00	0.559	0.481	0.414	0.409	0.373

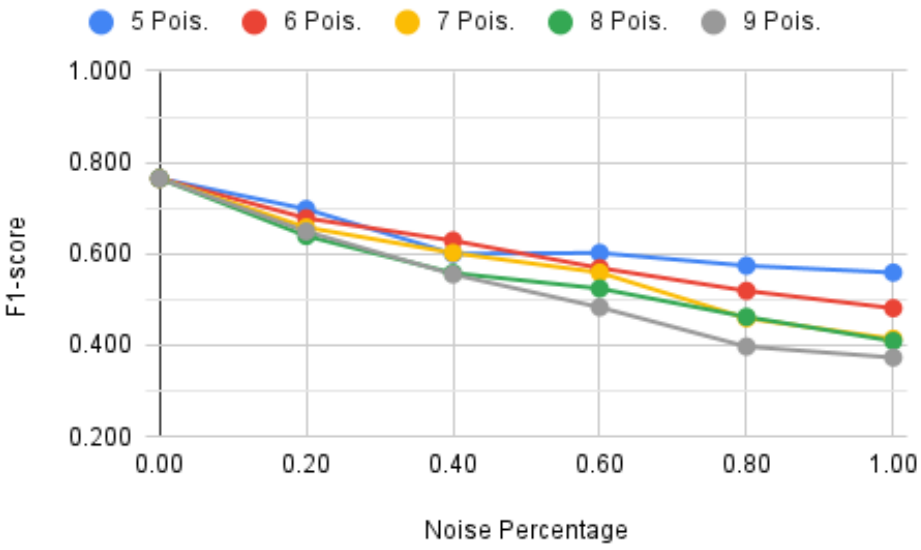


Figure B.9: Pruning - Weighted average recall considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table B.10: Pruning - Accuracy considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.698	0.678	0.658	0.639	0.649
0.40	0.600	0.629	0.602	0.558	0.555
0.60	0.602	0.569	0.559	0.524	0.483
0.80	0.574	0.519	0.459	0.462	0.397
1.00	0.559	0.481	0.414	0.409	0.373

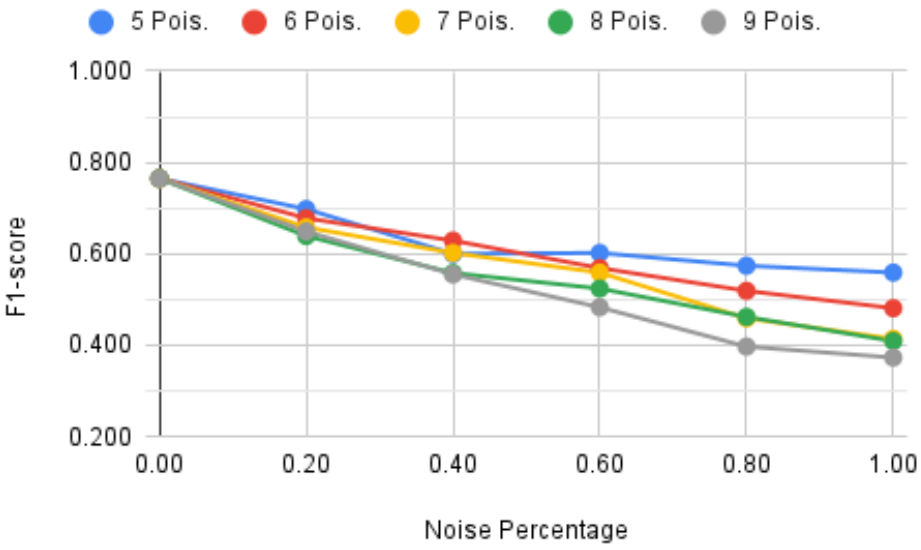


Figure B.10: Pruning - Accuracy considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

B.3 LiteRT - Majority of Poisoned Nodes Without Threshold

Table B.11: LiteRT - Weighted average F1-score considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.695	0.678	0.655	0.636	0.647
0.40	0.593	0.627	0.598	0.561	0.553
0.60	0.598	0.570	0.557	0.518	0.478
0.80	0.572	0.518	0.458	0.458	0.395
1.00	0.556	0.479	0.407	0.405	0.363

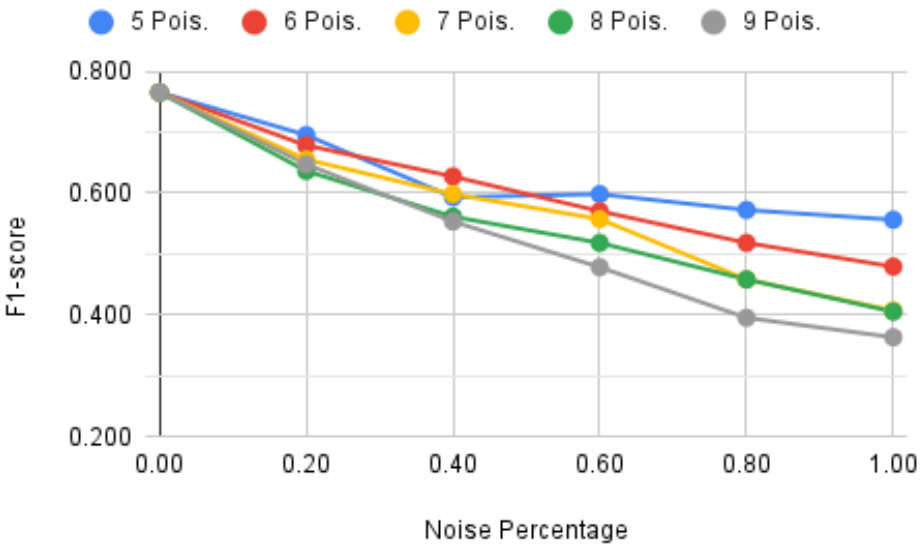


Figure B.11: LiteRT - Weighted average F1-score considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table B.12: LiteRT - Cross-entropy loss considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.878				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	1.184	1.315	1.352	1.413	1.400
0.40	5.574	3.339	2.013	1.484	1.639
0.60	5.508	4.300	2.342	1.663	1.856
0.80	5.863	5.014	3.018	1.942	2.060
1.00	6.134	5.570	3.521	2.241	2.130

Please note: The figure below employs a different scale compared to most of the graphs in this report. While the Y-axis of most graphs ranges from 0 to 3, this specific graph has been adjusted to a scale of 0 to 7 to account for the presence of outlier results.

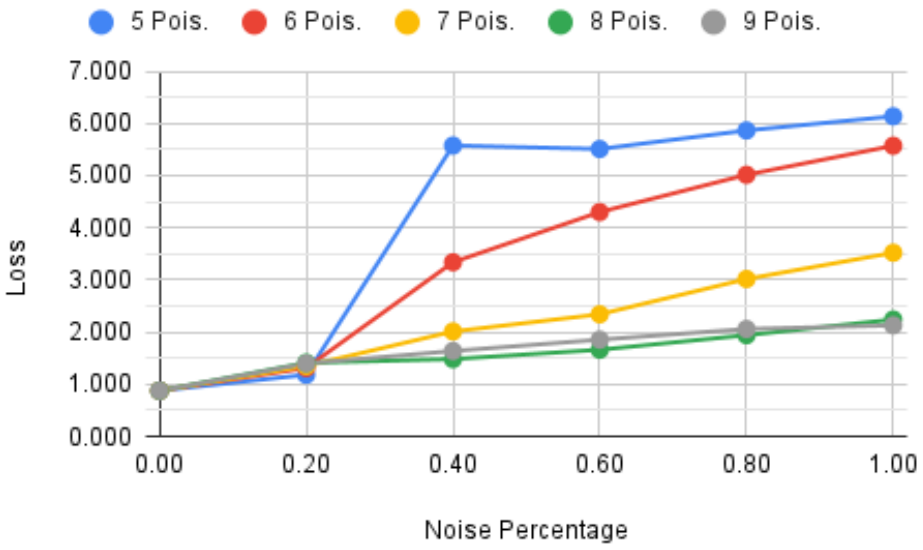


Figure B.12: LiteRT - Cross-entropy loss considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table B.13: LiteRT - Weighted average precision considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.770				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.714	0.703	0.693	0.677	0.683
0.40	0.658	0.667	0.656	0.647	0.636
0.60	0.656	0.649	0.639	0.615	0.603
0.80	0.643	0.627	0.591	0.592	0.568
1.00	0.630	0.608	0.576	0.577	0.549

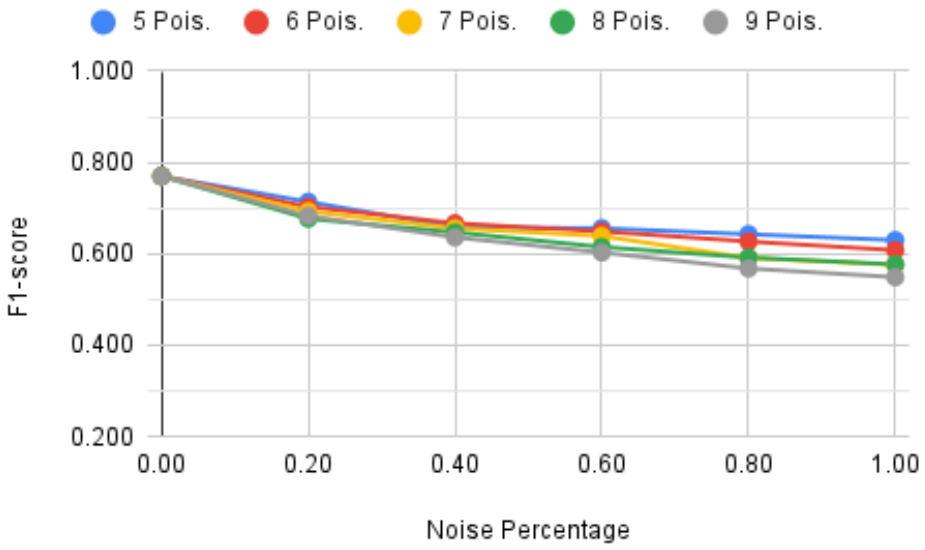


Figure B.13: LiteRT - Weighted average precision considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table B.14: LiteRT - Weighted average recall considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.698	0.678	0.658	0.639	0.649
0.40	0.595	0.629	0.601	0.559	0.555
0.60	0.599	0.568	0.557	0.522	0.483
0.80	0.574	0.519	0.459	0.462	0.397
1.00	0.560	0.481	0.414	0.409	0.373

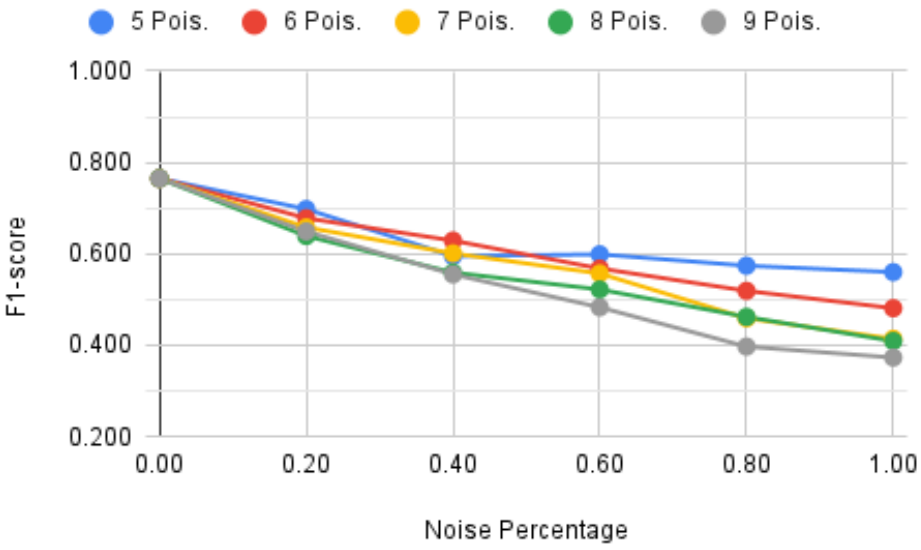


Figure B.14: LiteRT - Weighted average recall considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table B.15: LiteRT - Accuracy considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.698	0.678	0.658	0.639	0.649
0.40	0.595	0.629	0.601	0.559	0.555
0.60	0.599	0.568	0.557	0.522	0.483
0.80	0.574	0.519	0.459	0.462	0.397
1.00	0.560	0.481	0.414	0.409	0.373

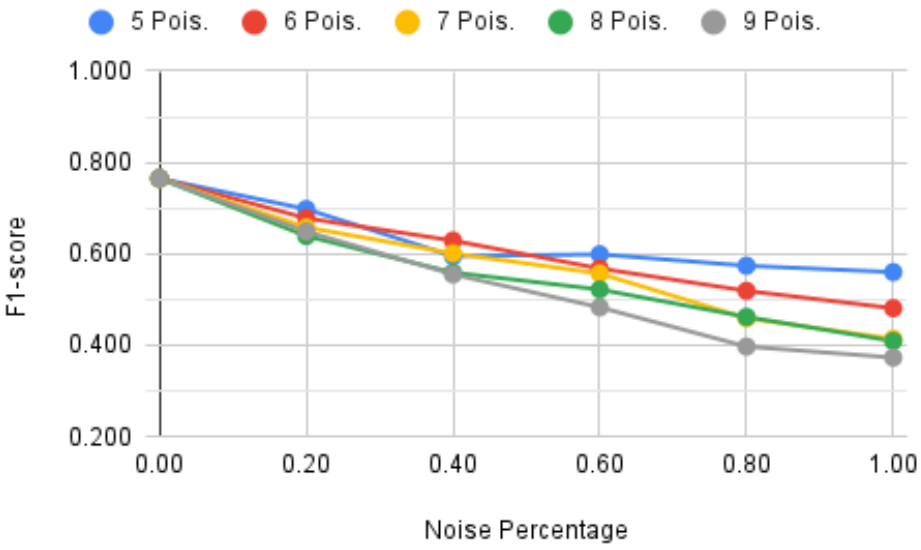


Figure B.15: LiteRT - Accuracy considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

B.4 Quantization - Majority of Poisoned Nodes Without Threshold

Table B.16: Quantization - Weighted average F1-score considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.751				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.684	0.645	0.607	0.601	0.627
0.40	0.577	0.611	0.572	0.495	0.540
0.60	0.554	0.529	0.527	0.471	0.499
0.80	0.546	0.467	0.404	0.421	0.387
1.00	0.537	0.426	0.365	0.355	0.370

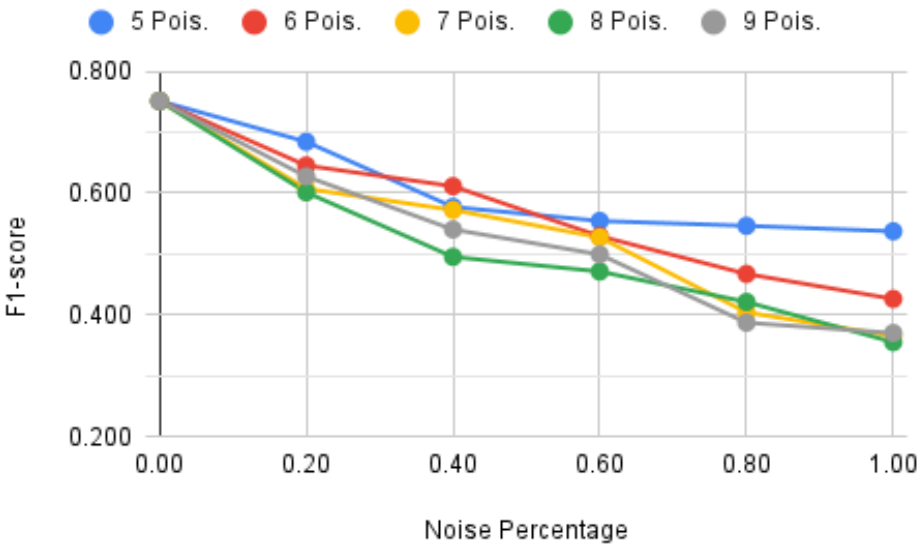


Figure B.16: Quantization - Weighted average F1-score considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table B.17: Quantization - Cross-entropy loss considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.902				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	1.297	1.377	1.456	1.482	1.349
0.40	1.496	1.632	1.604	1.801	1.657
0.60	1.577	1.694	1.726	1.789	1.830
0.80	1.648	1.860	1.982	1.942	2.133
1.00	1.643	1.968	2.082	2.152	2.112

Please note: The figure below employs a different scale compared to most of the graphs in this report. While the Y-axis of most graphs ranges from 0 to 3, this specific graph has been adjusted to a scale of 0 to 7 to account for the presence of outlier results.

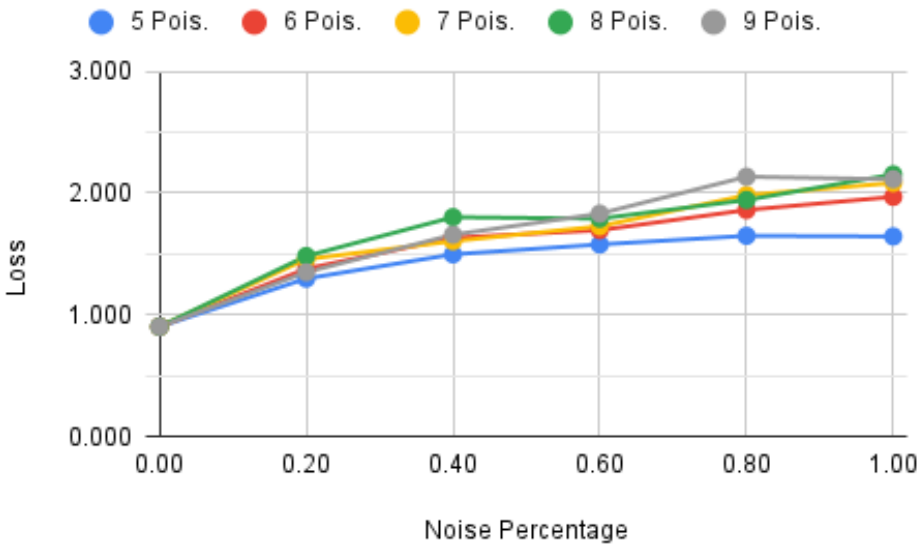


Figure B.17: Quantization - Cross-entropy loss considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table B.18: Quantization - Weighted average precision considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.758				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.706	0.684	0.669	0.668	0.677
0.40	0.658	0.652	0.644	0.626	0.636
0.60	0.646	0.638	0.626	0.606	0.597
0.80	0.635	0.616	0.575	0.581	0.554
1.00	0.623	0.598	0.554	0.565	0.540

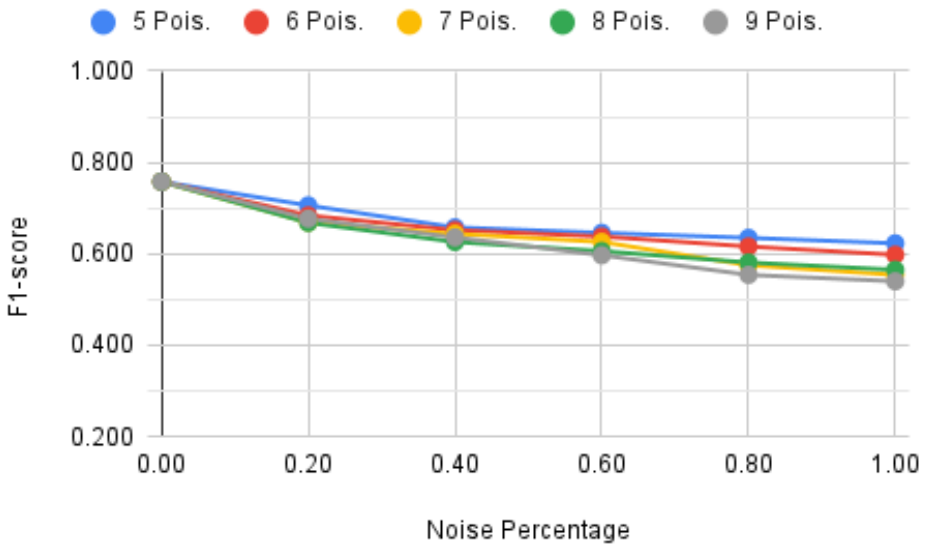


Figure B.18: Quantization - Weighted average precision considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table B.19: Quantization - Weighted average recall considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.752				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.687	0.647	0.611	0.600	0.629
0.40	0.578	0.613	0.572	0.494	0.539
0.60	0.555	0.525	0.527	0.470	0.501
0.80	0.548	0.467	0.414	0.420	0.391
1.00	0.541	0.426	0.380	0.354	0.381

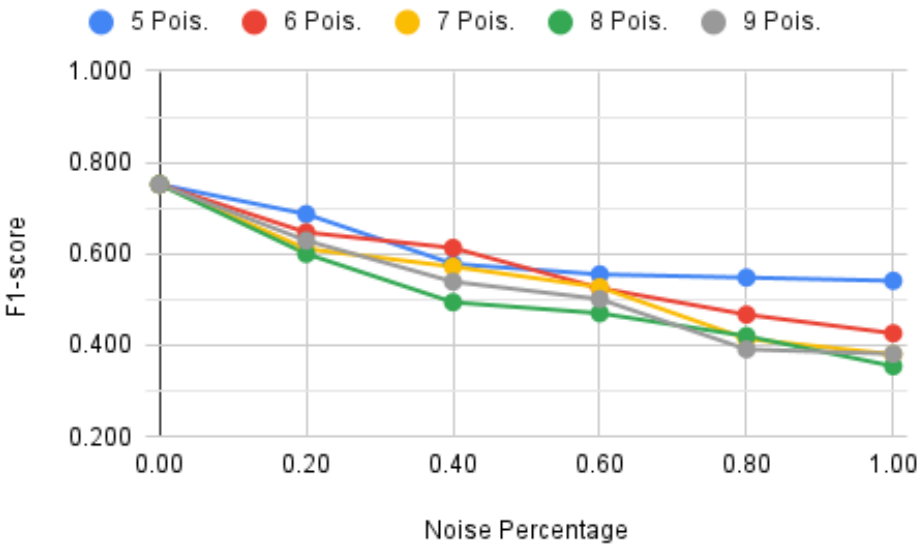


Figure B.19: Quantization - Weighted average recall considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

Table B.20: Quantization - Accuracy considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.752				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.687	0.647	0.611	0.600	0.629
0.40	0.578	0.613	0.572	0.494	0.539
0.60	0.555	0.525	0.527	0.470	0.501
0.80	0.548	0.467	0.414	0.420	0.391
1.00	0.541	0.426	0.380	0.354	0.381

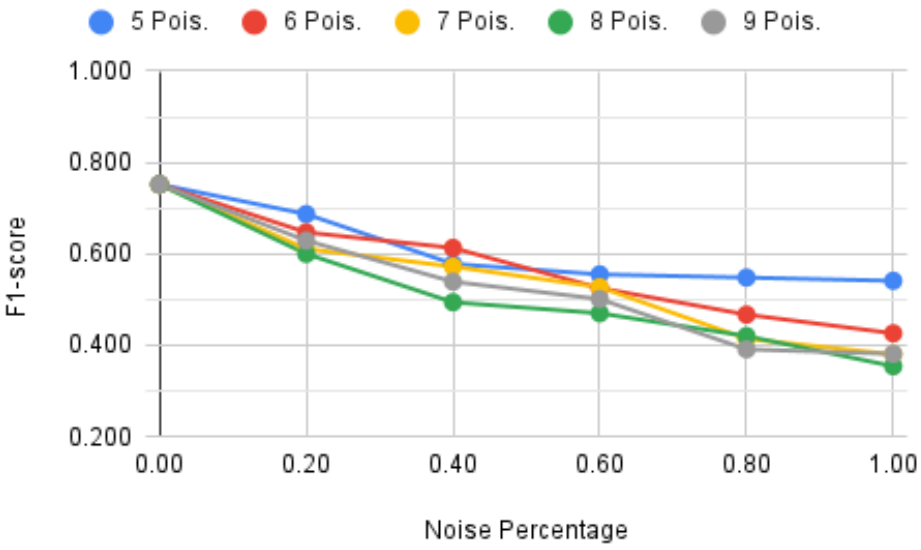


Figure B.20: Quantization - Accuracy considering a majority of poisoned nodes and without setting a minimum threshold of 70% F1-score.

B.5 Standard Model - Majority of Poisoned Nodes With Threshold

Table B.21: Standard Model - Weighted average F1-score considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.762				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.682	0.671	0.646	0.637	0.649
0.40	0.451	0.469	0.543	0.529	0.553
0.60	0.438	0.455	0.490	0.489	0.468
0.80	0.381	0.395	0.383	0.440	0.361
1.00	0.335	0.331	0.320	0.389	0.359
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	1	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	1
0.80	2	0	1	0	1
1.00	2	3	2	2	1
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	2	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

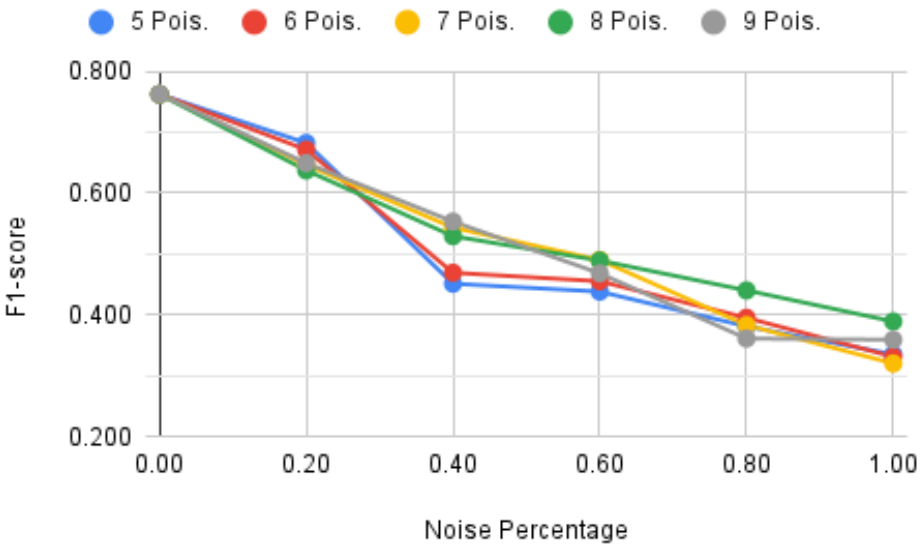


Figure B.21: Standard Model - Weighted average F1-score considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table B.22: Standard Model - Cross-entropy loss considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.912				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	1.255	1.367	1.405	1.456	1.423
0.40	1.912	1.947	1.711	1.807	1.664
0.60	1.928	1.886	1.873	1.823	1.898
0.80	2.046	2.019	2.112	1.953	2.139
1.00	2.240	2.194	2.284	2.066	2.164
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	1	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	1
0.80	2	0	1	0	1
1.00	2	3	2	2	1
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	2	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

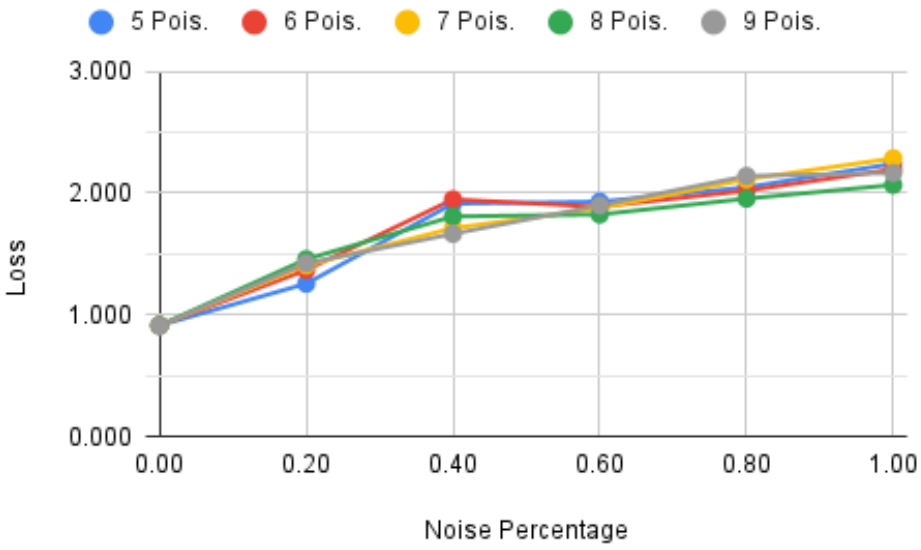


Figure B.22: Standard Model - Cross-entropy loss considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table B.23: Standard Model - Weighted average precision considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.766				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.708	0.701	0.688	0.679	0.683
0.40	0.607	0.602	0.633	0.633	0.638
0.60	0.593	0.593	0.601	0.596	0.594
0.80	0.542	0.575	0.548	0.573	0.570
1.00	0.539	0.554	0.539	0.546	0.558
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	1	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	1
0.80	2	0	1	0	1
1.00	2	3	2	2	1
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	2	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

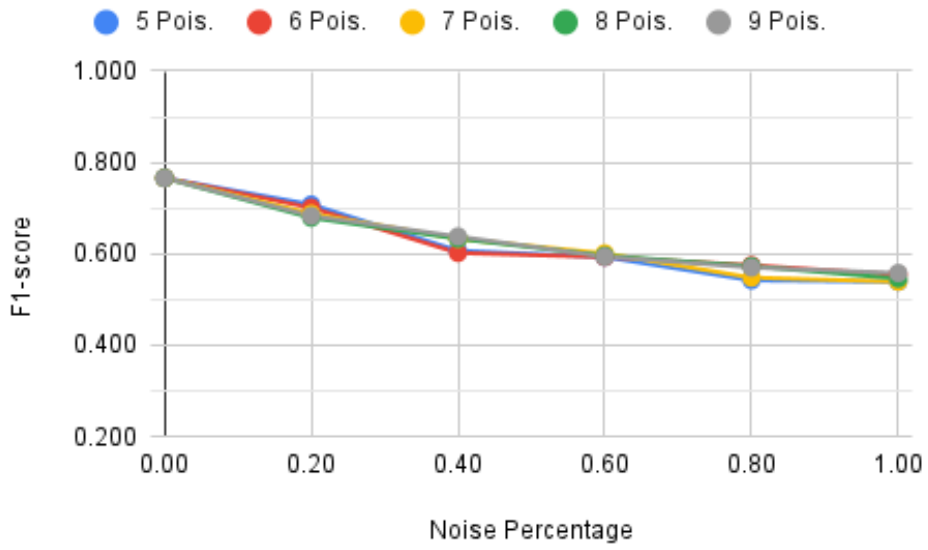


Figure B.23: Standard Model - Weighted average precision considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table B.24: Standard Model - Weighted average recall considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.762				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.684	0.671	0.650	0.638	0.651
0.40	0.449	0.469	0.546	0.527	0.557
0.60	0.443	0.460	0.494	0.493	0.471
0.80	0.391	0.407	0.385	0.440	0.371
1.00	0.350	0.351	0.327	0.389	0.372
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	1	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	1
0.80	2	0	1	0	1
1.00	2	3	2	2	1
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	2	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

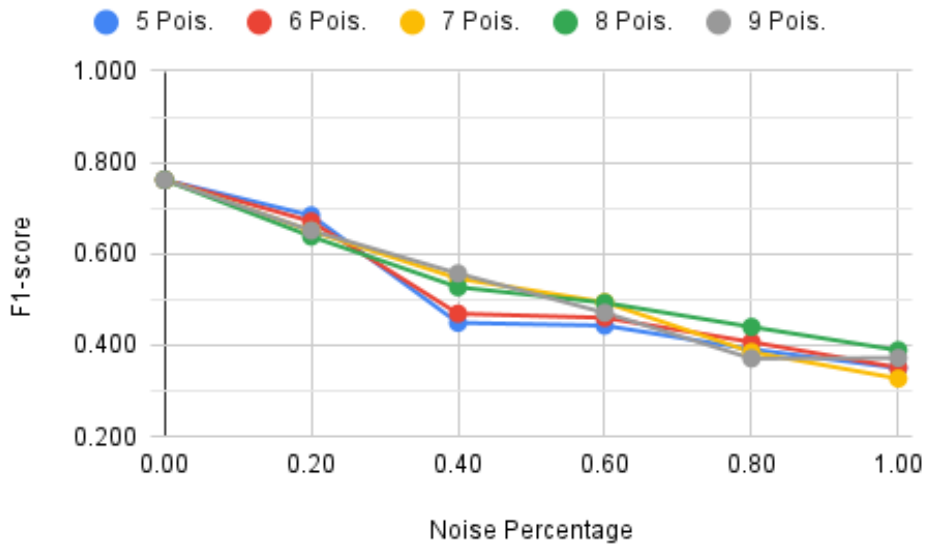


Figure B.24: Standard Model - Weighted average recall considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table B.25: Standard Model - Accuracy considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.762				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.684	0.671	0.650	0.638	0.651
0.40	0.449	0.469	0.546	0.527	0.557
0.60	0.443	0.460	0.494	0.493	0.471
0.80	0.391	0.407	0.385	0.440	0.371
1.00	0.350	0.351	0.327	0.389	0.372
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	1	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	1
0.80	2	0	1	0	1
1.00	2	3	2	2	1
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	2	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

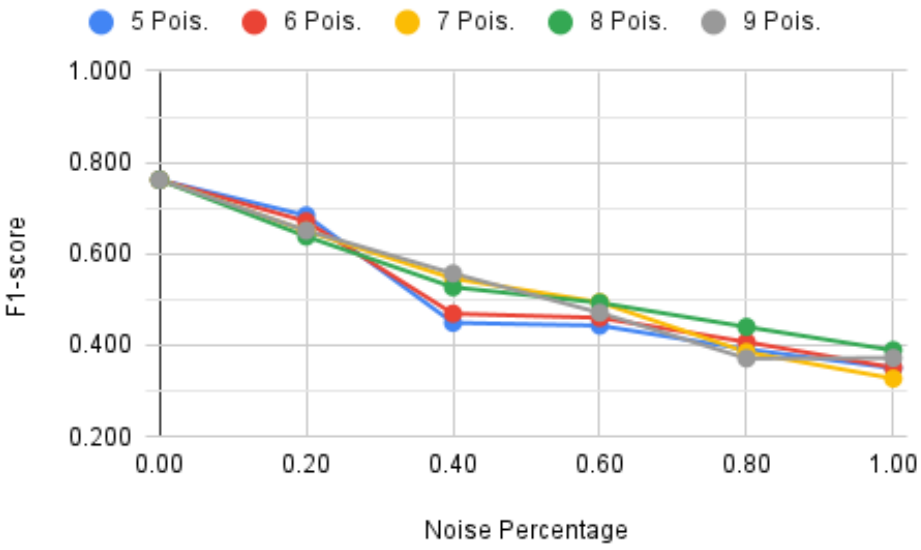


Figure B.25: Standard Model - Accuracy considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

B.6 Pruning - Majority of Poisoned Nodes With Threshold

Table B.26: Pruning - Weighted average F1-score considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.695	0.678	0.655	0.636	0.647
0.40	0.458	0.520	0.554	0.541	0.553
0.60	0.458	0.462	0.487	0.481	0.478
0.80	0.411	0.389	0.370	0.416	0.395
1.00	0.354	0.345	0.301	0.354	0.363
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	0
0.80	0	0	0	0	0
1.00	0	0	0	0	0
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

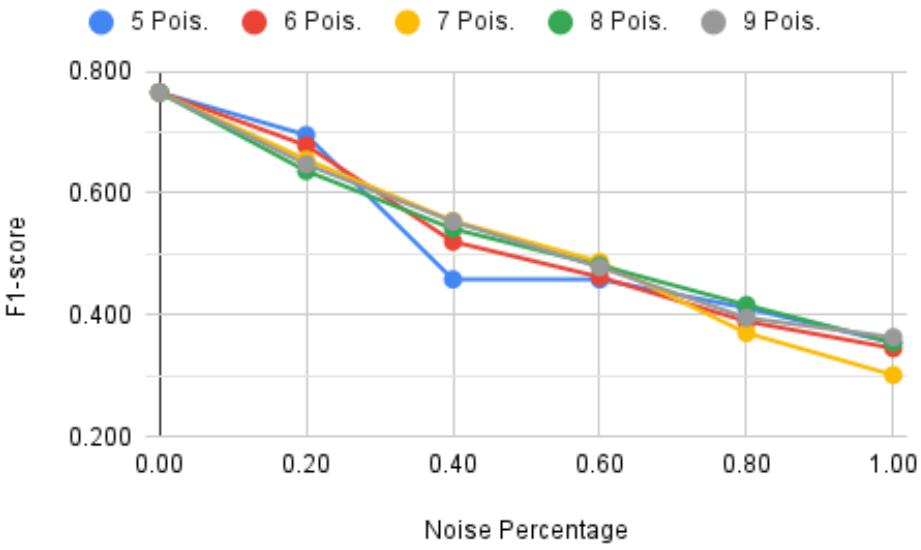


Figure B.26: Pruning - Weighted average F1-score considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table B.27: Pruning - Cross-entropy loss considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.878				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	1.184	1.315	1.352	1.413	1.400
0.40	1.889	1.803	1.643	1.746	1.639
0.60	1.870	1.857	1.856	1.822	1.856
0.80	2.009	2.020	2.136	1.994	2.060
1.00	2.169	2.157	2.318	2.162	2.130
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	0
0.80	0	0	0	0	0
1.00	0	0	0	0	0
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

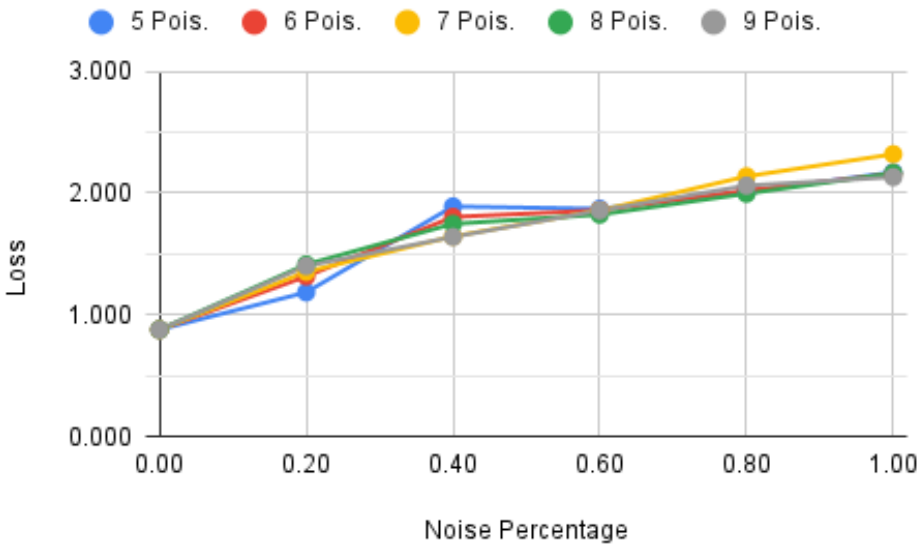


Figure B.27: Pruning - Cross-entropy loss considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table B.28: Pruning - Weighted average precision considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.770				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.714	0.703	0.693	0.677	0.683
0.40	0.609	0.618	0.638	0.638	0.636
0.60	0.598	0.597	0.604	0.595	0.603
0.80	0.559	0.571	0.556	0.568	0.568
1.00	0.530	0.546	0.523	0.540	0.549
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	0
0.80	0	0	0	0	0
1.00	0	0	0	0	0
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

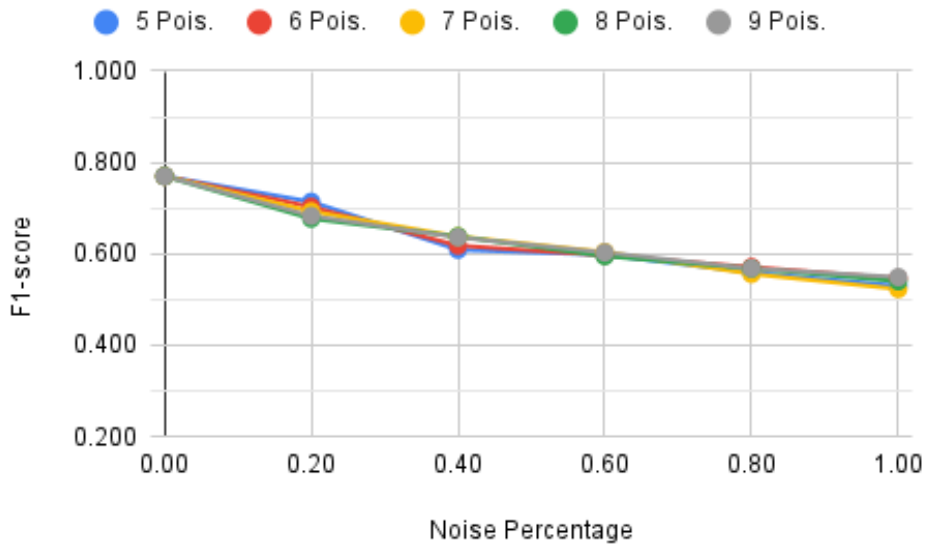


Figure B.28: Pruning - Weighted average precision considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table B.29: Pruning - Weighted average recall considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.698	0.678	0.658	0.639	0.649
0.40	0.455	0.521	0.557	0.539	0.555
0.60	0.461	0.466	0.488	0.485	0.483
0.80	0.414	0.402	0.375	0.418	0.397
1.00	0.364	0.362	0.320	0.360	0.373
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	0
0.80	0	0	0	0	0
1.00	0	0	0	0	0
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

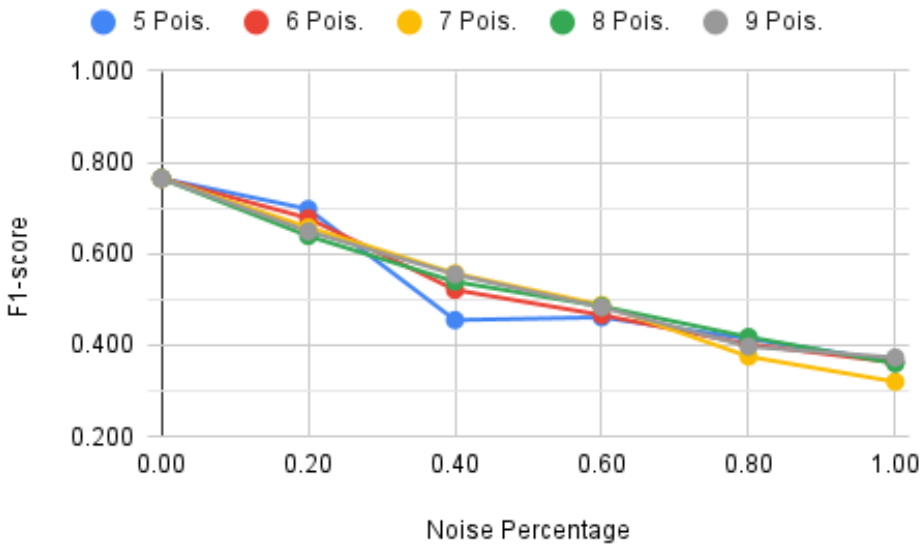


Figure B.29: Pruning - Weighted average recall considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table B.30: Pruning - Accuracy considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.698	0.678	0.658	0.639	0.649
0.40	0.455	0.521	0.557	0.539	0.555
0.60	0.461	0.466	0.488	0.485	0.483
0.80	0.414	0.402	0.375	0.418	0.397
1.00	0.364	0.362	0.320	0.360	0.373
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	0
0.80	0	0	0	0	0
1.00	0	0	0	0	0
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

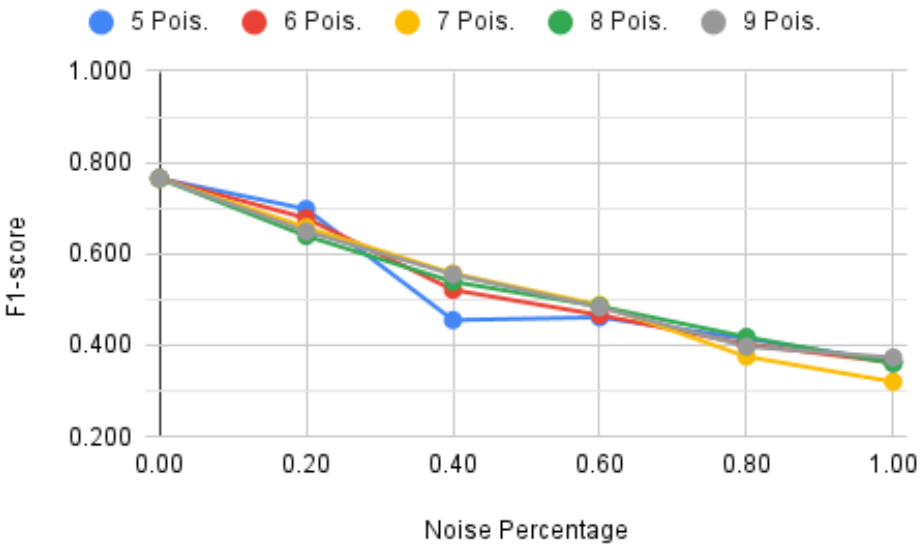


Figure B.30: Pruning - Accuracy considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

B.7 LiteRT - Majority of Poisoned Nodes With Threshold

Table B.31: LiteRT - Weighted average F1-score considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.695	0.678	0.655	0.636	0.647
0.40	0.458	0.520	0.554	0.541	0.553
0.60	0.458	0.462	0.487	0.481	0.478
0.80	0.411	0.389	0.370	0.416	0.395
1.00	0.354	0.345	0.301	0.354	0.363
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	0
0.80	0	0	0	0	0
1.00	0	0	0	0	0
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

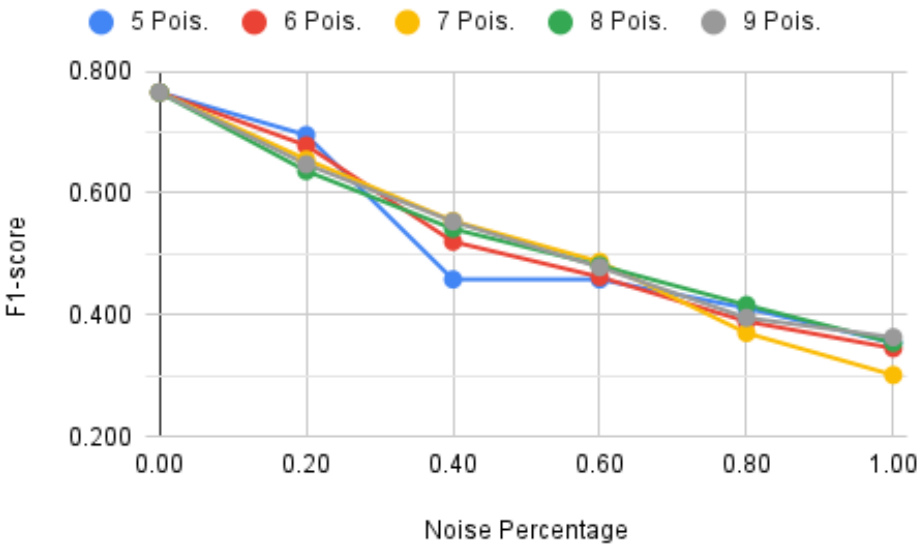


Figure B.31: LiteRT - Weighted average F1-score considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table B.32: LiteRT - Cross-entropy loss considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.878				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	1.184	1.315	1.352	1.413	1.400
0.40	1.889	1.803	1.643	1.746	1.639
0.60	1.870	1.857	1.856	1.822	1.856
0.80	2.009	2.020	2.136	1.994	2.060
1.00	2.169	2.157	2.318	2.162	2.130
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	0
0.80	0	0	0	0	0
1.00	0	0	0	0	0
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

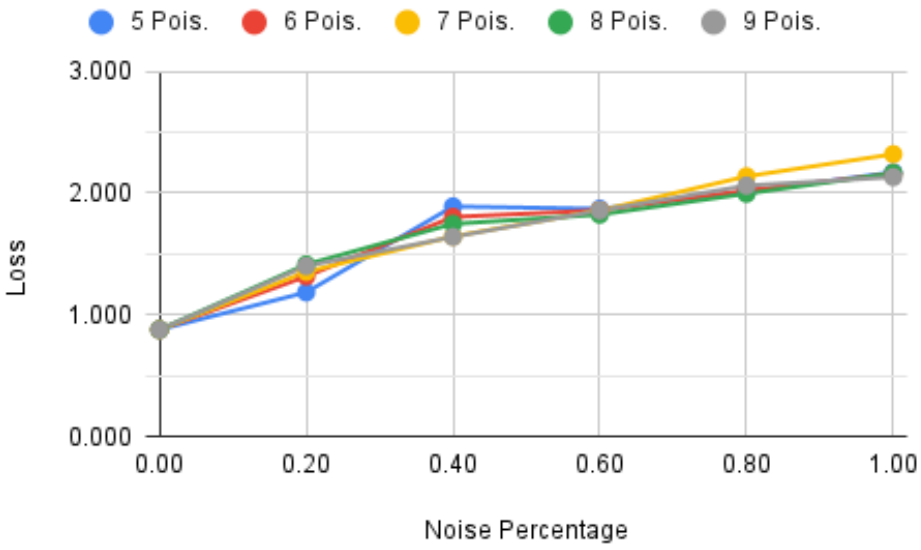


Figure B.32: LiteRT - Cross-entropy loss considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table B.33: LiteRT - Weighted average precision considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.770				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.714	0.703	0.693	0.677	0.683
0.40	0.609	0.618	0.638	0.638	0.636
0.60	0.598	0.597	0.604	0.595	0.603
0.80	0.559	0.571	0.556	0.568	0.568
1.00	0.530	0.546	0.523	0.540	0.549
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	0
0.80	0	0	0	0	0
1.00	0	0	0	0	0
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

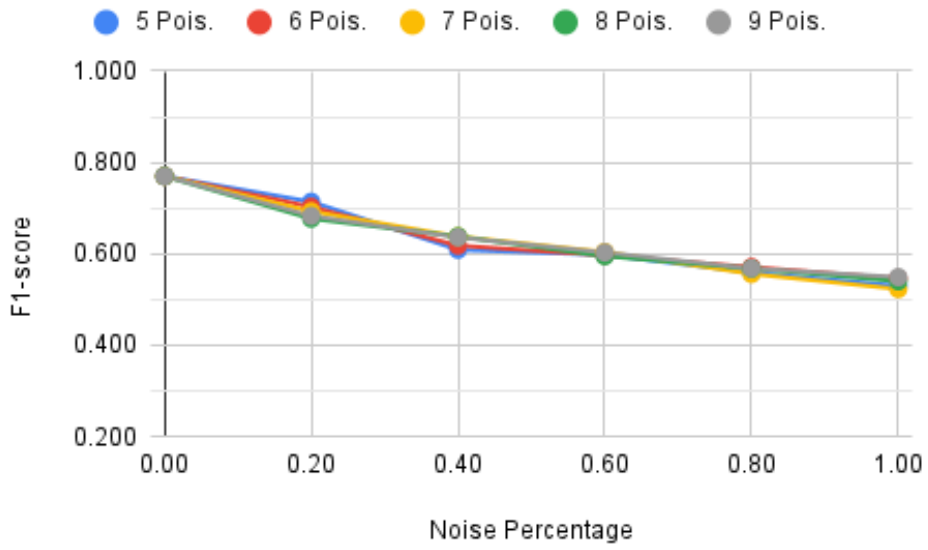


Figure B.33: LiteRT - Weighted average precision considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table B.34: LiteRT - Weighted average recall considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.698	0.678	0.658	0.639	0.649
0.40	0.455	0.521	0.557	0.539	0.555
0.60	0.461	0.466	0.488	0.485	0.483
0.80	0.414	0.402	0.375	0.418	0.397
1.00	0.364	0.362	0.320	0.360	0.373
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	0
0.80	0	0	0	0	0
1.00	0	0	0	0	0
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

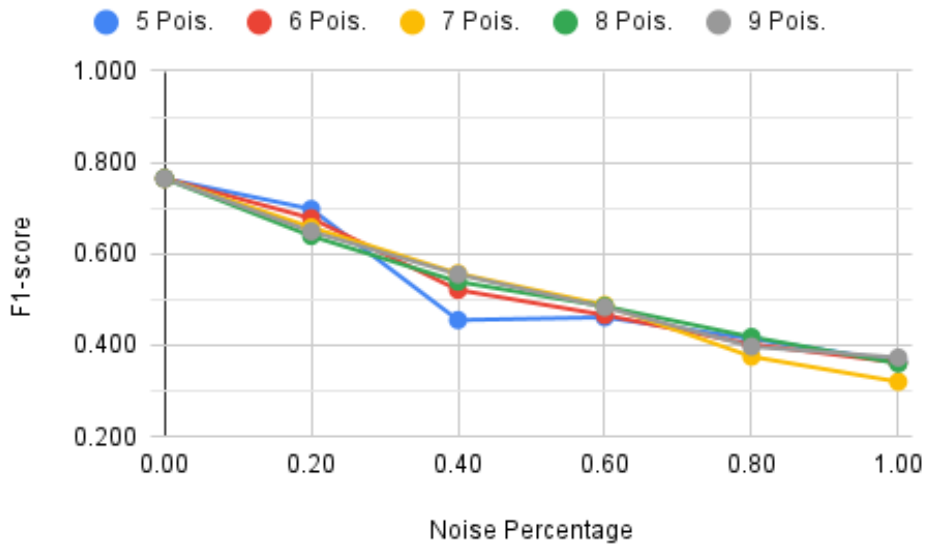


Figure B.34: LiteRT - Weighted average recall considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table B.35: LiteRT - Accuracy considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.765				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.698	0.678	0.658	0.639	0.649
0.40	0.455	0.521	0.557	0.539	0.555
0.60	0.461	0.466	0.488	0.485	0.483
0.80	0.414	0.402	0.375	0.418	0.397
1.00	0.364	0.362	0.320	0.360	0.373
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	0
0.80	0	0	0	0	0
1.00	0	0	0	0	0
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

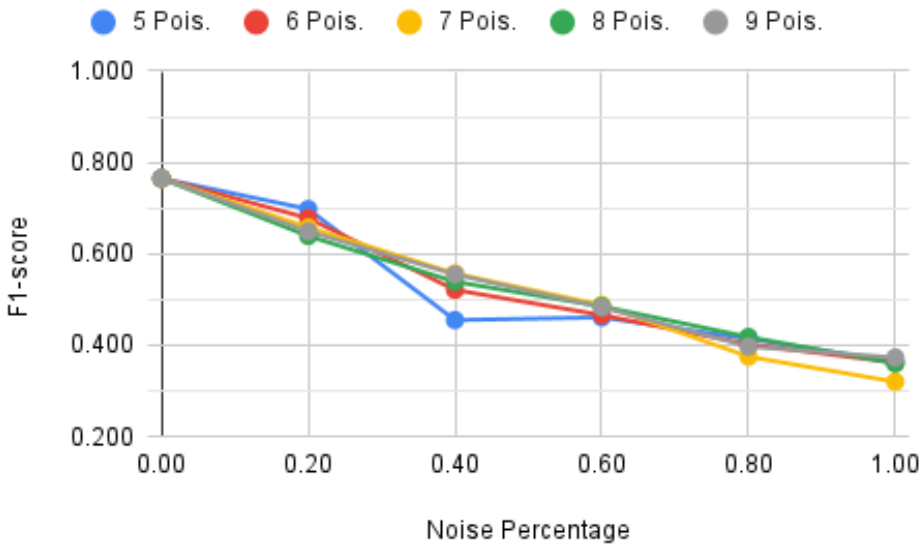


Figure B.35: LiteRT - Accuracy considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

B.8 Quantization - Majority of Poisoned Nodes With Threshold

Table B.36: Quantization - Weighted average F1-score considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.751				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.684	0.645	0.607	0.601	0.627
0.40	0.414	0.491	0.493	0.525	0.540
0.60	0.428	0.434	0.469	0.482	0.499
0.80	0.389	0.384	0.315	0.424	0.387
1.00	0.338	0.358	0.262	0.346	0.370
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	0
0.80	0	0	0	0	0
1.00	0	0	0	0	0
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

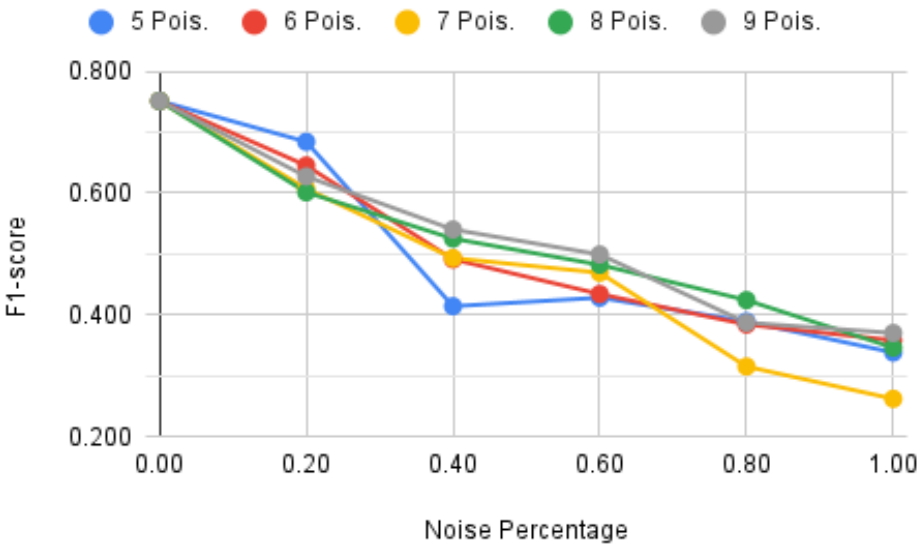


Figure B.36: Quantization - Weighted average F1-score considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table B.37: Quantization - Cross-entropy loss considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.902				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	1.297	1.377	1.456	1.482	1.349
0.40	1.995	1.845	1.806	1.776	1.657
0.60	1.902	1.937	1.938	1.769	1.830
0.80	2.073	2.057	2.283	1.959	2.133
1.00	2.239	2.167	2.478	2.212	2.112
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	0
0.80	0	0	0	0	0
1.00	0	0	0	0	0
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

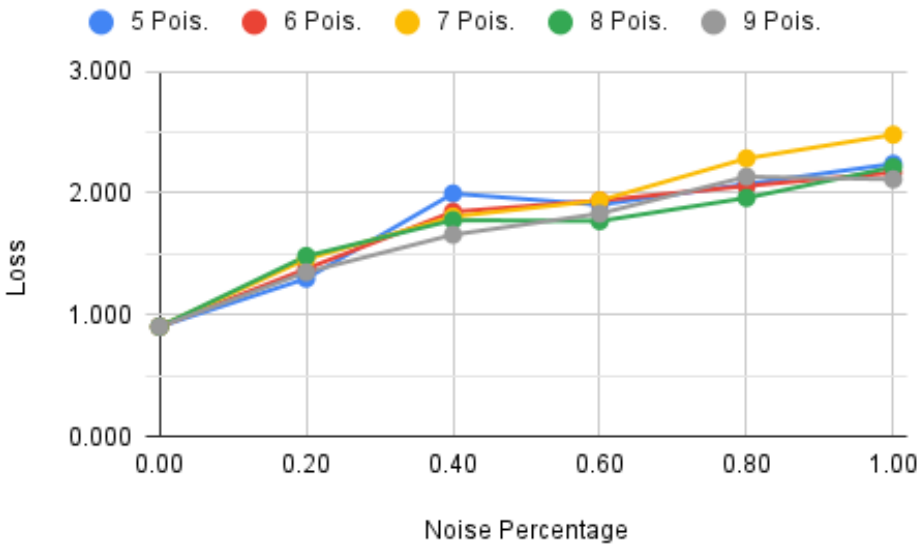


Figure B.37: Quantization - Cross-entropy loss considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table B.38: Quantization - Weighted average precision considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.758				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.706	0.684	0.669	0.668	0.677
0.40	0.595	0.613	0.628	0.631	0.636
0.60	0.585	0.590	0.591	0.599	0.597
0.80	0.547	0.563	0.545	0.573	0.554
1.00	0.518	0.538	0.521	0.545	0.540
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	0
0.80	0	0	0	0	0
1.00	0	0	0	0	0
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

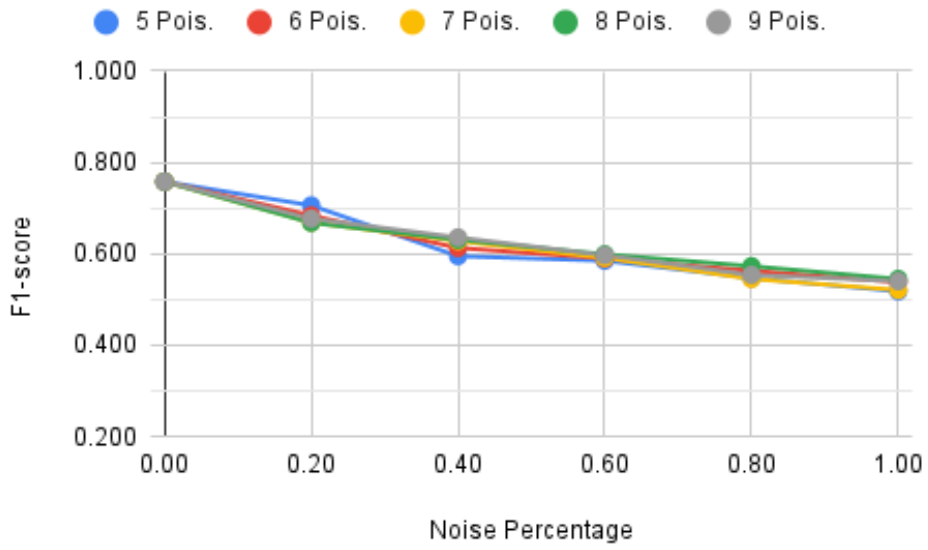


Figure B.38: Quantization - Weighted average precision considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table B.39: Quantization - Weighted average recall considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.752				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.687	0.647	0.611	0.600	0.629
0.40	0.406	0.490	0.490	0.522	0.539
0.60	0.430	0.444	0.469	0.485	0.501
0.80	0.393	0.396	0.320	0.430	0.391
1.00	0.346	0.365	0.278	0.349	0.381
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	0
0.80	0	0	0	0	0
1.00	0	0	0	0	0
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

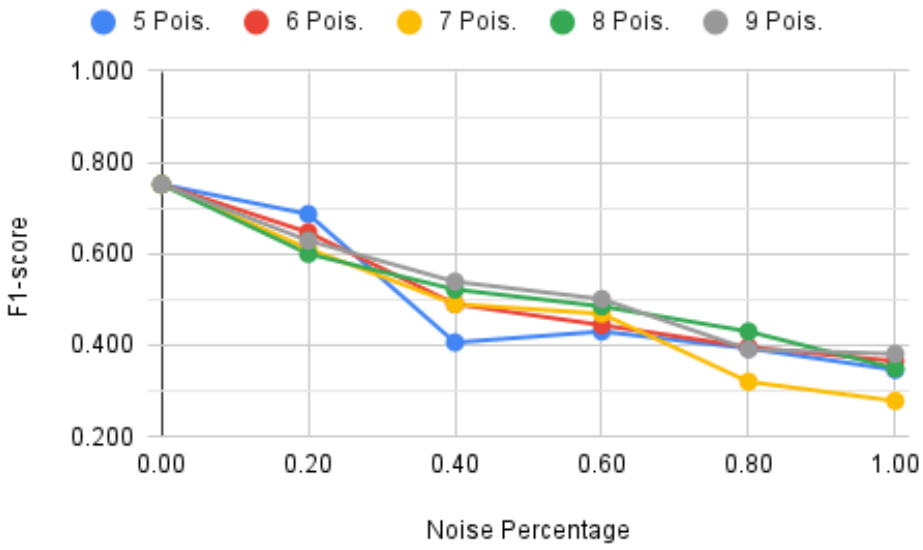


Figure B.39: Quantization - Weighted average recall considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Table B.40: Quantization - Accuracy considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

9 Nodes Without Poison	0.752				
Noise in Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0.687	0.647	0.611	0.600	0.629
0.40	0.406	0.490	0.490	0.522	0.539
0.60	0.430	0.444	0.469	0.485	0.501
0.80	0.393	0.396	0.320	0.430	0.391
1.00	0.346	0.365	0.278	0.349	0.381
Discovered Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	0
0.40	0	0	0	0	0
0.60	0	0	0	0	0
0.80	0	0	0	0	0
1.00	0	0	0	0	0
Mismatched Poisoned Nodes	5 Pois.	6 Pois.	7 Pois.	8 Pois.	9 Pois.
0.20	0	0	0	0	-
0.40	4	3	2	1	-
0.60	4	3	2	1	-
0.80	4	3	2	1	-
1.00	4	3	2	1	-

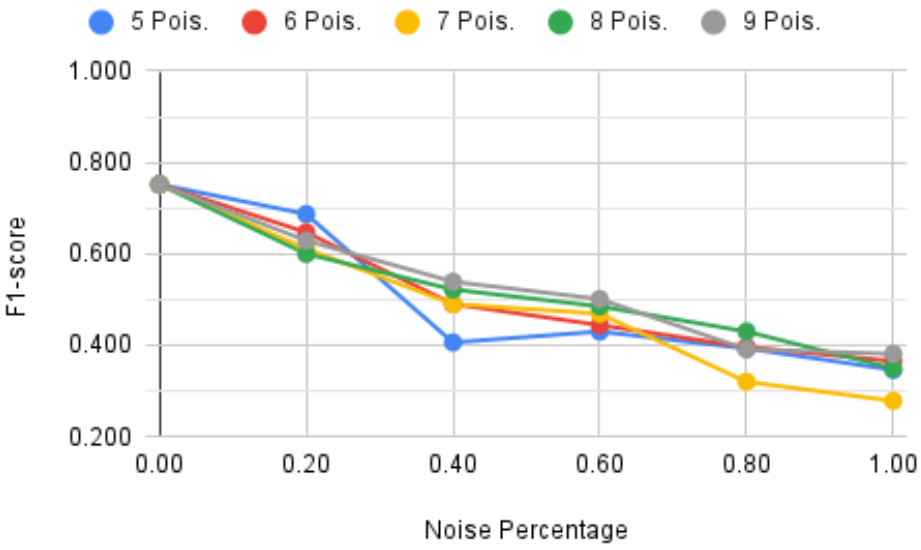


Figure B.40: Quantization - Accuracy considering a majority of poisoned nodes and setting a minimum threshold of 70% F1-score.

Appendix C - Directory of Results and Contextual Alignment with Document Sections

This appendix introduces the results directory for all experiments discussed in Chapter 6, Appendix A, and Appendix B. The results directory can be accessed through the links provided in references [158] and [159]. These links provide access to multiple servers, where the directory of results has been replicated to ensure availability in the event of a server failure. In addition to the results, the link also contains files that support the code presented in [137].

The root folder of the directory contains two folders. The first, titled “Supporting Files,” includes the base model used in the experiments, as well as the Excel file used to generate the tables and figures presented in Chapter 6, Appendix A, and Appendix B. The second folder, “Runs,” stores the tested models and their corresponding results.

The “Runs” folder is organized into four folders: “1 - Noise,” “2 - Unbalanced,” “3 - Minority Poisoned,” and “4 - Majority Poisoned.” Folder "1 - Noise" contains mainly data related to the results of Section 6.2, Folder "2 - Unbalance" pertains to Section 6.3, Folder "3 - Minority Poisoned" covers the results of Subsection 6.4.1, and Folder "4 - Majority Poisone" includes all the results presented in Appendix B.

Within these folders, the tests are named using the following variables: the number of nodes that were poisoned, the percentage of noise added to the data, followed by the level of unbalanced data. For tests where applicable, the name also indicates the number of poisoned nodes and the corresponding percentage of noise added to those nodes.

The presented results follow the established structure and names defined in [137]. For naming data noise, two digits are used to represent it, where “00” means 0.00 noise, “02” means 0.20 noise, and so on. Regarding unbalanced data, the levels noted in the names are one level lower than what is reported here. Therefore, “0” unbalance in the name corresponds to “1” in this report, and so forth. An example of a folder name is "9nodes_00noise_0unbalance_1poisoned_02percentage," indicating the results for 9 nodes with 0.00 noise, unbalance level 1, and 1 poisoned node with 0.20 relative noise percentage over the data.

Inside each folder, there is a "models" folder containing all the models obtained during the tests, information about the variables used in that experiment, the weights of

the standard and pruning models, as well as details on how the dataset was divided among the nodes in the "run_information" folder. The “results” folder holds the actual experimental results.

The “results” folder contains two folders: “savedModel” and “tflite.” The "savedModel" folder shows the results of the standard and pruned models, while the “tflite” folder presents the results from models converted using the LiteRT tool, including the quantized versions.

In each of the folders within the “results” folder, you’ll find results for each node of the DFL model, in addition to results for the global model. For the global model, there are two models from each experiment. The model in the “global_model_all” folder displays results without the 70% F1-score threshold, while the “global_model” folder shows results when the threshold is applied, as mentioned in Subsections 6.4.2 and 6.4.3. The structure of the “models” folder mirrors that of the “results” folder, but contains the models instead of the results.

In all folders containing results for nodes or the global models, two confusion matrices are presented as figures: one normalized and one unnormalized. Additionally, each folder includes a classification report detailing F1-score, cross-validation, precision, recall, and accuracy for each class, along with macro and weighted averages. There is also a file that reports the overall loss of the model on the test data. The results for the global model considering the 70% threshold also indicate how each node “voted” with its test data relative to the other nodes, noting whether they passed the 50% approval rate test.

The “run_information” folder contains files detailing how the dataset was divided among the various nodes for training, validation, and testing. Additionally, it includes the weights for the standard and pruning models, along with the initial variables used to execute the code detailed in [137].

Instituto Superior de Engenharia
Politécnico de Coimbra